

AI 상담 챗봇 현장 백서

그렇듯하게 틀리는 것들

여섯 직무가 6주 동안 밟은 지뢰와, 그 뒤에 세운 원칙 160개

LLM과 RAG로 제품을 만들기 시작한 사람, 그리고 그 옆에서 일하는 모든 직무에게

2026년 10월

머리말

이 책은 무엇인가

작은 팀이 AI 세무 상담 챗봇 하나를 만들었다. 병의원 사장님이 세무 질문을 하면, 언어 모델(LLM)이 근거가 되는 지식을 찾아(RAG) 답한다. 세무사는 그 답을 검수하고, 검수 결과는 다시 지식으로 쌓인다. 판정할 수 있는 질문은 규칙 엔진이 먼저 판정하고, 나머지는 선례와 해석으로 자문한다. 모델은 국내 LLM API 하나뿐이고, 데이터베이스는 클라우드 Postgres 하나다. 화면은 사장님·세무사·관리자 셋으로 나뉜다.

개발하는 동안 매일 기록을 남겼다. 그 가운데 2026년 8월 말부터 9월 말까지 6주의 기록에서 **다른 AI 제품을 만들 때도 통하는 교훈 186개**를 뽑았다. 이를 여섯 직무로 나눠 **원칙 160개**로 다시 묶었다. 이 책은 그 결과다. 이 제품을 몰라도 읽을 수 있도록, 사례에 나오는 내부 이름은 풀어 썼다.

왜 「그렇듯하게 틀리는 것들」인가

자매편인 게임 개발 백서의 제목은 「조용히 틀리는 것들」이었다. 화면은 멀쩡하고 테스트는 통과하는데 실제로는 틀린 것들을 다뤘다. LLM 제품에서는 한 겹이 더 있다. **틀린 것이 조용하기만 한 게 아니라 그렇듯하다.**

- 모델의 답은 언제나 문장이 매끄럽다. 근거 없는 판정도, 지어낸 수치도 매끄럽다.
- 채점기 역시 모델이라 일반론에 관대하다. 회차마다 몇 점씩 흔들린다.
- 측정 함수는 실패를 삼키고 기본값을 돌려준다. 레이트리밋은 "미분류"로, 타임아웃은 "인용 0"으로 보인다. 그래서 숫자가 그렇듯하게 나빠진다.
- 사람의 기억과 해석도 그렇다. 정황이 들어맞는 기억, 두 변수가 늘 같이 움직이는 관측, 이름이 기술을 가리키는 로드맵 항목이 의심받지 않는다.

이 책의 원칙은 대부분 하나의 질문으로 모인다. 「**이것이 그렇듯하게 틀렸다면, 누가 언제 알아챌까?**」

누구를 위한 책인가

- LLM·RAG 제품을 처음 만드는 팀 — 데모는 잘 되는데 운영에서 무엇이 어긋나는지 미리 보고 싶은 사람.
- AI 엔지니어 옆에서 일하는 백엔드·프론트엔드 개발자 — 모델이 끼면 자기 직무의 무엇이 달라지는지 알고 싶은 사람.
- 도메인 전문가와 함께 제품을 만드는 PO — 전문가의 판단이 들어갈 자리를 어떻게 만들고 지킬지 고민하는 사람.
- 제품 위에서 논문을 쓰는 연구자 — 실험이 제품을 오염시키지 않게, 결과를 정직하게 보고하고 싶은 사람.

읽는 법

- **1장 「모든 직무 공통」**을 먼저 읽는다. 측정, 채점, 기록, 프로덕션 쓰기에 관한 원칙이다.
- 그다음 **자기 직무의 장**을 읽는다. 장 첫머리의 「이 장의 원칙」 목록만 훑어도 요지가 잡힌다.

- 마지막으로 **이웃 직무의 장**을 읽는다. 사고는 대개 두 직무의 경계에서 났다. 정책이 프롬프트로 옮겨질 때, 지식의 모양이 검색을 바꿀 때, 권한을 화면이 대신 판단할 때, 실험 도구가 제품 경로에 섞일 때가 그랬다.
- 원칙마다 실제로 겪은 **사례**가 붙어 있다. 무슨 일이 있었고, 왜 아무도 몰랐고, 어떻게 잡았는지의 순서다. 장 끝의 **점검표**는 작업 전후에 스스로 묻는 질문이다.

여섯 직무

직무	말는 것	장
프로덕트 오너	도메인 정책, 운영 모델(검수·기여·정산), 우선순위와 범위, 외부 대응	2장
AI 엔지니어	프롬프트, 분류·추출·판정, 가드, 모델에게 맡길 몫의 경계	3장
지식 엔지니어	지식의 단위와 구조, 적재·합성·재구조화, 검색과 융합, 외부 코퍼스 수집	4장
백엔드·인프라 엔지니어	스키마와 권한, 배치, 외부 API 운영, 배포와 인프라	5장
프론트엔드 엔지니어	세 사용자의 화면, 실시간 구독, 시각화	6장
리서치	제품 위의 실험, 채점, 논문 투고	7장

작은 팀에서는 한 사람이 둘 이상의 직무를 겸한다. 장은 사람이 아니라 일의 종류로 나뉘었다. 사례 속 검수 코멘트와 평가 가운데 평가자 계정 이름으로 들어간 것은 감사 모드 LLM의 산출물이다. 사람인 세무사가 개입한 자리는 승인과 원문·인용 대조뿐이다.

차례

1장 모든 직무 공통	6
1.1 측정이 스스로를 속이는 자리	7
1.2 지표와 채점기를 믿기 전에	10
1.3 숫자에서 원문으로 내려간다	14
1.4 그럴듯한 기억·문서·지시를 믿기 전에	16
1.5 데이터는 누가 만들었고, 누가 읽는가	20
1.6 프로덕션에 쓰기 전에	22
1.7 일하는 방식	25
2장 프로덕트 오너	28
2.1 질문을 바로 세운다	29
2.2 무엇을 먼저, 무엇을 지금 안 할지	31
2.3 실험의 끝을 미리 긋는다	33
2.4 운영 규칙은 단순하게, 강제는 아래층에서	35
2.5 사람이 확정하는 자리	37
2.6 누가 무엇을 볼 수 있나	39
3장 AI 엔지니어	42
3.1 말로 부탁한 제약은 구조로 옮긴다	43
3.2 모델에게 맡길 것, 코드가 할 것	45
3.3 실패를 판정으로 둔갑시키지 않는다	47
3.4 프롬프트의 글자는 전부 지시다	48
3.5 모델을 탓하기 전에 과제를 의심한다	51
3.6 가드와 계측은 범위를 확인하고 붙인다	53
4장 지식 엔지니어	56
4.1 지식의 단위와 실제 모양	57
4.2 임계값은 쓰임에 묶인다	58
4.3 커버리지는 팔때기로 재고 계기판으로 쓴다	60
4.4 층과 융합 — 성격이 다른 지식을 한 답에 모으기	61
4.5 재생성 파이프라인과 사람의 손	62
4.6 새 소스를 들일 때 — 수집 경로를 실측하고 한 번에 하나씩	64
5장 백엔드·인프라 엔지니어	67
5.1 규칙은 DB가 강제한다	68

5.2 데이터를 바꾸는 경로와 상태값	70
5.3 오류 없이 비는 SQL — 마이그레이션과 상태 전이	72
5.4 외부 LLM API를 운영한다	73
5.5 배포 — 둘 이상의 자리가 따로 성공한다	75
5.6 인프라 — 수요, 비용, 이전	77
5.7 장시간 작업의 환경 — 밤, 절전, 재시작	78
6장 프론트엔드 엔지니어	81
6.1 실시간 구독 — 순서와 신호	82
6.2 빈칸과 폴백	83
6.3 화면이 하는 말	84
6.4 프론트와 백엔드의 계약	85
6.5 실브라우저에서만 보이는 것	86
7장 리서치	89
7.1 데이터를 누가 만들었나	90
7.2 채점기와 판독기를 믿어도 되나	90
7.3 사람 대조 표본을 어떻게 뽑고 보내나	91
7.4 벤치는 무엇을 재고 있나	93
7.5 투고 절차는 무엇을 되돌릴 수 없게 하나	93
7.6 연구 세션을 어떻게 운영하나	94

1장

모든 직무 공통

검증·측정·일하는 방식 — 직무와 상관없이 모두 읽는 장

이 책의 사례는 한 AI 세무 상담 챗봇에서 나왔다. 사장님이 세무 질문을 하면 LLM이 근거 지식을 찾아 답한다. 답하기 전에 관련 지식을 검색해 프롬프트에 붙이는 이 방식을 RAG(검색 증강 생성)라고 한다. 세무사는 그 답을 검수하고, 검수 결과는 다시 지식으로 돌아간다. 계정과목 인정 여부 같은 판정은 결정적인 규칙 엔진이 따로 내고, LLM은 판정과 근거를 문장으로 풀어 쓴다. 전체를 국내 LLM API 하나로 만들었다. 검수 기록은 세목(접대비·복리후생비 같은 세금 항목)별로 나누고 문장으로 응축해 「문장 단위로 다시 쓴 지식 사전」을 만들었다. 이 사전을 다시 짜는 배치 작업 한 바퀴를 이 장에서는 「회차」라고 부른다. 개발은 대표와 AI 코딩 에이전트가 세션 단위로 나눠 했다. 이 장은 6주(8월 말~10월 초) 기록 중 모든 직무에 통하는 교훈을 모았다. LLM 제품에서 틀린 것은 대개 틀려 보이지 않는다. 답변은 언제나 문법이 맞고 숫자까지 들어 있다. 채점기는 일반론에 관대하다. 풀백은 실패를 그럴듯한 판정으로 바꿔 내놓는다. 기억·앞 세션의 결론·주석·필드 이름도 검증된 적 없이 검증된 것처럼 읽힌다. 그래서 이 장의 원칙은 모두 한 질문으로 모인다. 「**이것이 그럴듯하게 틀렸다면, 누가 언제 알아챌까?**」 교훈 85개를 원칙 40개, 일곱 갈래로 묶었다.

이 장의 원칙

1. 풀백이 있는 함수로 쉼 때는 풀백이 삼킨 것부터 센다
2. 모든 LLM 호출에 명시적 상한을 걸고, 한 곳이 아니라 전부를 감사한다
3. 재시도는 한 층에서만 하고, 측정은 제품과 같은 동시성으로 한다
4. 요약보다 원시값을 먼저 남기고, 「0건」과 「당지 않음」을 다른 값으로 적는다
5. 측정 전에 그 입력이 바꾼 코드를 실제로 지나는지 확인한다
6. 시험 도구를 먼저 시험한다 — 고치기 전 실패를 보고, 몇 개를 돌았는지 찍는다
7. 성공은 목적 지표로 판정하고, 지표가 스스로 속는 실패를 따로 막는다
8. 유지·후퇴 기준은 실험 전에 숫자로 선언하고 그대로 따른다
9. 1회 측정으로 효과를 말하지 않는다 — 조건당 세 번, 같은 축인지 먼저
10. 빠르게 하는 변경도 고정 표본으로 품질을 재고, 비싼 측정 전에 앞 단계를 굳힌다
11. LLM 채점은 같은 회차 안의 순위로 읽고, 봉인한 사람 대조로 방향을 짚는다
12. 표본의 모양부터 본다 — 다수 찍기 값, 0건, 작은 표본
13. 임계값은 실측 편차와 전수 판독 위에 두고, 자동 가드는 원인에 건다
14. 평균 다음엔 건 단위로 겹쳐 보고, 막히면 원문을 읽는다
15. 이상한 숫자는 그 숫자를 만든 행을 직접 열어 판정한다
16. 함께 움직이는 두 변수로 원인을 하나로 단정하지 않는다
17. 이상이 보이면 화면과 모델보다 그 앞단의 데이터와 엔진을 거슬러 본다

18. 답하기 전에 그 질문을 왜 하는지, 전제가 성립하는지부터 본다
19. 기억과 전언보다 기록·실측·공식 원천을 다시 연다
20. 비싼 작업의 근거가 된 이전 결론은 원문 전수로 다시 확인한다
21. 「이미 있다」 「다른 곳이 잡는다」 는 문장은 코드로 확인한다
22. 이름을 믿지 말고 값과 카탈로그를 연다
23. 원칙과 문구는 조건과 함께 적고, 전제가 바뀌면 다시 읽는다
24. 프롬프트로 한 부탁은 지켜졌다고 믿지 않는다 — 형식과 코드로 강제한다
25. 데이터 출처는 계정·화면 이름이 아니라 생산 경로로 적는다
26. 신원과 권한은 화면이 아니라 서버에서 세우고, 역할별로 실측한다
27. 사람이 손댄 것은 모든 갱신 경로에서 보존한다
28. 값을 바꾸거나 데이터를 넣기 전에 그것을 읽고 더하는 자리를 전부 센다
29. 결정은 쓰기 없는 계측으로 내리고, 쓰기는 dry-run 기본·역등으로 만든다
30. 드라이런과 사전검토는 진짜 함수를 부르고, 쓰기 경계에서만 막는다
31. 지을 대상은 화면에서 소스 테이블까지 추적하고, 범위는 상태로 정한다
32. 운영 데이터를 바꾸는 일은 되돌릴 길과 뒷정리까지 한 묶음으로 짠다
33. 새 경로는 기본값에서 기존과 같게 붙이고, 이해하는 쪽부터 배포한다
34. 서버 이전은 「같은 결과」 로 판정하고 업그레이드와 섞지 않는다
35. 감시와 회수는 감시받는 작업과 같이 죽지 않는 곳에 둔다
36. 연구·채점과 제품 수정은 세션을 나누고, 경계는 구동 경로로 긋는다
37. 규칙과 구조는 건드리는 기존 로직이 적은 쪽으로 고르고, 미룰 때는 다시 열 조건을 숫자로 적는다
38. 판단 기준은 한 곳에 하나만 두고, 기계와 사람이 같은 글을 읽게 한다
39. 커밋되지 않는 작업일수록 끝날 때 기록과 「확인하지 못한 것」 을 남긴다
40. 이스케이프나 한글이 든 명령은 셀 인라인 대신 파일로 쓴다

1.1 측정이 스스로를 속이는 자리

LLM을 부르는 코드에는 실패를 숨기는 장치가 많다. 예외를 잡아 기본값을 돌려주는 폴백(fallback), SDK가 알아서 하는 재시도, 길게 잡힌 기본 타임아웃이다. 제품에서는 친절한 장치다. 그러나 측정에서는 실패를 판정으로 둔갑시킨다. 요청이 몰려 API가 거절하는 레이트 리밋(HTTP 429)이 「미분류」 라는 답으로, 타임아웃이 「인용하지 않음」 이라는 답으로 바뀐다. 결과는 언제나 그럴듯한 숫자다. 이 절은 측정 도구가 측정 대상보다 먼저 틀린 자리들이다.

폴백이 있는 함수로 잴 때는 폴백이 삼킨 것부터 센다

예외를 잡아 기본값을 돌려주는 함수로 재면 「예외 0건」 이 나온다. 셀 길이 없기 때문이다. 제품 함수는 결과와 실패 사유를 함께 돌려주게 만든다. 모델이 정말로 「미분류」 를 고른 경우에만 사유가 비어 있어야 한다. 모델이 도구를 부르지 않은 응답도 실패로 센다. 측정 스크립트는 환경 파일을 명시해 읽는다. 주 경로와 폴백이 같은 결과 칸을 채우는 곳에서는 둘을 따로 센다. 합계만 보면 폴백이 실패를 삼킨다.

사례 · 실패가 판정으로 바뀐 회차

무슨 일이 지식 문장을 세목에 배정하는 분류 함수는 예외가 나면 「미분류」를, 문장을 응축하는 합성 함수는 빈 목록을 돌려줬다. 분류에서는 레이트 리미트, 합성에서는 타임아웃이 그 폴백에 삼켜졌다. 그 위에서 「회차가 무너졌다(미분류 40/38/75%)」 「미분류 재시도는 효과가 없다(33 → 33 → 32%)」 「인용률이 떨어졌다」는 결론이 나왔다. 환경 파일을 읽지 않은 측정 스크립트에서는 키가 없다는 예외까지 삼켜져 「0초 만에 폴백」처럼 보였다. **왜 몰랐나** 예외를 셀 수 없는 함수로 썼으니 예외는 늘 0건이었다. 결과는 낮은 배정률, 낮은 인용률이라는 그럴듯한 숫자였다. **잡은 방법** 8위커 재현 로그에 `{'RateLimitError': 16}`이 찍혔다. 실패를 따로 세는 계측 사본으로 합성 3회를 다시 썼다. 타임아웃 뒤편을 빼자 회차 간 상대 폭이 7.9%에서 0.9%로 줄었다.

사례 · 다 찬 목차

무슨 일이 지식 사전의 빈 목차 칸을 모델이 채우는 단계가 있다. 프롬프트에 「이미 있는 것과 겹치는 건 내지 말 것」이 있었다. 모델은 1.8초 만에 후보 목록의 머리를 그대로 베였고, 11개 전부가 기존 레이블과 겹쳐 거부됐다. **왜 몰랐나** 오류 없이 코드가 대신 채우는 폴백으로 넘어갔다. 결과 목차는 30칸이 다 차 있었다. **잡은 방법** 모델 뒤편 코드 뒤편을 따로 센 계측이 「모델 0 / 코드 8」을 보여 줬다. 코드가 덮이지 않은 후보만 걸러 모델에게 주자 모델 뒤편 8, 거부 0이 됐다. → 1.4 프롬프트로 한 부탁

모든 LLM 호출에 명시적 상한을 걸고, 한 곳이 아니라 전부를 감사한다

SDK 기본 타임아웃은 생각보다 길고, 재시도까지 곱하면 더 길다. 상한은 문제가 난 한 곳이 아니라 호출 전부엔 건다. 폴백이 있는 자리라면 상한은 정확도를 깎지 않는다. 최악의 시간만 자른다. 긴 단계는 진행률을 남긴다. 진행률이 없으면 「정상 진행」과 「폭주」가 같은 화면으로 보인다.

사례 · 21분 무진전

무슨 일이 지식 사전을 다시 짜는 회차의 한 단계가 SDK 기본값(600초 × 재시도 2회)에 물려 21분 동안 아무 진전이 없었다. 배치로 바꿀 때 분류 호출에만 상한을 걸고 나머지는 두었다. 전날 배치 실험에서 본 1250초도 같은 값이었다. **왜 몰랐나** 서버는 정상이었고 에러 로그도 없었다. 그 단계는 배치마다 작업 상태를 갱신하지 않아 화면에서 구분할 수 없었다. **잡은 방법** 21분 무진전을 관찰한 뒤, 시간 상한과 재시도 1회를 박은 클라이언트를 열 곳에 넣었다.

재시도는 한 층에서만 하고, 측정은 제품과 같은 동시성으로 한다

재시도가 우리 호출 바깥(SDK 안)에서 일어나면 계측은 조용히 거짓말한다. 「240초 물렸다가 재시도로 3초 만에 성공」이 「243초 걸린 성공 1건」으로 접힌다. 재시도는 한 층에서만 하고 그 층에서 잤다. 로그가 없는 지연은 분포가 어디에 뭉치는지 보고 타임아웃·재시도 설정과 맞춰 본다. 동시성도 같다. 제품이 순차인 자리는 순차로, 병렬인 자리는 병렬로 잤다. 측정 둘이 같은 API 키를 동시에 쓰지 않게 한다.

사례 · 존재하지 않는 정상 호출

무슨 일이 합성 호출 소요의 p90(빠른 쪽부터 90% 지점)이 243.2초로 기록돼 있었다. 그 숫자 때문에 상한을 낮추지 못하고 있었다. 체감 상한도 240초가 아니라 480초였다. **왜 몰랐나** 재시도가 SDK 안에서 일어났고, 요약 분위수만 남아 있었다. **잡은 방법** 실패 1건이 정확히 480.7초, 즉 240×2 였다. SDK 재시도를 끄고 원시값을 남겨 한 회차 더 썼다. 진짜 p90은 4.87초, 최대 15초였다. 상한을 240초에서 60초로 내렸고, 잘린 정상 호출은 0건이었다.

사례 · 빨리 재러다 다른 것을 켜 측정

무슨 일이 속도를 내려고 8위커로 돌린 측정에서 429가 16% 낮고, 413건 전량 배정률이 27.4%·14.5%까지 내려갔다. 같은 조건을 제품처럼 순차로 4회 돌리면 51.3~53.3%, 상대 편차 3.8%, 호출 실패 0/600이었다. 다음 주에는 답변의 긴 꼬리 지연을 반복 루프 탓으로 봤다. 그러나 60초를 넘긴 35회 중 루프는 2회뿐이었고, 지연이 124~129초와 241초에 뭉쳐 있었다. 120초 타임아웃에 재시도 1회를 더한 모양이었다. **왜 몰랐나** 제품은 순차라 평시에는 레이트 리미트가 드러나지 않는다. SDK 재시도는 로그를 남기지 않고, 재시도가 바로 성공하니 오류도 없다. **잡은 방법** 제품 함수 그대로 순차 64회를 돌려 멈춤 0을 확인했다. 동시 조건의 멈춤률(약 6%)이 순차에서도 같다면 64회 내내 안 멈춤 확률은 약 2%다. 그래서 같은 키에 요청이 몰릴 때의 산물로 판정했다. → 5장

요약보다 원시값을 먼저 남기고, 「0건」과 「당지 않음」을 다른 값으로 적는다

저장할 때는 나중에 어떤 질문을 던질지 모른다. 분위수·개수 같은 요약은 그 자체로 그럴듯한 보고라서, 원시값이 없다는 사실이 다음 질문이 올 때까지 드러나지 않는다. LLM의 중간 산출물은 결정적이지 않아 회차가 끝나면 다시 만들 수 없다. 원시값부터 남긴다. 분모를 스키마에 박지 말고 모든 경우를 결과 종류로 구분해 남긴다. 모르는 값은 false로 뭉개지 말고 null을 허용한다. 「검색했는데 0건」과 「검색 단계에 당지 않음」은 다른 값이다.

사례 · 추정으로 남은 숫자

무슨 일이 합성 소요를 분위수로만 저장했더니 「호출 10%쯤이 벽시계 시간의 85%쯤을 먹는다」의 「5회쯤」이 추정치로 남았다. 확인하려면 회차를 하나 더 돌려야 했다. 근거 문서 분포를 개수만 남긴 탓에 「왜 그 문서 조각이 들어왔나」도 되짚을 수 없었다. 응답에 근거 목록이 없어 사전형 지식이 실제로 끼었는지 확인할 수 없는 공백도 있었다. **잡은 방법** 다음 회차부터 원시값을 남겼다. 검색 근거는 원시 목록과 개수를 둘 다 남기고, 0건은 {}, 미도달은 null로 갈랐다. 새 지식을 넣을 때는 「계측 먼저·정밀도 감시·한 번에 한 소스·라이선스·권위 등급·소급 가능」의 여섯 규칙을 정했다. 계측 없이 넣으면 품질이 왜 변했는지 영영 모른다. → 4장

측정 전에 그 입력이 바꾼 코드를 실제로 지나는지 확인한다

「변화 없음」은 「효과 없음」으로 읽힌다. 그러나 측정 질문이 바꾼 코드를 지나지 않아도 결과는 똑같이 「변화 없음」이다. 효과를 재기 전에 그 입력이 바꾼 경로를 실제로 타는지 로그나 응답 메타로 확인한다. 변화가 없으면 먼저 경로를 의심한다. 점검 도구도 같다. 배포 점검과 관리용 CLI는 서비스와 같은 설정 로딩 경로로 띄운다. 다른 설정을 읽는 점검의 통과는 서비스 상태를 말해 주지 않는다.

사례 · 무효 측정

무슨 일이 답변 작성에 늘 주입하는 세무 원칙 문서(이하 「규범」)를 v2로 고치고 효과를 기본 경로에서 켜다. 전후 8/8이 같았다. 그 질문은 「판정 대상이 아님」이라는 고정 안내문으로 끝나서 문장을 쓰는 LLM을 아예 타지 않았다. **잡은 방법** 경로를 확인하고 문장을 쓰는 자문 경로에서 다시 켜다. 조문 인용이 v2 3/8, v1 0/8로 효과가 있었다. 로컬에서 0 → 6/8이던 효과가 프로덕션에서는 약했다. 측정 경로도 실제 서비스 조건에 가깝게 잡아야 한다.

사례 · 다른 세계를 본 점검

무슨 일이 규범을 DB에서 읽도록 바꾼 첫 배포에서, 점검 로그가 「출처: md 파일」이라고 찍었다. 점검 CLI가 서비스 진입점을 거치지 않아 환경 파일을 읽지 않은 것이다. **왜 몰랐나** 런타임은 DB를 읽고 있어 서비스는 정상이었다. 점검만 다른 세계를 보고 있었다. **잡은 방법** 출처 표시가 기대와 다른 것을 보고 추적했다.

시험 도구를 먼저 시험한다 — 고치기 전 실패를 보고, 몇 개를 돌았는지 찍는다

헛통과는 실패가 아니라 통과로 숨는다. 시험 결과에는 실행한 항목 수를 같이 찍는다. 화면 확인은 본문이 로딩 상태를 벗어났는지로 한다. 검증기·측정기는 일부러 틀린 입력을 넣어 잡는지 먼저 보고 쓴다. 고치기 전에 시험이 실패하는 것부터 기록해 A/B를 남긴다. 경합 버그는 sleep으로 기다리지 말고 네트워크 응답을 붙잡아 창을 강제로 연다. 배포 직후 E2E(화면을 처음부터 끝까지 누르는 자동 시험)는 지금 화면이 새 빌드인지부터 확인하고, 아니면 아무것도 바꾸지 않고 멈춘다. 부분 실패가 나오면 앱을 고치기 전에 DB 상태로 진위를 가린다.

사례 · 헛통과 셋

무슨 일이 한 주에 헛통과가 셋 나왔다. 회귀 스크립트의 ONLY= 는 역할 필터인데 경로를 넣었더니 0개 화면을 돌고 통과했다. 사이드바 글자로 화면을 확인하니 「로딩 중…」 인 본문이 통과했다. 첫 화면 측정의 「글자가 3초 동안 안 바뀌면 완성」 은 준비 대기가 3초를 넘자 로딩 화면을 완성으로 썼다. **왜 몰랐나** 셋 다 초록불이었다. **잡은 방법** 기대보다 빠른 결과와 화면 수를 의심했다. 측정을 「로딩 표시가 사라지고 2초 불변」 으로 바꾸고 기준선도 같은 스크립트로 다시 썼다. 데이터 쪽 대화 궤적 검증기는 일부러 틀린 입력 5종을 전부 잡는 것을 먼저 보고 투입했다.

사례 · 대기 시간이 달아 버린 경합 창

무슨 일이 DB 변경을 브라우저로 밀어 주는 실시간 구독에서 새 메시지가 유실되는 버그를 고쳤다. 첫 재현 시험 「로그인 → 3초 대기 → 새 행 추가 → 확인」 은 고치기 전 코드에서도 통과했다. 3초 사이에 구독이 이미 붙어 경합 창이 닫혔기 때문이다. 정산 E2E에서는 3건 중 1건만 처리된 것처럼 보여 앱의 경합으로 읽혔는데, DB를 보니 스크립트가 게시 완료를 기다리지 않고 페이지를 옮긴 탓이었다. **잡은 방법** Playwright로 최초 적재 응답을 붙잡아 두고 그 사이 다른 세션이 행을 넣게 했다. 고치기 전 유실, 고친 뒤 2회 연속 보임을 남겼다. → 6장

1.2 지표와 채점기를 만들기 전에

LLM 제품의 품질 숫자는 대부분 다른 LLM이나 자동 규칙이 매긴다. 채점기는 일반론에 관대하고, 회차마다 흔들리고, 무엇을 놓치는지 말하지 않는다. 숫자가 원하던 방향으로 움직이면 의심은 더 줄어든다. 이 절은 숫자를 받아 들기 전에 할 일이다. 이 절의 「배정률」 은 지식 문장이 세목에 배정된 비율, 「커버리지」 「인용률」 은 지식 사전 문장이 원재료를 얼마나 덮고 인용했는가를 가리킨다.

성공은 목적 지표로 판정하고, 지표가 스스로 속는 실패를 따로 막는다

부산물 지표(생성 수, 채운 칸 수)로 성공을 판정하면 바람직한 결과를 실패로 찍거나, 실패를 성공으로 찍는다. 「이 실패가 일어나면 지표가 오르는가」 를 묻는다. 지표는 분해 가능한 계측과 함께 읽고, 분해할 수 있을 때만 좋아졌다·나빠졌다는 말한다. 분모 조정은 제외 기준이 결과와 독립적으로 정당할 때만 한다. 모델이 빠뜨린 것은 조용히 넘기지 않고 화면까지 올린다.

사례 · 좋아 보이는 실패

무슨 일이 목차 병합 단계가 같은 레이블 30개를 돌려주면 선택지가 하나뿐이라 배정률이 오히려 높게 나온다.

「기타」 문장을 커버리지에 더하면 검색에 안 잡히는 문장이 답을 덮는 것처럼 썬해진다. 커버리지 66.3%와 59.3%를 그대로 비교하면 개선으로 읽히지만, 차이는 유실된 원재료 조각으로 설명됐다. 반대 방향도 있었다. 자동 그룹화가 기존 묶음을 재사용하면 생성 수가 0이 되는데, 화면은 그것을 「실패했습니다」로 읽었다. **왜 몰랐나** 가드와 대시보드는 숫자가 오르면 안심한다. 처음 만들 때는 늘 새로 생성했으므로 생성 수가 곧 성공처럼 보였다.

잡은 방법 실패 모드를 하나씩 상상해 지표의 움직임을 따졌다. 중복 레이블을 걸어 내고, 「기타」는 인용에 더하지 않았다. 그룹화의 성공은 배정된 수로 판정하고, 빠뜨린 수를 화면에 올렸다. 미인용 건에 붙은 「문법적 오류」 태그는 인용된 46건에도 붙어 있어서 분모 제외를 보류했다.

유지·후퇴 기준은 실험 전에 숫자로 선언하고 그대로 따른다

LLM 개선 실험은 회차마다 「조금만 더 고치면」이 가능해 보인다. 기준이 없으면 개선 회차가 끝없이 이어진다. 시작 전에 「어디를 넘으면 유지, 어디 밑이면 물러선다」를 수치로 적는다. 물러설 때는 LLM이 못 하는 일을 결정적 로직으로 옮기고, 그 대가를 명시해 받아들인다. 기능을 다 만든 뒤 증명 단계로 넘어갈 때도 완성 기준을 문장으로 박는다. 그래야 계기판 숫자를 목표에서 내릴 수 있다.

사례 · 72%를 넘는 회차 0

무슨 일이 「질문을 받아 사실을 뽑고 되묻는 앞단 모델」이 「답할 정보가 충분한가」까지 판정할 수 있는지 재기로 했다. 시작 전에 기준을 문서로 적었다. 중간 vs 충분 구분이 72%(답변 길이만 보는 단순 규칙의 값)를 확실히 넘으면 유지, 세 가지를 다 해도 우연 수준이면 역할 축소. 7개 방식 × 446건에서 넘는 회차는 없었다(체크리스트 방식 50.8%, 오라클 68.6%, 다른 회사 LLM 블라인드 54.2%). **왜 몰랐나** 오라클 v2는 항목 정확도를 50.7%에서 73.2%로 올렸다. 다음 회차가 늘 가까워 보였다. **처리** 대표가 축소를 확정했다. 모델은 추출과 되묻기만 하고, 충분 판정은 LLM 없는 상한(턴당 질문 2개·최대 2턴)에 맡겼다. 「정보를 다 준 사용자도 1턴은 질문을 받는다」는 대가를 문서에 적었다. → 3장

사례 · 계기판과 목표

무슨 일이 지식 사전의 기능 1~19를 다 배포한 뒤에도 커버리지를 올리는 작업이 이어졌다. 자동 경로의 천장 63~65%에는 이미 닿아 있었다. **잡은 방법** 완성 기준을 한 줄로 적었다. 「세무사가 하루 쓰면 그만큼 좋아져 있고, 다음 회차가 그것을 지우지 않으며, 지표가 편차 안에서 재현된다.」 재현성·추적성·임팩트 세 조건으로 나눴고, 커버리지는 계기판으로 내렸다.

1회 측정으로 효과를 말하지 않는다 — 조건당 세 번, 같은 축인지 먼저

한 번 잰 숫자는 편차를 알려 주지 않는다. 회차 편차가 처방 효과보다 클 수 있다. 조건당 최소 세 번 재고 개별값을 같이 본다. 차이가 편차 안이면 개선이라고 말하지 않는다. 원하던 방향의 1회 측정은 「떨어지지 않았다」까지만 주장하고, 그 숫자로 임계값을 만지지 않는다. 비교하기 전에 두 수치가 같은 분모·같은 범위인지부터 확인한다.

사례 · 편차보다 작은 개선

무슨 일이 합성 인용률의 현행 조건을 세 번 잰더니 37/70/70%로 같았다. 지난 세션의 세목별 1회 수치(가족 인건비 22%)는 나쁜 회차를 한 번 본 것일 수 있었다. 처방의 상대 개선 3.5%는 편차 3.8%보다 작아서 「개선 아님」으로 기록했다.

사례 · 원하던 방향의 숫자

무슨 일이 비슷한 세목을 접는 회차에서 배정률이 64.4%에서 78.7%로 올랐다. 큰 묶음(대목) 첫 실증에서는 최대 묶음이 43%에서 32%로 내려갔다. 둘 다 원하던 방향이었다. **왜 몰랐나** 앞선 실험 결론은 「중복 제거는 커버리지를 안 올린다」였고, 배정률 이력은 73.1/70.5/73.8/64.4/78.7%로 원래 출렁였다. 최대 묶음 수치는 남은 묶 기준이라 이전 회차와 축이 달랐다. **잡은 방법** 기존 결론과 대조하고 두 수치의 정의를 다시 봤다. 전체 기준으로 다시 재니 최대 묶음은 28개 중 9개(32%)였다.

빠르게 하는 변경도 고정 표본으로 품질을 재고, 비싼 측정 전에 앞 단계를 굳힌다

성능 개선은 품질 회귀 측정과 한 묶음이다. 아낀 시간과 잃은 품질을 같은 표에 놓는다. 되돌린 함수에는 경고를 남겨 다시 붙이지 않게 한다. 측정 비용도 줄인다. 측정하려는 단계의 바로 앞 단계를 파일로 굳히고, 그 값이 프로덕션과 같은지 먼저 본다. 앞 단계의 편차가 측정하려는 신호를 덮기 때문이다. 새 실험 전에 이미 쌓인 데이터로 공짜 분석부터 한다.

사례 · 3분을 아끼려다 절반을 버린 구조

무슨 일이 분류를 20건씩 묶어 보내면서 「정확도는 건별과 같고 시간만 준다」고 판단했다. 고정 표본 100건으로 다시 재니 묶음 미분류 84%, 건별 45%였다. 모델이 앞쪽 몇 건만 읽고 나머지를 미분류로 흘렸다. **왜 몰랐나** 품질 표본이 5건뿐이었고(1건 불일치), 폴백 구조 때문에 「정확도는 같다」는 논리가 그럴듯했다. **잡은 방법** 커버리지 20%의 원인을 2×2 대조군으로 가르다가 잡았다.

사례 · 37분에서 2분으로

무슨 일이 목차 재구성 측정 한 번에 37분이 걸렸다. 고정 표본(시드 11)과 캐시한 후보로 바꾸자 1~2분이 됐다. 합성 편차를 잰 때는 분류 배정을 한 번 돌려 파일로 굳혔다(304/413, 프로덕션 302/413과 재현). 「문서 조각이 늘어 인용률이 떨어졌다」는 가설은 기존 작업 두 개의 통계를 조각 수로 묶어 LLM 호출 0회로 기각했다. 두 방향이 서로 반대였다.

LLM 채점은 같은 회차 안의 순위로 읽고, 봉인한 사람 대조로 방향을 잔다

같은 답에도 LLM 채점의 절대치는 회차마다 움직인다. 비교 대상은 모두 같은 회차에서 채점하고 순위만 믿는다. 기준선을 남길 때는 그 회차의 지식 상태와 채점기 설정을 같이 적는다. 채점기의 오류 방향은 미리 뽑아 봉인한 사람 대조 표본으로 잔다. 과대 판정이 없고 누락만 있으면 숫자를 하한으로 쓴다. 사후에 정한 제외 규칙에는 민감도를 붙인다. 패턴 스캔으로 찾은 개수도 「최소 n건」으로 쓰고, 스캔이 구조적으로 놓치는 출처를 함께 적는다.

사례 · 같은 검색 결과, 다른 점수

무슨 일이 같은 검색 결과를 두 회차 채점했다. 4방향 비교는 갈래 전반이 +4~6pp 이동했고, 5방향 비교는 기본 채점 $\pm 1pp$, 엄격 채점 $\pm 2\sim 5pp$ 가 흔들렸다. 채점은 갈래 표시 없이 섞은 합집합에 제품과 같은 국내 모델을 1회 (temperature 0) 돌린 것이다. 사람 라벨이 아니다. 기본 채점은 일반론에 관대해서 엄격 채점을 따로 두었다. **왜 몰랐나** 검색 결과가 같으니 점수도 같으리라 여기기 쉽다. 회차 간 비교로 보면 개선이나 악화로 읽힌다. → 7장

사례 · 16/16과 4/7

무슨 일이 논문 실험에서 「가짜 근거」 판정은 LLM 맹검 채점이었다. 시드로 미리 뽑아 봉인한 30건을 세무사 판정과 맞췄다. LLM이 가짜로 본 16건은 16/16 결함이었다. 정상으로 본 7건 중 4건도 결함이었다. 과대 판정은 없고 누락이 있으니, 논문의 숫자는 하한이 된다. **잡은 방법** 봉인 키와 대조표의 인용 문자열이 일치하는지 assert로 먼저 확인한 뒤 집계했다. 회신을 보고 정한 제외 규칙은 민감도(17/17, 15/16)를 함께 냈다. 사람의 역할은 「표본 정오 확인」으로만 적었다. 「캘리브레이션」으로 부풀리지 않았다.

사례 · 스캔이 놓친 확정 오류

무슨 일이 지식 사전의 오류 후보 스캔이 30건을 찾았다. 그런데 이미 확인된 오류 1건이 태그 없는 출처에서 와서 스캔에 걸리지 않았다. 확인된 사례를 스캔 결과와 대조해 알았다. 그 뒤로 스캔 수치는 「최소 30건」으로, 놓치는 출처와 함께 적었다.

표본의 모양부터 본다 — 다수 찍기 값, 0건, 작은 표본

불균형 평가셋에서는 「전부 다수 클래스로 찍은 모델」의 값과 단순 규칙의 값을 비교선으로 표에 같이 둔다. 부분집합 정확도와 클래스별 재현율도 본다. 표본 0은 0%가 아니다. 관측이 없는 것과 문제가 없는 것을 갈라 적는다. 작은 표본의 차이는 조건을 나눠 표본을 키운 뒤 판정한다. 정답 라벨이 없는 분류는 「분포와 사례를 봤다」와 「정확도를 췌다」를 구분해 적는다.

사례 · 74.3% = 74.3%

무슨 일이 예시를 붙인(few-shot) 충분성 판정의 전체 정확도 74.3%가 그럴듯해 보였다. 평가셋의 「불충분」 비율이 정확히 74.3%였다. 「충분」 재현율은 1.8%였다. 예시가 충분 재현율을 오히려 낮춘 것도 묻혀 있었다. **잡은 방법** 평가셋 분포와 대조했고, 진짜 지표를 「중간 vs 충분」 부분집합으로 정했다.

사례 · 6배 차이와 0행

무슨 일이 답변의 반복 루프가 새 변경 쪽에 몰려 보였다(1/64 vs 6/64). 변형 4 × 근거 2 × 32로 나눠 다시 세자 6/128 vs 10/128로 구분되지 않았다. 약 5~8%의 기저 현상이었다. 같은 주, 실사용 계측 테이블은 0행이었다 (나중에 47행이 쌓였지만 전부 스모크 시험이었다). 「실사용 턴에서 문제 0건」은 「문제 없음」처럼 들리지만, 실사용 빈도는 모른다고 적었다. 세목 재분류도 17개 범주가 모두 쓰였고 미분류 9%였지만 전후 비교는 사례 4건뿐이었다. 남은 9%가 「분류 불가」인지 「분류기 실패」인지는 숙제로 남겼다.

임계값은 실측 편차와 전수 판독 위에 긋고, 자동 가드는 원인에 건다

다른 소스의 임계값을 물려받으면 잘린 것은 결과에 보이지 않는다. 소스마다 임계값을 따로 둔다. 편차를 아직 안 췌으면 상대 임계값 대신 가장 약한 절대 조건(「문장 0개」)만 건다. 첫 오탐과 간격이 너무 좁은 값은 피한다. 회차를 자동으로 멈추는 가드는 결과 지표가 아니라 원인(호출 실패율, 유실 조각 비율)에 건다. 좋은 회차를

놓치는 비용과 나쁜 회차를 통과시키는 비용을 비교해 기울인다. 기준선에서는 실패 회차를 뺀다. 가드가 실제로 발화한 적이 없으면 그것도 적는다.

사례 · 0.4985와 0.50

무슨 일이 검색은 문장을 숫자 벡터로 바꾼 임베딩끼리 코사인 유사도(방향이 얼마나 같은가, 1에 가까울수록 비슷하다)로 잰다. 지식 사전의 응축 문장은 이 값의 분포가 원래 낮아서, 관련 있는 매치 0.4985가 기존 검색 컷 0.50에 소리 없이 잘렸다. 같은 주의 다른 임계값은 실측 위에 그었다. 나쁜 회차 가드는 실측 편차의 6배, 유실 조각 상한 0.10은 관측값 0~4%의 2.5배였다. 재부착 코사인 0.70은 쌍 171건을 전부 찍어 읽고 첫 오탐(0.6303) 직전에 그었다. 0.635는 간격이 0.006뿐이라 피했다.

사례 · 스스로를 무력화하는 가드

무슨 일이 매번 전체를 다시 만드는 회차에 가드를 넣었다. 호출 실패율이 2%를 넘거나 배정률이 $\max(35\%, \text{직전} \times 0.75)$ 밑이면 이전 세대를 보관하기 직전에 멈춘다. **왜 몰랐나** 인용률 같은 결과 지표에 선을 그으면 촛촛할 때 정상 회차를 막고 느슨할 때 아무것도 못 막는다. 기준선에 실패한 회차를 넣으면, 가드가 한 번 멈춘 회차가 기준선이 되어 스스로를 무력화한다. **잡은 방법** 실제 기준선에 대고 8개 시나리오를 읽기 전용으로 검증했다(정상 통과, 붕괴 중단, 배정률은 정상인데 429가 16%인 회차 중단).

1.3 숫자에서 원문으로 내려간다

숫자는 「 어디까지 」를 말하고 「 왜 」는 말하지 않는다. LLM 제품에서 「 왜 」는 거의 언제나 문장 안에 있다. 평균에서 건 단위로, 건 단위에서 원문으로 내려가야 결함과 정상 동작이 갈린다.

평균 다음엔 건 단위로 겹쳐 보고, 막히면 원문을 읽는다

조건별 평균 → 건 단위 겹쳐 보기 → 원문 읽기 순으로 내려간다. 평균은 특정 집합을 지운다. 원문을 읽어야 「이건 결함이 아니라 정상 동작」이라는 판정도 할 수 있다. 정규식 같은 자동 판독 지표는 첫 회차에 걸린 표본 전체를 사람이 읽어 오탐 유형을 걸어 낸다. 측정 판독 규칙이 제품 가드와 같아야 하면, 제품을 바꿀 때 측정도 같이 고친다.

사례 · 평균이 지운 52건

무슨 일이 목차 A/B/C 세 안의 평균 배정률은 비슷했다. 건 단위로 겹치자 52건(34.7%)은 어느 목차로도 갈 곳이 없었다. 원문 15건을 읽고서야 원인이 나왔다. 검수 기록 속 AI 답변이 질문과 다른 주제였다. 합성에서 「3회 내내 인용 안 됨」 25건도 원문을 읽으니, 확정 지식 없이 「AI 답변이 틀렸다」는 기록만 있었다. 인용하지 않는 것이 맞았다.

사례 · 과대 집계한 1차 정규식

무슨 일이 답변에 판정형 문장이 섞여 드는 것을 세는 1차 정규식이 엔진 근거 문구(「한도 초과분은 부인됩니다」)와 올바른 문장(「한도와 무관하게」)까지 잡았다. **왜 몰랐나** 정규식 집계는 숫자 하나로 나와 근거 문장을 안 보게 된다. **잡은 방법** 걸린 문장을 전부 읽고 규칙을 좁혀 다시 분류했다. 방향은 1차 원시 수치와 같았다(23 vs 22).

이상한 숫자는 그 숫자를 만든 행을 직접 열어 판정한다

숫자만 보면 버그와 정상이 구분되지 않는다. 계속 숫자가 이상하면 그 숫자를 만든 근거 행을 직접 조회한다. 실패 목록이 비었다면 「물어봤는데 기준 미달」인지 「물어보지도 못함」인지 갈라 읽는다. 클러스터링처럼 결과가 그럴듯한 자동 묶음은, 믿기 전에 묶음 하나를 끝까지 까 본다.

사례 · 버그처럼 보인 6

무슨 일이 묶음 승계 첫 실증에서 「새로 만든 묶음 6」이 재사용 버그처럼 보였다. **잡은 방법** 이름이 겹치는 활성 묶음을 조회하니 0행이었다. 새 세목용 새 이름이었다. 전후 배치를 대조하는 SQL로 승계 19건이 전부 이전과 같은 묶음인 것도 확인했다.

사례 · 74%를 잡은 중복 클러스터

무슨 일이 중복 지식을 찾으려고 의미 유사도 0.85로 묶었더니 526건의 74%(104개 클러스터, 390건)가 잡혔다. 「클러스터마다 가장 오래된 것만 남긴다」는 초기 설계대로였다면 286건이 지워질 뻔했다. 대부분 정당한 기여였다. **왜 몰랐나** 의심하던 3건이 정확히 잡혀 방법이 맞아 보였다. 질문·답변·코멘트를 한 덩어리로 만든 임베딩은 질문 글에 끌려가서, 같은 대화의 다른 구간이나 같은 시나리오 틀로 만든 다른 대화가 함께 묶였다. 연쇄로 묶는 방식이 이를 키웠다. **잡은 방법** 의심 클러스터 하나(6건)를 대화 ID와 구간 ID로 손으로 대조했다. 진짜 중복은 같은 구간에 160ms 간격으로 들어간 2건뿐이었다. 신호를 대화·구간 일치로 바꾸자 자동 후보가 286건에서 45건(41개 그룹)이 됐다. 의미 유사도는 「사람이 볼 목록」에만 쓴다. → 4장

함께 움직이는 두 변수로 원인을 하나로 단정하지 않는다

두 변수가 늘 같이 움직이면 어느 쪽 해석도 관측과 모순되지 않는다. 큰 공사의 근거가 되는 해석은 공변하는 변수를 하나씩 고정해 따로 켜 뒤에 쓴다. 목표에 못 미친 채 끝나더라도, 뒤집힌 해석은 설계 문서에 정정해 남긴다. 회귀에서 한 번 나온 실패도 「잡음」으로 적기 전에 그 화면이 어떤 데이터를 기다리는지 본다. 적재 순서를 바꾸는 변경은 숨은 경합을 드러낸다.

사례 · 채널당이 아니라 바인딩당

무슨 일이 웹소켓 프레임워크 보고 「실시간 구독의 준비 신호는 채널당 약 250ms」라고 해석해 설계에 적었다. 그 해석을 믿고 컬렉션 15개를 채널 하나로 묶었는데, 관리자 첫 화면은 4,827ms에서 4,392ms로 거의 줄지 않았다(목표 2,500ms). **왜 몰랐나** 그때까지는 컬렉션 하나가 채널 하나여서 채널 수와 구독 바인딩 수가 늘 같았다. **잡은 방법** 바인딩·채널 조합 5종을 직접 켜다. 바인딩 15 · 채널 1에서 마지막 준비가 1.9~2.3초로, 바인딩 수에 비례했다. 실제로는 바인딩당 약 60~90ms에 연결 바닥 약 1.0초였다.

사례 · 잡음이 100%가 되다

무슨 일이 회귀 1회차에서 관리자 검수 상세 화면이 한 번 「찾을 수 없습니다」로 떴다. 다시 돌리면 정상이라 「무관한 적재 경합」으로 적었다. 채널을 묶자 재현이 100%가 됐다. **왜 몰랐나** 그 화면은 대화 목록이 다 실릴 때까지 기다리지 않았다. 대화 목록이 자기 채널로 먼저 준비돼 경합에서 거의 늘 이겼다. 묶은 뒤에는 가장 큰 대화 목록이 꼴찌가 되어 경합이 뒤집혔다.

이상이 보이면 화면과 모델보다 그 앞단의 데이터와 엔진을 거슬러 본다

화면이 한 덩어리로만 보이면 렌더보다 데이터를 먼저 본다. 메타데이터는 그것을 읽는 화면이 생기기 전에 분포를 한 번 찍어 본다. 클라이언트가 채운 자리표시 값은 서버가 적재할 때 실제 값으로 덮어쓴다. 모델 답변의

이상한 문구도 근거 지식만이 아니라 프롬프트에 박히는 결정 엔진의 출력에서 온다. 엔진은 결정적이고 권위가 있어 의심 대상에서 빠지기 쉽다.

사례 · 전부 한 색인 그래프

무슨 일이 세무사가 지식을 거미줄 그래프로 보는 화면이 전부 한 색이었다. 대화 복원 코드가 세목을 늘 「미분류」로 박아 넣었고, 그 값이 적재까지 흘러 지식 전량(521건)이 미분류였다. **왜 몰랐나** 화면에는 「미분류」 클러스터 하나로 정상처럼 보였다. 세목으로 묶는 화면이 생기기 전까지 아무도 그 값을 읽지 않았다. **잡은 방법** 그래프 스모크에서 대표가 「왜 전부 한 색이냐」고 물었다. 화면의 폴백을 의심하기 전에 데이터를 봤다. 서버가 자리표시 값을 덮어쓰게 하고 413건을 소급 재분류했다(미분류 100% → 9%).

사례 · 회원권 명의

무슨 일이 복리후생비 답변에 「회원권 명의」가 자주(32회 중 21회·17회) 나와 다른 사안의 내용이 섞여 든 것처럼 보였다. 실제로는 규칙 엔진의 필요 증빙 목록에 남은 헬스장 사안의 잔재였다. 판정 카드는 「복리후생비 인정」이라고 하는데 본문은 「급여성 상여」라고 반박하는 충돌도, 원인은 엔진 근거 문구였다. **잡은 방법** 문구의 출처를 엔진 출력까지 거슬러 찾았다. 근거 지식의 검수 코멘트가 이미 그 증빙을 「환각 증빙」으로 지적해 두었다. 맞는 쪽(판정·금액)을 남기고 엔진 문구를 고치자 본문의 「전액 인정」 명시가 21회에서 53회가 됐다. → 3장

1.4 그럴듯한 기억·문서·지시를 믿기 전에

그렇듯하게 틀리는 것은 LLM만이 아니다. 기억, 앞 세션의 결론, 작업 지시서, 주석, 필드 이름, 프롬프트에 적은 금지 문장도 그렇다. 사람과 AI 에이전트가 세션을 번갈아 일하는 현장에서는 이것들이 다음 작업의 전제가 된다. 전제는 검증된 적이 없어도 검증된 것처럼 읽힌다. 이 절의 원칙은 전부 「열어 본다」로 끝난다.

■ 답하기 전에 그 질문을 왜 하는지, 전제가 성립하는지부터 본다

맞는 답도 묻던 질문이 아니면 쓸모가 없다. 로드맵 항목을 재검토하기 전에 시작 동기를 한 문장으로 확인한다. 항목 이름이 기술 이름이면 특히 그렇다. 「얼마나 잘 되나」를 재기 전에 「그 경로에 있기는 한가」를 먼저 본다. 화면에 보이는 지식과 모델이 실제로 받는 지식은 다를 수 있다.

사례 · 다른 질문에 대한 맞는 답

무슨 일이 로드맵의 「GraphRAG」 항목을 「검색 엔진을 그래프 기반으로 바꿀까」로 읽고, 성능과 가치 두 관점에서 재검토해 비채택으로 결론지었다. 대표가 정정했다. 그 항목의 동기는 「세무사가 지식을 거미줄 그래프로 보고 직접 고치는 중간층」이었다. **왜 몰랐나** 항목 이름이 기술 이름이라 목적이 문서에서 흐려져 있었다. 재검토 결론 자체는 논리적으로 맞아서 의심할 틈이 없었다. **처리** 저장된 연결(413개 노드, 3,304개 연결선)로 전체 그래프를 그렸다.

사례 · 아무도 부르지 않는 지식

무슨 일이 대표는 세무사 화면의 사전형 지식이 질문과 얼마나 잘 맞는지 걱정했다. 실제로 그 지식은 프론트엔드에 박힌 정적 시트여서 검색에 아예 쓰이지 않았다. 적재 함수는 있었지만 부르는 곳이 없었다. **왜 몰랐나** 세무사 화면에 지식이 멀쩡히 보였다. 화면에 있으면 쓰이고 있다고 여기게 된다. **잡은 방법** 백엔드에서 호출처를 검색했다. 이 진단이 지식 검색을 세 층으로 다시 짜는 로드맵의 출발점이 됐다.

기억과 전언보다 기록·실측·공식 원천을 다시 연다

기억과 정황이 서로를 받치면 가설은 의심받지 않는다. 데이터의 정체를 판단하기 전에 같은 질문을 전에 조사한 문서가 있는지부터 찾는다. 조사 결론은 다음 세션이 찾을 수 있는 자리에 남긴다. 정체가 확정돼야 지울지 말지를 논한다. 외부에서 온 전언도 같다. 재수정 기회가 없는 제출은 직전에 공식 페이지를 다시 연다. 마감이 둘로 적혀 있으면 이른 쪽을 따른다.

사례 · 논문 데이터라는 기억

무슨 일이 관리자의 「완료」 화면에 7월 중순의 미확정 검수 항목이 쌓여 있었다. 대표는 이것을 학회 논문 실험 데이터로 기억했다. 실제로는 제품 개발 작업(100건 일괄 감사, RAG 재검증)이 남긴 검수 대기열이었다. 논문이 DB에 남긴 흔적은 지식 168건의 상태 변경 하나뿐이었고 그것도 이미 복구돼 있었다. **왜 몰랐나** 정황이 들어맞았다. 같은 주에 RAG 컴·곰 비교 리포트가 연달아 나왔고, 한 커밋이 그 리포트들을 논문 실험 기록으로 묶었다. **잡은 방법** 이전 세션이 같은 질문을 조사해 남긴 문서를 찾아 읽었다. 7월 기록을 다시 읽어 미확정의 진짜 원인(권한 정책 누락)도 확인했다. → 1.6 0행 갱신

사례 · 6쪽이라는 메일

무슨 일이 학회 채택 메일은 최종본을 6쪽이라 했고, 공식 제출 안내 페이지는 4쪽이라 했다. 메일을 근거로 6쪽판을 주력으로 정했다가 제출 전날 4쪽판으로 뒤집었다. 저작권 양식 마감도 사이트와 메일이 서로 달랐다. **왜 몰랐나** 메일은 학회가 보낸 명문 근거라 믿을 만해 보였다. 그러나 메일 본문은 여러 학회가 재사용하는 틀이라 틀릴 수 있다. **잡은 방법** 제출 직전에 공식 안내 페이지를 다시 열었다. 비용은 5분이고, 안 열었을 때의 비용은 게재 실패다. → 7장

비싼 작업의 근거가 된 이전 결론은 원문 전수로 다시 확인한다

기록된 원인은 「반박되지 않은 가설」 일 수 있다. 이전 측정이 그 결론과 모순되지 않았다는 것은, 다른 설명을 시험하지 않았다는 뜻일 뿐이다. 재적재·재구성처럼 비싼 작업 직전에는 그 근거가 된 결론을 원문 전수로 다시 본다.

사례 · 하지 않아도 됐던 재적재

무슨 일이 이전 단계와 로드맵은 「검색 결과에 남의 사안 수치가 섞여 드는 원인은 지식 묶음 속 옛 AI 답변 부분」이라고 적고 있었다. 그런데 근거로 삼은 다섯 건 모두 질문 부분에도 같은 수치가 있었다. 전수를 정규식으로 보니 남의 사안 수치는 세 부분에 흩어져 있었다. AI 답변 부분을 빼는 처방은 무효였고, 재적재도 필요 없었다. **잡은 방법** 재적재를 결정하기 전에 대표가 「원인부터 특정하라」고 했다. 부분별 전수로 다시 봤다.

「이미 있다」「다른 곳이 잡는다」는 문장은 코드로 확인한다

문서와 주석은 그럴듯하게 쓰여 있고, 그 문장이 필요해지는 날이 오기 전까지 아무도 검증하지 않는다. 작업 지시의 「이미 있다」는 호출처까지 확인한다. 책임을 다른 장치에 넘기는 문장(「그건 X가 잡는다」)을 쓰면, 그 장치의 코드에서 해당 입력이 실제로 걸리는지 본다. 주석의 「나중에 ~하면 ~로」는 그때가 오면 다시 검증하고, 틀렸으면 근거와 함께 뒤집어 적는다. 운영 조연도 그 함수가 실제로 받는 입력을 본 뒤에 한다.

사례 · 죽은 코드와 반대로 말하는 주석

무슨 일이 작업 지시서는 「개별 종료 기능이 관리자 화면에 이미 있다」고 했다. 그 함수는 어디서도 불리지 않는 죽은 코드였다. 서버 서비스 설정의 주석은 「큰 서버면 워커를 2~4개로」라고 권했는데, 이는 멈춘 작업을 회수하는 설계와 정반대였다. 1GB 서버에서는 따로 일이 없었지만 12GB 서버로 옮기는 순간 따라갈 위험이 실재했다. **잡은 방법** 호출처를 검색하고, 주석을 회수 코드와 대조했다.

사례 · 아무도 잡지 않던 날조 조문

무슨 일이 설계는 「지어낸 조문 번호는 수치 날조 가드가 잡는다」고 적었다. 그런데 조문 번호는 설계상 그 가드의 수치 토큰에서 빠져 있었다. 날조 조문은 아무도 잡지 않고 있었다. **잡은 방법** 답변 단계의 가드를 짜며 수치 가드의 범위를 확인했고, 조문 전용 가드를 따로 두었다. 그 가드가 「N월」 같은 기간도 보지 않는다는 것은 열린 과제로 적었다.

사례 · 한 번 더 누르면 된다는 조언

무슨 일이 자동 그룹화 결과가 한 묶음(29개 중 19개, 66%)에 몰리자 「한 번 더 누르면 된다」고 권했다. 그 함수는 아직 묶이지 않은 세목만 처리하므로, 다시 눌러도 남은 1건만 건드린다. **잡은 방법** 세션 중에 코드를 다시 보고 정정했다. 처방은 재실행이 아니라 다 빨아들이던 「기타·특수 상황」 묶음을 지우는 것이었다. 그 뒤 2차는 28개 중 12개(43%)였다.

이름을 믿지 말고 값과 카탈로그를 연다

이름 규칙과 연도는 「당연히 그렇겠지」 싶은 값이다. 추정한 이름·값은 틀려도 오류 없이 엉뚱한 것을 건드린다. 외부 데이터의 필드는 실제 값 몇 건으로 뜻을 확인한 뒤 쓴다. 스키마 객체 이름은 카탈로그에서 조회하고, 날짜 상수는 연도부터 검산한다. 비교·매칭에 쓰는 문자열은 저장값 정의에서 그대로 가져오고, 괄호·라벨 같은 꾸밈은 비교가 끝난 뒤에 붙인다.

사례 · 이름과 다른 값들

무슨 일이 국세청 질의회신 데이터의 「전문(full_text)」 필드에는 회신만 있고 요지가 없었다. 본문을 「요지 + 회신」으로 조합해 고쳤다. DB 상태 제약을 교체할 때는 실제 제약 이름을 카탈로그에서 찾아 썼다. 확정 시각 상수는 처음에 2025년 기준으로 계산했다가 연도부터 다시 확인해 고쳤다. 프론트엔드의 미분류 상수는 「(미분류)」, 백엔드 저장값은 「미분류」였다. 괄호 하나 때문에 미분류 설명이 안 떴고, 직전 커밋의 「맨 뒤로 정렬」도 동작하지 않았다. **왜 몰랐나** 미분류가 우연히 건수 순으로도 뒤쪽이라 정렬 실패가 보이지 않았다. 대표가 실사용 중 「설명이 안 뜬다」고 지적했다.

원칙과 문구는 조건과 함께 적고, 전제가 바뀌면 다시 읽는다

「그 밖의 전부」에 붙는 문구는 새로 생긴 갈래가 자동으로 물려받는다. 공용 함수 안의 머리말은 눈에 띄지 않는다. 설계 근간이 바뀌면 공용 프롬프트 문구를 새 전제로 다시 읽는다. 출처 문구는 지식 원천별로 조립하고, 문구가 출처를 부풀리지 않게 한다. 원칙을 적을 때도 「언제까지, 무엇이 입증되면」을 같이 적는다. 조건이 빠진 원칙은 연구 규칙처럼 읽힌다. 조건이 충족돼 바꿀 때는 통과선과 되돌리기 조건을 먼저 정하고, 되돌리기 쉬운 자리(설정)에서 바꾼다.

사례 · 물려받은 출처 문구

무슨 일이 자문 답변의 첫머리 「세무사들이 남긴 검수 의견을 근거로」는 사전형 지식만 아니면 무조건 붙었다. 국세청 질의회신 원문 지식이 붙는 순간 그것까지 「세무사 검수」라고 부르게 될 참이었다. 공통 머리말의 「판정은 규칙 엔진의 권위」도, 판정 권위를 답변 모델로 옮긴 새 설계에서는 거짓이었다. 「검수 의견」 자체도 실은 감사 모드 LLM의 산출물이었다. **잡은 방법** 새 답변 단계를 짜다 드러났다. 출처 문구를 지식 원천별로 나눴다.

사례 · 「어느 단계에서도 바꾸지 않는다」

무슨 일이 「기본 검색 방식은 어느 단계에서도 바꾸지 않는다」는 원칙이 있었다. 다시 보니 이 원칙은 「입증 전까지」라는 조건부였다. 4방향·5방향 비교와 판정형 192회 측정으로 조건이 충족돼, 이를 뒤 프로덕션 기본값을 여러 지식층을 합쳐 찾는 방식으로 바꿨다. **처리** 대표가 원래 이유를 되짚어 통과선(새 방식이 기존보다 나쁘지 않을 것)과 되돌리기 조건(스모크에서 이상이 보이면 즉시, 이후는 실사용 턴으로 재판정)을 정했다. 코드 기본값은 두고 서버 설정 한 줄로 바꿔 로컬과 벤치는 그대로 두었다. → 2장

프롬프트로 한 부탁은 지켜졌다고 믿지 않는다 — 형식과 코드로 강제한다

모델이 산문 지시를 어겨도 에러는 나지 않는다. 개수 상한은 슬라이스나 도구 스키마로 건다. 빈도처럼 셀 수 있는 것은 코드가 센다. 「빠짐없이·정확히 하나·n개」 같은 집합 제약은 「목록 확정 → 그 목록 안에서만 고르기」처럼 실패가 불가능한 출력 형식으로 옮긴다. 이미 덮였는지처럼 규칙이 명확한 판정도 코드가 하고, 모델에게는 코드가 못 하는 판단만 남긴다. 금지 규칙에는 출력 가드를 째지어 둔다. 그래도 같은 입력에서 결과 분포는 흔들리므로 계측은 계속한다.

사례 · 「최대 20개」에 149개

무슨 일이 「최대 20개」에 149개, 「8~14개」에 128개, 「3~6개」에 배치당 18~32개가 나왔다. 도구 스키마의 최소·최대 개수는 지켜졌다. 스키마는 개수만 지키게 할 뿐 뜻이 같은 것을 합치게 하지는 못했다. 「모든 세목은 정확히 하나의 묶음에, 묶음은 3~8개」를 부탁하자 누락 9·중복 1, 다른 문구로는 누락 2·중복 3이 나왔다. 세목→묶음 1:1 스키마로 뒤집으니 이번에는 개수를 어겨 17개로 쪼갬다. **왜 몰랐나** 기존 구현도 원래 9건을 빠뜨리고 있었지만 화면에는 「미분류」로만 보였다. **잡은 방법** 같은 29개 세목, temperature 0으로 네 조건을 나눠 켜다. 1단계가 개수 제한 안에서 묶음 목록을 확정하고 2단계가 그 목록 중에서만 고르게 하자 누락·중복이 0이 됐다. 그래도 묶음 크기는 10/11/14/17로 흔들렸다.

사례 · 「수치를 지어내지 마세요」

무슨 일이 자문 경로 프롬프트에 「수치를 지어내지 마세요」가 있었는데 96건 중 11건이 수치를 지어냈다. 판정형 답변은 접대비 한도를 64회 중 23회, 그것도 회차마다 다른 값으로 말했다. **왜 몰랐나** 지어낸 한도는 그럴듯한 숫자였고, 판정 카드는 맞았다. **잡은 방법** 「질문·엔진·근거·규범 어디에도 없는 수」를 세는 지표를 만들었다. 판정형 날조 답변이 192건 중 29건 → 규칙만 넣어 10건 → 규칙과 가드를 함께 써서 0건이 됐다. 자문은 11/96에서 0이 됐다. 가드는 숫자만 지우지 않고 문장 단위로 빼며(숫자만 지우면 문장이 깨진다), 하나라도 뺐으면 확인 안내를 붙인다. 값은 고치지 않는다. 어느 값이 맞는지는 전문가 몫이다. 출처 대조에는 이전 사용자 발화만 넣어, 앞선 답변의 날조가 다음 턴 근거로 되살아나지 않게 했다. → 3장

1.5 데이터는 누가 만들었고, 누가 읽는가

지식 제품에서 데이터 한 줄의 값어치는 누가 만들었는지에 달렸다. 세무사가 쓴 코멘트와 LLM이 쓴 코멘트는 같은 칸에 같은 모양으로 들어간다. 이 제품에서도 검수 코멘트와 5점 평가 중 평가자 계정 이름으로 들어간 것은 감사 모드 LLM의 산출물이었다. 사람(세무사)이 개입한 자리는 승인 게이트와 원문·인용 대조뿐이다. 반대 방향도 있다. 한 데이터를 누가 읽는지 모르면 새 데이터 한 줄이 엉뚱한 기능으로 새어 든다.

데이터 출처는 계정·화면 이름이 아니라 생산 경로로 적는다

데이터를 누가 만들었는지는 계정 이름이나 화면 용도로 추정하지 않고 생성 경로로 적는다. 정정한 문장도 같은 기준으로 한 번 더 확인한다. 정정은 검증을 거친 것처럼 느껴져 다시 의심받지 않는다. 외부로 나가는 자료는 출처 표기(사람인가 AI인가)를 따로 점검한다. 성격이 다른 검토 요청은 한 표에 섞지 않는다.

사례 · 두 번 오독한 출처

무슨 일이 앞의 「논문 데이터라는 기억」을 바로잡으면서, 원문은 결론을 다시 「세무사들이 실작업으로 남긴 검수 대기열」이라고 적었다. 그러나 평가자 계정 셋의 코멘트와 세션 평가는 감사 모드 LLM의 산출물이었다. 첫 오독을 고친 문장이 두 번째 오독이었다. **왜 몰랐나** 계정 이름이 「auditor」였고 화면도 세무사용 검수실이었다. 첫 오독을 바로잡은 직후라 새 결론이 맞아 보였다. **잡은 방법** 주간 기록을 다시 쓰면서 출처 규칙에 따라 고쳤다. 논문 인용에도 같은 출처 표기를 쓰기로 했다.

사례 · 외부로 나갈 뻔한 내부 줄임말

무슨 일이 내부 문서가 검수 코멘트의 출처를 「세무사 코멘트」라고 적고 있었다. 세무사에게 보낼 확인 자료에 이 표기가 그대로 들어갈 뻔했다. **왜 몰랐나** 내부에서 굳은 줄임말이라 아무도 다시 보지 않았다. **잡은 방법** 확인 자료를 만들며 표기를 점검했다. 외부본에는 「검수 코멘트(AI 평가 도구 작성, 참고용)」로 썼다. 제품 확인용 검토 요청 31문장은 논문 맹검 표본 30건과 대조해 같은 문장이 없음을 확인했다(두 글자 단위 유사도 최대 0.30). 읽는 자료와 답을 적는 회신표도 나눴다.

신원과 권한은 화면이 아니라 서버에서 세우고, 역할별로 실측한다

요청 본문으로 받은 신원 위에 세운 「N명 승인」은 장식이다. 거버넌스 규칙보다 인증된 신원을 먼저 세운다. 단, 기여 귀속(누가 썼나)을 인증 신원(누가 눌렀나)으로 덮지 않는다. 접근 제어도 화면이 아니라 DB에서 강제한다. 이 제품은 RLS(행 수준 보안 — DB가 행마다 누가 읽고 쓸 수 있는지 정하는 규칙)를 썼다. 데이터 노출을 설계하기 전에 역할별 토큰으로 API를 직접 불러 실제로 보이는 범위를 센다. 열람 판정은 함수 하나에 모은다.

사례 · 본문으로 받은 신원

무슨 일이 세무사 한 명의 확정이 모든 답변의 시스템 프롬프트를 바꾸게 됐는데, 신원은 요청 본문으로 받고 있었다. 그 위에 「두 명 승인」을 얹으면 본문만 바꿔 위조할 수 있다. **왜 몰랐나** 데모 계정만 쓰는 환경이라 위조가 실제로 일어나지 않았다. **잡은 방법** 대표가 순서를 「인증 → 이의 기간 → 관리자 브레이크」로 정했다. 쓰기 API 전체에 토큰 인증을 걸고, 비로그인 방문자가 쓰는 챗은 뺐다. 본문에 다른 사람 ID와 내 토큰을 보내자 서버 기록은 토큰 쪽이었다.

사례 · 화면 관행이었던 비식별

무슨 일이 세무사 계정 하나가 대화 218건 전부를 내용까지 읽을 수 있었다. 검수 이력이 없는 계정도 같았다. 화면이 배정분만 보여 주고 있었을 뿐이다. 축소 조건을 그대로 쓰면 역할을 안 가려서 사장님도 남의 대화를 읽게 된다는 것도 찾았다. **왜 몰랐나** 브라우저 콘솔이나 API를 직접 부르지 않는 한 드러나지 않는다. **잡은 방법** 설계 전제를 확인하려고 세무사 토큰으로 DB API를 직접 불렀다. 대표는 「남겨 두면 비식별은 강제가 아니라 화면 관행일 뿐」이라며 축소를 공사에 넣었다. → 5장

사람이 손댄 것은 모든 갱신 경로에서 보존한다

재생성 파이프라인은 사람이 고친 것을 조용히 지운다. 보호 장치를 만들면 그것이 **모든** 갱신 경로를 막는지 확인한다. 보존 여부는 별도 플래그가 아니라 수정 이력에서 판정한다. 보존 결과를 셀 때는 그 편집이 실제 사용자 작업인지 검증용 편집인지도 가른다.

사례 · 한 회차 만의 원상복귀

무슨 일이 「기타」에서 옮긴 문장이 정확히 한 회차 만에 원래 자리로 돌아왔다. 목차를 매번 전체 다시 만들고 이전 세대를 보관하는 구조였는데, 보호 플래그는 옛 재합성 경로만 막고 있었다. 지식이 자라지 않고 매번 다시 만들어지고 있었다. **왜 몰랐나** 보관은 정상 동작이었다. 사람이 손댄 양이 작아 비용이 안 보였다. **잡은 방법** 직접 옮긴 문장의 운명을 회차 직후 DB에서 추적했다. 손댄 문장(수정·이동·끊기 전부)을 보존하고, 레이블 임베딩이 가장 가까운 새 세목에 다시 붙였다(레이블 글자 일치는 생존 33~67%라 기각). 프로덕션 2회 모두 보존 3/3, 재부착 2/3이었다. 단, 보존 대상 3건 중 적어도 둘은 세무사의 실작업이 아니라 검증 과정의 편집이었다. 실사용 표본으로 착각하지 않는다.

값을 바꾸거나 데이터를 넣기 전에 그것을 읽고 더하는 자리를 전부 센다

넣는 순간에는 아무것도 깨지지 않는다. 다른 기능이 새 데이터를 제 것처럼 쓰기 시작할 뿐이다. 새 데이터를 기존 테이블에 넣기 전에 그 테이블을 읽는 소비자를 모두 적는다. 하나라도 섞이면 안 되면 별도 스키마로 둔다. 한 값의 단위나 규모를 바꿀 때는 그 값을 더하는 모든 자리를 찾고, 단위가 다른 행은 종류 칸으로 갈라 합계에서 뺀다. 같은 일을 하는 로직은 한 함수로 모으고, 파라미터를 바꿀 때는 그것을 쓰는 모든 경로를 함께 본다.

사례 · 같은 테이블을 읽는 세 갈래

무슨 일이 설계는 국세청 심판례·질의회신 원문 지식을 기존 지식 테이블에 「출처」만 달리해 넣는 안이었다. 그 테이블은 지식 사전 합성의 원재료, 검수 선례 검색, 거미줄 그래프가 모두 읽고 있었다. 출처를 가릴 칸도 없었다. **잡은 방법** 실제 테이블 구조를 확인했고, 대표가 별도 스키마로 정정했다. → 4장

사례 · 우연히 무해했던 합계

무슨 일이 원장의 잔액 칸이 기여 크레딧(+10)과 정산 지급액을 구분 없이 더하고 있었다. 정산 재원을 원화로 바꾸면 세무사의 「누적 크레딧」이 정산액에 완전히 덮일 구조였다. 이전에는 작은 추상 숫자(「cr」)라 섞여도 눈에 띄지 않았다. 같은 종류로, 예산 초과분을 짚개 담는 로직이 새 경로에만 있고 옛 경로는 초과분을 조용히 버리고 있었다. 예산을 6000으로 반으로 줄이면 그 유실이 두 배가 될 참이었다. **잡은 방법** 단위와 예산을 바꾸는 작업 중에 그 값을 쓰는 자리를 따라가다 발견했다. 종류 칸으로 합계를 가르고, 분할은 공유 함수 하나로 모았다.

1.6 프로덕션에 쓰기 전에

지식 제품의 운영 데이터는 되돌리기 어렵다. 한 번 적재된 지식은 다음 답변의 근거가 되고, 지운 기여는 누군가의 정산 몫이다. 이 절의 사고들은 모두 명령이 성공으로 끝났다. DB 검증 쿼리도 통과했다. 그런데 지운 것이 틀린 대상이었거나, 백업이 세션과 함께 사라질 자리에 있었거나, 서버가 「아마 됐을」 상태로 떠 있었다.

결정은 쓰기 없는 계측으로 내리고, 쓰기는 dry-run 기본·먹등으로 만든다

쓰고 나서 재면 잘못된 결정이 이미 데이터에 박힌다. 공유 DB에 쓰는 스크립트는 기본을 dry-run(실제로 쓰지 않고 할 일만 보여 주기)으로, 실행은 먹등(여러 번 돌려도 결과가 같음)으로 만든다. 결정은 쓰기 없는 계측으로 먼저 내리고, 쓰기는 사람이 확인한 뒤에 한다. 단, 메모리 안에서 켜 숫자는 실제 DB 경로로 다시 재기 전까지 잠정이다.

사례 · DB 쓰기 0으로 정한 컷

무슨 일이 새 원문 지식은 프로덕션 DB를 건드리지 않고 메모리 안 벡터 검색으로 158문항을 두 번 잼다. 유사도 컷, 검색 범위, 출처 간 서열이 그 숫자로 정해졌다. 스키마는 작성만 해 두었다. 적재 스크립트는 쓰기 옵션이 없으면 dry-run이고, 사건 번호와 내용 해시로 먹등이다. 벡터는 텍스트 해시를 키로 캐시해 적재 때 그대로 재사용했다. **순서** 계측 → 대표 확정 → 쓰기.

드라이런과 사전검토는 진짜 함수를 부르고, 쓰기 경계에서만 막는다

로직을 베낀 드라이런은 베낀 것만 검증한다. 원본과 어긋나도 드라이런은 통과한다. 미리보기·사전검토·드라이런은 실제 경로의 함수를 그대로 부르고, 쓰기 함수만 대역으로 바꾼다. 측정용 파서도 제품 파서와 전량 일치하는지 확인하고 쓴다.

사례 · 보존 3, 재부착 2, 쓰기 0

무슨 일이 사람이 손댄 문장을 보존·재부착하는 로직을 프로덕션에 넣기 전에 드라이런이 필요했다. 드라이런 스크립트가 진짜 재부착 함수를 부르고 쓰기 함수만 막았다. 결과는 보존 3 / 재부착 2 / 「기타」 1, DB 쓰기 0이었다. 측정판 파서는 413건 전량에서 제품 파서와 일치했다(폴백 0). 앞서 중복 사전검토를 만들 때도, 사전검토가 비교하는 텍스트와 실제 적재되는 텍스트를 같은 조립 함수로 만들게 했다. 따로 만들면 적재 형식이 바뀌는 순간 사전검토만 다른 텍스트를 비교하고, 결과는 여전히 숫자로 나온다.

지울 대상은 화면에서 소스 테이블까지 추적하고, 범위는 상태로 정한다

화면 문제를 데이터로 고칠 때는 그 화면이 어느 테이블을 읽는지 코드에서 먼저 확인한다. 고친 뒤에는 검증 쿼리만 믿지 말고 그 화면을 다시 연다. 「지운 것이 안전한가」와 「지운 것이 맞는 것인가」는 다른 확인이다. 삭제 범위는 날짜가 아니라 진행 상태와 하류 반영 여부로 정한다. 지식이 다른 곳에 독립 사본으로 남는지, 이의 같은 다른 절차가 걸려 있는지 본다. 매매한 상태는 남기고 별도 결정으로 넘긴다. 자동 삭제 후보는 의미 유사도가 아니라 구조적 신호로 고른다 → 1.3 74%를 잡은 중복 클러스터.

사례 · 안전했지만 틀린 삭제

무슨 일이 「완료」 화면의 미확정 항목을 치우려고 확정된 세션 평가 39건을 지웠다. 그 화면은 다른 세 테이블을 읽고 있었다. 2차에서 대상을 바로잡아 제출 상태 감사 52건, 리뷰 16건, 문장별 코멘트 186건을 지웠다. **왜 몰랐나** 1차 삭제 자체는 안전했다. 지식 손실 0건, 에러 0건이었다. 검증은 「안전한가」에만 답했다. **잡은 방법** 대표가 평가자 계정으로 다시 로그인했는데 화면이 그대로였다. 그 뒤 화면 컴포넌트의 소스 테이블을 추적했다.

사례 · 한 단계 더 나간 2건

무슨 일이 「미확정 항목 정리」라는 한 덩어리 요청 안에 진행 단계가 다른 데이터가 섞여 있었다. 확정 세션 평가 39건은 지식 테이블에 독립 사본 39건이 전부 활성화로 있어서 지웠다. 검수실은 지식에 반영되지 않은 「제출됨」만 지우고, 한 단계 더 나간 「검수 저장(이의 가능)」 2건은 뺐다. **왜 몰랐나** 날짜 구간으로만 자르면 이의 절차가 걸린 데이터까지 함께 사라진다. **잡은 방법** 대표가 범위를 「검수실 제출됨만」으로 못 박고, 구현은 지식 사본 대조와 리뷰 상태로 근거를 확인했다.

운영 데이터를 바꾸는 일은 되돌릴 길과 뒷정리까지 한 묶음으로 짤다

운영 데이터 삭제는 「영향 재확인 → 백업 → 단계마다 행 수 검증 → 커밋」을 표준으로 쓴다. 안전 조건은 사람의 확인이 아니라 스크립트의 자동 중단으로 건다. 삭제 전후로 하류 지식 수치를 대조한다. 권한 정책 아래의 쓰기는 영향받은 행 수를 확인하고, 기대가 1행 이상인데 0행이면 실패로 올린다. 백업과 스크립트는 세션 임시 공간이 아니라 세션보다 오래 사는 접근 통제된 자리에 두고, 그 위치를 삭제 기록에 적는다. 실데이터로 검증할 때는 「실행 → 확인 → 원복·삭제 → 재조회」를 한 단위로 짤다. 되돌릴 수 없는 일괄 작업은 같은 함수로 시뮬레이션해 영향 건수를 먼저 본다.

사례 · 스크립트가 스스로 멈추는 삭제

무슨 일이 프로덕션 DB에 직접 삭제를 두 번 실행했다. 스크립트는 지식 영향을 다시 확인해 0건이 아니면 멈추고, 초안이 아닌 리뷰가 섞이면 멈추게 짤다. JSON으로 백업하고, 지식 테이블부터 지우며 단계마다 행 수를 검증했다. 삭제 전후 지식 수치(코멘트 활성화 287, 세션 평가 활성화 120)가 같았다. **남은 구멍** 백업 JSON 2개와 스크립트 3종이 세션 임시 공간에만 있고 저장소에 없었다. 세션 환경이 사라지는 순간 복구 수단도 사라진다. 주간 검증 정리에서 「확인하지 못한 것」으로 적었다.

사례 · 0행 갱신

무슨 일이 세션 평가 테이블에 관리자 쓰기 권한 정책이 없어서, 관리자의 결정이 「0행 갱신」으로 끝나고 있었다. 7월에 미확정 항목이 쌓인 근본 원인이었다. **왜 몰랐나** 권한에 막힌 UPDATE는 에러가 아니라 0행 갱신으로 끝난다. 화면에서는 실패로 보이지 않았고, 미완 데이터가 몇 주 뒤 다른 모습으로 나타났다.

사례 · 실데이터 검증의 뒷정리

무슨 일이 테스트 스위트 대신 프로덕션 데이터로 직접 검증했다. 지식 수정 제안은 실제 지식 1건으로 승인한 뒤 원문으로 다시 승인해 원복했다. 정산 E2E로 만든 회차·원장·메일은 매번 SQL로 지웠다. 중복 정리는 실제 저장 함수를 재현한 시뮬레이션으로 오탐 수(286 → 45)를 먼저 췌고, 정리 뒤에는 같은 구간 재제출 그룹이 0개인지 재조회했다. **왜 그렇게 했나** 로컬 목업으로는 실제 재임베딩과 실제 분포의 문제가 안 보인다. 반대로 뒷정리를 빠뜨리면 테스트 잔여물이 실데이터처럼 남는다.

새 경로는 기본값에서 기존과 같게 붙이고, 이해하는 쪽부터 배포한다

실험 경로가 기존 경로에 섞이면 회귀가 조용히 들어온다. 새 경로는 기존 경로를 한 줄도 건드리지 않는 직교 축(켜는 스위치가 따로 있는 축)으로 붙인다. 값이 없거나 모르는 값이면 기존과 100% 같게 동작하게 하고, 같은 입력으로 회귀 없음을 증명한다. 배포 순서도 뜻을 지킨다. 요청에 새 필드가 생기면 그것을 이해하는 쪽(백엔드)을 먼저, 보내는 쪽을 나중에 낸다. 「안 깨진다」는 「같은 뜻으로 동작한다」가 아니다.

사례 · 3방향 비교가 가능해진 파라미터

무슨 일이 지식 사전 검색을 별도 URL 파라미터로 붙였다. 값이 없거나 모르는 값이면 기존 코드 경로를 그대로 탄다. 기존의 RAG 컴·곰 스위치와 직교해서 「RAG 없음 / 기존 RAG / 지식 사전」 3방향 비교가 구조적으로 가능해졌다.

잡은 방법 같은 질문으로 3방향을 불러 파라미터를 뺀 결과가 기존과 완전히 같음을 확인했다. 기존 검색 모듈이 새 모듈을 import하지 않는 것도 코드로 확인했다.

사례 · 즉시 실행이 된 「예약」

무슨 일이 프론트엔드가 먼저 나가면 새 「예약」 단추가 옛 백엔드에서 즉시 실행으로 돈다. 옛 엔드포인트가 본문의 새 필드를 무시하기 때문이다. 실제로 한 번 프론트엔드를 먼저 내보낸 절차 오류가 있었다. **왜 몰랐나** 새 필드가 선택 사항이라 아무것도 깨지지 않았다. 뜻만 바뀐 채로 동작했다. → 5장

서버 이전은 「같은 결과」로 판정하고 업그레이드와 섞지 않는다

서버는 버전을 고정한 lock 파일로 설치한다. 이전과 라이브러리 업그레이드를 한 번에 섞으면 문제가 생겼을 때 원인을 서버와 라이브러리로 가를 수 없다. 원본은 저장소가 아니라 서버 실물이다. 구 서버는 지우지 말고 정지해 원복 자리로 둔다. 서버의 저장소가 충돌하면 덮어쓰기 전에 diff로 내용이 같은지부터 본다. 서버에서 관리자 권한으로 저장소 파일을 만지지 않는다.

사례 · 아마 됐을 이전

무슨 일이 의존성이 전부 「이 버전 이상」으로만 적혀 있어서 새 서버에 OpenAI 호환 클라이언트 라이브러리가 2.x에서 3.x로 메이저 업그레이드돼 깔렸다. 방화벽 80/443 규칙과 환경 파일은 부트스트랩 스크립트가 만들지 않았고, 로컬의 환경 파일은 두 달 낡아 있었다. **왜 몰랐나** 코드가 쓰는 API가 3.x에도 대개 남아 있어 「아마 됐을」 것이다. 그래서 더 위험했다. 빠진 방화벽 규칙은 서버가 떠도 밖에서 붙지 않는 모양으로만 드러난다. **잡은 방법** 구 서버와 설치 패키지 목록을 비교했다. 이전 뒤 동일성 6항목(상태 확인, 지식 413건 = 413건, 규범 1314자, 환경 파일 해시, 패키지 35개 일치, 방화벽 8줄)과 실제 운영 루프 E2E(최종 승인 → 지식 413 → 418)로 판정했다.

사례 · 한 달 넘는 반쪽 체크아웃

무슨 일이 서버의 git pull 이 관리자 소유 파일 때문에 중간에 실패했다. 커밋 기준은 옛것인데 파일 내용은 새 버전인 반쪽 상태로 한 달 넘게 남았다. **왜 몰랐나** 파일 내용이 이미 최신이라 서버는 정상으로 동작했다. git 상태만 어긋나 있었다. **잡은 방법** 다음 pull이 「로컬 미커밋 변경 다수」로 충돌했다. 전 파일을 원격과 diff해 내용이 같음을 확인한 뒤 임시 보관 → pull → 복원으로 풀었다. 서버에만 있던 메모 파일도 보존했다.

감시와 회수는 감시받는 작업과 같이 죽지 않는 곳에 둔다

감시하는 쪽과 감시받는 쪽이 같이 죽으면 경보가 없다. 장시간 작업 전에 절전 같은 공통 정지 원인을 끄고, 끝나면 되돌릴지 확인한다. 감시가 침묵하면 「정상」이 아니라 「감시도 죽었을 수 있다」로 읽는다. 배포나

크래시가 끊은 작업은 아무도 「실행 중」에서 꺼내 주지 않는다. 서버가 뜰 때 회수하고, 평시에도 오래 멈춘 작업을 정리하는 리퍼를 둔다. 그 기준 시간은 정상 진행 중의 가장 긴 침묵보다 넉넉하게 잡는다.

사례 · 같이 잠든 감시

무슨 일이 로컬 PC에서 하루 넘게 돈 질의회신 수집이 PC 절전으로 두 번 멈췄다(01~05시, 16:27~20:54). 감시 도구도 같은 PC에서 같이 멈췄다. **잡은 방법** 두 번 멈춘 뒤 진행이 없는 것을 보고 알았다. 서버 쪽에서는 재시작이 백그라운드 작업을 끊는 문제를 막으려고 기동 시 회수와 평시 리퍼를 두었고, 기준을 30분으로 잡아 「가장 긴 정상 침묵 < 기준」을 확인했다. 즉시 실행 경로에서는 리퍼가 일하는 작업을 죽일 수 있다는 것도 그때 찾았다.

1.7 일하는 방식

마지막 절은 기능과 상관없이 매일 하는 일에 관한 것이다. 세션을 어떻게 나누는지, 규칙을 어떻게 고르는지, 기준을 어디에 두는지, 끝날 때 무엇을 남기는지. 여기서 생기는 틈은 몇 주 뒤에야 다른 모습으로 드러난다.

연구·채점과 제품 수정은 세션을 나누고, 경계는 구동 경로로 긋는다

이 제품은 대회 규정상 국내 LLM만 써야 했다. 같은 코드베이스에서 일하면 논문 채점 같은 오프라인 예외가 제품 코드로 번지기 쉽다. 판단 기준은 「LLM이 채점하는가」가 아니라 「제품 구동 경로 안의 추론 호출인가」다. 오프라인 예외는 연구 범위 명시 / 제출물 아님 / 런타임에서 import 안 함을 모두 만족할 때만 쓰고, 제품 세션에서는 아예 쓰지 않는다. 금지 호출을 막는 정적 검사가 없다면, 방어선이 문서와 사람의 주의력뿐이라는 사실도 적어 둔다.

사례 · 같은 세션의 벤치 하네스

무슨 일이 3방향 비교가 가능해지자 같은 세션에서 벤치마크 하네스가 제안됐다. 대표가 논문 작업과 제품 수정을 한 세션에 맡기는 위험을 지적했다. 외산 모델로 채점하는 것이 문제인지도 다시 따졌다. **처리** 정책 문서에 분리 원칙을 새로 적었다. → 2장

규칙과 구조는 건드리는 기존 로직이 적은 쪽으로 고르고, 미룰 때는 다시 열 조건을 숫자로 적는다

더 유연해 보이는 선택지는 기존 로직을 고치는 비용이 겉으로 안 드러난다. 운영 규칙을 고를 때 「이 선택이 건드리는 기존 로직」을 먼저 센다. 파생값은 입력받지 말고 계산만 하게 하면 합계 검증 없이도 불변식이 지켜진다. 큰 구조 교체는 교체 비용이 나중에 커지는지(경계가 있는지)와 제품의 논지와 축이 맞는지로 판단한다. 「지금 안 하면 나중에 힘들다」는 경계를 확인하지 않으면 늘 그럴듯하다. 미룰 때는 다시 열 조건을 숫자로 남긴다.

사례 · 크레딧 귀속과 정산 재원

무슨 일이 지식을 고쳤을 때 기여 크레딧을 누구에게 줄지 네 안 중 「원작성자 유지 + 수정 이력만 기록」을 골랐다. 정산 재원은 「총수익 × 분배 비율」로 계산만 하고 관리자가 덮어쓸 수 없게 했다. 크레딧 이동·분할이나 재원 직접 입력은 더 유연해 보였지만, 정산 로직을 고치거나 분배 합 검증을 새로 만들어야 했다.

사례 · 미룬 검색 엔진 교체

무슨 일이 그래프 기반 검색으로의 교체를 비채택으로 유지했다. 검색기는 이미 인터페이스 경계로 분리돼 있어 나중에 바뀌도 파이프라인을 안 건드린다. 그래프가 풀어 줄 뻔한 문제(중복 오탐)는 구간 단위 휴리스틱으로 풀렸고, 413건 규모에서는 이득이 없었다. **처리** 다시 열 신호 셋을 적었다. 지식 5,000~10,000건 도달, 직종 2개 이상, 지금 화면으로 못 푸는 실패 사례.

판단 기준은 한 곳에 하나만 두고, 기계와 사람이 같은 글을 읽게 한다

분류 범주는 이름만이 아니라 이름과 한 줄 정의로 준다. 분류기가 쓰는 기준과 사람이 화면에서 읽는 기준을 같은 글 하나로 둔다. 색·라벨처럼 여러 화면이 쓰는 매핑은 공용 모듈 하나에 두고, 범주가 바뀔 때 함께 고친다. 「다 다르게 보인다」 같은 판단은 눈 대신 계산으로 확인한다.

사례 · 이름만 준 분류기

무슨 일이 범주 이름만 나열한 첫 세목 분류기는 종신보험 수익자 지정과 자녀 명의 펀드 증여를 미분류로 흘렸다. 범주마다 설명 문장을 붙이자 종신보험·증여 펀드는 상속·증여로, 현금 매출 누락은 매출 관리로, 기부금은 기타로 바르게 갈렸다. **왜 몰랐나** 이름만으로도 대부분은 맞아서 미묘한 실패가 분포에 묻혔다. **처리** 같은 설명 문장을 화면의 범주 설명으로도 그대로 옮겼다.

사례 · 1° 차이의 두 색

무슨 일이 문자열 해시로 색상(hue)을 뽑아 라벨 18개를 칠하니 1~5° 간격의 쌍이 다섯이었다(출장비와 미분류는 1°). 황금각으로 미리 배정하자 최소 간격이 12°가 됐다(이론상 최대 20°). 두 화면이 색 로직을 따로 들고 있었고, 한쪽은 원시값을, 다른 쪽은 바꾼 라벨을 쓰고 있었다. **왜 몰랐나** 세목이 전부 미분류였을 때는 색이 하나뿐이라 충돌할 일이 없었다. **잡은 방법** 대표가 「전부 다른 색인지 체크해 달라」고 했고 실제로 계산했다. 요청이 없었으면 재지 않았을 값이다. → 6장

커밋되지 않는 작업일수록 끝날 때 기록과 「확인하지 못한 것」을 남긴다

제품 커밋 흐름 밖의 작업은 빠져도 아무것도 깨지지 않는다. 그래서 기록이 가장 쉽게 빠진다. 세션 끝에는 한 일과 함께, 확인하지 못한 것(라벨 없이 분포만 본 것, 실사용 표본이 없는 것, 발화한 적 없는 가드, 임시 공간에만 있는 백업)을 따로 적는다. 기록 습관은 제품 세션과 연구 세션을 구분하지 않고 같게 적용한다.

사례 · 남지 않은 논문 세션

무슨 일이 학회 논문 원고를 쓴 사흘간의 세션이 개발 기록을 남기지 않았다. 원고와 집계 파일은 로컬 전용 폴더에 있어 커밋에도 흔적이 없었다. **잡은 방법** 2주 뒤 봉인 대조를 개봉할 때 기록이 없어서, 세션 로그를 검색해 복원했다. → 7장

이스케이프나 한글이 든 명령은 셸 인라인 대신 파일로 쓴다

같은 명령이 이스케이프·한글이 없을 때는 잘 돈다. 깨진 결과는 엉뚱한 오류처럼 보인다. 이스케이프나 한글이 든 수정은 파일을 쓰거나 편집 도구로 한다. 한글 본문은 파일로 보낸다. 산출물은 프로그램 안에서 인코딩(UTF-8)을 명시해 쓴다. 알려진 함정은 문서에 적는 것만으로는 막히지 않는다.

사례 · 다섯 번 깨진 명령

무슨 일이 셸 인라인 파이썬의 `\b · \n` 이 무너져 문법 오류가 났다. 설계 문서에 적어 둔 합정을 또 뺐은 것이다. heredoc 안 파이썬 소스의 `\n` 이 실제 줄바꿈이 된 일이 세 번 있었다. 셸 리다이렉트로 쓴 한글 JSON이 cp949로 깨졌다. Windows Git Bash에서 `curl -d '한글'` 본문이 깨져 400이 났다. / 로 시작하는 인자는 경로로 바뀌었다. **처리** 한글 본문은 `--data-binary @파일` 로 보내고, 산출물은 코드 안에서 UTF-8로 쓰게 바꿨다.

작업 전후 점검표

- ☐ 이 측정에 쓴 함수에 폴백·SDK 재시도·기본 타임아웃이 있는가? 폴백이 삼킨 실패를 따로 썼는가?
- ☐ 측정을 제품과 같은 동시성으로 돌렸는가? 다른 측정과 같은 API 키를 동시에 쓰지 않았는가?
- ☐ 측정 질문이 바꾼 코드를 실제로 지나는가? 점검 도구가 서비스와 같은 설정을 읽는가?
- ☐ 시험이 고치기 전에 실패하는 것을 보았는가? 결과에 실행한 항목 수가 찍혀 있는가?
- ☐ 원시값을 남겼는가? 「0건」과 「당지 않음」, 「모름」과 false를 갈라 적었는가?
- ☐ 유지·후퇴 기준을 실험 전에 숫자로 적었는가? 계기판 숫자를 목표로 삼고 있지 않은가?
- ☐ 조건당 세 번 잰는가? 차이가 회차 편차보다 큰가? 비교하는 두 수치의 분모와 범위가 같은가?
- ☐ LLM 채점을 회차 간 절대치로 비교하고 있지 않은가? 봉인한 사람 대조로 오류 방향을 잰는가? 스캔 수치를 하한으로 적었는가?
- ☐ 다수 클래스 찍기 값과 단순 규칙 값을 비교선으로 두었는가? 표본이 0이거나 작은 것을 「문제 없음」으로 읽지 않았는가?
- ☐ 평균 다음에 건 단위로 겹쳐 보고, 걸린 원문을 직접 읽었는가?
- ☐ 이 결론의 근거(기억·앞 세션 결론·지시서·주석·필드 이름)를 원문과 코드로 다시 열어 보았는가?
- ☐ 프롬프트에 적은 개수·집합·금지 규칙을 출력 형식이나 코드 가드로도 강제했는가?
- ☐ 이 데이터를 누가 만들었는지 생산 경로로 적었는가? 외부로 나가는 자료의 출처 표기를 점검했는가?
- ☐ 권한을 역할별 토큰으로 직접 불러 실측했는가? 새 데이터를 읽는 소비자를 전부 적었는가?
- ☐ 프로덕션 쓰기가 dry-run 기본·먹등인가? 백업은 세션보다 오래 사는 자리에 있는가? 0행 갱신을 실패로 보는가? 끝나고 화면을 다시 열어 보았는가?

2 장

프로덕트 오너

무엇을 만들지, 무엇을 먼저 할지, 무엇을 안 할지, 누가 확정할지를 정하는 사람

이 장은 제품의 방향을 정하는 사람을 위한 것이다. 로드맵 항목을 고르고, 실험을 계속할지 접을지 정하고, 운영 규칙과 열람 범위를 결정하고, 사람이 확정해야 할 자리를 지정하는 사람이다. 사례는 한 AI 세무 상담 챗봇에서 나왔다. 사장님이 세무 질문을 하면 LLM이 근거 지식을 찾아 답한다. 이렇게 검색한 근거를 붙여 답을 만드는 방식을 RAG(검색 증강 생성)라고 한다. 세무사는 그 답을 검수하고, 검수한 내용은 다시 지식으로 들어간다. 모델은 국내 LLM API 하나로 만들었다. 이 프로젝트에서 의사결정권자 자리는 회사의 대표가 맡았다. 이 장에서는 그 자리를 「대표」라고 부른다. 이 자리의 실수는 대개 논리적으로 맞는 결론의 모습을 하고 있다. 결론은 맞는데 묻던 질문이 아니다. 숫자가 보이니 그 숫자가 목표가 된다. 자동 처리가 깔끔해 보이는데 누군가의 몫이 지워진다. 화면이 배정분만 보여 주니 열람이 막혀 있다고 믿는다. 모두 그럴듯하고, 오류 메시지는 하나도 나지 않는다. 그래서 이 장은 한 질문으로 모인다. **이 결정이 그럴듯하게 틀렸다면, 누가 언제 알아챌까?** 첫 태그가 PO인 교훈은 열세 개였다. 그중 일곱은 모든 직무에 통해 1장이 주인이다. 그래서 대표의 판단이 뚜렷하게 들어간 다른 직무의 교훈을 함께 끌어왔다. 모두 39개의 교훈을 여섯 갈래, 스물일곱 개의 원칙으로 묶었다. 질문을 바로 세우는 자리, 순서와 보류를 정하는 자리, 실험의 끝을 미리 긋는 자리, 운영 규칙을 고르는 자리, 사람이 확정하는 자리, 누가 무엇을 보는지 정하는 자리다.

이 장의 원칙

1. 재검토하기 전에 「이 항목을 왜 시작했나」를 한 문장으로 확인한다
2. 「얼마나 잘 되나」를 묻기 전에 「그 경로에 있기는 한가」를 묻는다
3. 쓰는 사람의 한 줄 질문은 재지 않던 값을 재게 한다 — 물었으면 숫자로 답한다
4. 데이터의 정체는 기억과 정황이 아니라 기록과 원문 전수로 정한다
5. 리드타임이 가장 긴 작업을 먼저 시작한다
6. 데이터를 쓰는 쪽을 먼저 설계하고, 공급하는 배선은 그다음에 깎는다
7. 지금 안 하는 일은 다시 열 조건을 숫자로 적어 둔다
8. 원칙에는 조건을 함께 적고, 조건이 차면 근거를 들어 바꾼다
9. 자원과 비용은 코드의 실제 의존성과 실측으로 답한다
10. 실험을 시작하기 전에 유지·후퇴 기준을 숫자로 선언한다
11. 빌드에서 증명으로 넘어갈 때 완성 기준을 문장으로 박는다
12. 목표 지표에 실패한 조치도 다른 지표로 다시 보고 살릴 것은 넘긴다
13. 지식 소스는 「무엇이 비었나」를 수치로 보인 뒤 하나씩 더한다
14. 운영 규칙은 다른 로직을 안 건드리는 가장 단순한 쪽으로 정한다
15. 새 분류 체계는 이미 쓰이는 범주의 이름을 그대로 품는다

16. 「한 번에 하나만」 같은 운영 규칙은 DB 제약에 맡긴다
17. 오래 걸리고 되돌리기 어려운 작업은 기본을 예약으로 둔다
18. 보호 장치가 멈춘 것을 「실패」라고 부르지 않는다
19. 정책이 없는 데이터는 자동 처리하지 말고 경고한 뒤 사람이 정한다
20. 사람이 손댄 것은 자동 회차가 뒤집지도, 치우지도 않는다
21. 잔여물은 버리지도 섞지도 말고, 사람 눈에만 보이는 칸에 둔다
22. 모든 답변에 들어가는 문안은 사람이 확정하고, 넘치면 멈춘다
23. 거버넌스 규칙보다 신원을 먼저 세운다
24. 지우는 범위는 날짜가 아니라 상태로, 쓰기는 계측과 확인 뒤에 한다
25. 열람 범위는 화면 관행이 아니라 강제여야 한다 — 역할별로 직접 센다
26. 권한은 DB가, 표시는 화면이 정한다 — 권한을 여는 데이터에는 닫는 절차를 둔다
27. 밖으로 나가는 자료는 출처 표기를, 제품 경로는 규정의 경계를 따로 점검한다

2.1 질문을 바로 세운다

대표가 가장 자주 바로잡은 것은 답이 아니라 질문이었다. 실무자가 내놓은 결론은 대개 논리적으로 맞았다. 다만 대표가 묻던 질문에 대한 답이 아니거나, 질문의 전제가 이미 깨져 있었다. 이런 결론은 반박할 틈이 없어서 오래 살아남는다. 반대로 대표가 쓰는 사람으로서 던진 짧은 질문 한 줄은 아무도 재지 않던 값을 재게 했고, 몇 번은 로드맵 전체의 출발점이 됐다.

재검토하기 전에 「이 항목을 왜 시작했나」를 한 문장으로 확인한다

로드맵 항목의 이름은 종종 기술 이름이다. 기술 이름은 목적을 가린다. 그 이름만 보고 재검토하면 「이 기술이 성능에 이득인가」라는 엉뚱한 질문에 정확히 답하게 된다. 재검토 전에 시작 동기를 한 문장으로 적어 대표에게 확인받는다. 맞는 답도 묻던 질문이 아니면 쓸모가 없다. 이 원칙의 일반형은 → 1.4에서 다룬다.

사례 · 그래프 항목

무슨 일이 로드맵에 「GraphRAG」라는 항목이 있었다. GraphRAG는 지식을 노드와 연결선의 그래프로 엮어 검색하는 방식이다. 실무 쪽은 이것을 「검색 엔진을 GraphRAG로 바꿀까」로 읽었다. 성능과 가치 두 관점에서 따져 비채택으로 결론지었다. 대표가 결론을 듣자마자 정정했다. 이 항목의 동기는 「세무사가 지식 베이스를 거미줄 그래프로 보고 직접 고치는 중간층」이었다. **왜 몰랐나** 항목 이름이 기술 이름이라 목적이 문서에서 흐려져 있었다. 재검토 결론 자체는 논리적으로 맞아서 의심할 틈이 없었다. **잡은 방법** 대표의 정정 뒤 저장된 연결 정보로 전체 그래프(노드 413개, 연결 3,304개)를 그려 세무사 화면에 붙였다. 검색 엔진 교체는 따로 비채택으로 남겼다.

「얼마나 잘 되나」를 묻기 전에 「그 경로에 있기는 한가」를 묻는다

품질을 걱정하는 질문에는 전제가 숨어 있다. 그 기능이 실제로 제품 경로에서 쓰이고 있다는 전제다. 화면에 보이는 지식과 모델이 실제로 받는 지식은 다를 수 있다. 질문에 답하기 전에 코드에서 그 경로가 성립하는지부터 본다. 전제가 깨져 있으면 질문 하나가 로드맵 하나를 연다. 일반형은 → 1.4에서 다룬다.

사례 · 사전형 지식

무슨 일이 대표는 세무사가 보는 용어 사전형 지식의 「매칭률」을 걱정했다. 실제로는 그 지식이 프론트엔드에 박힌 정적 데이터라 검색에 아예 쓰이지 않았다. 품질 기여가 0이었다. 적재 함수는 있었지만 그것을 부르는 곳이 없었다. **왜 몰랐나** 세무사 화면에는 그 지식이 멀쩡히 보였다. 화면에 있으면 쓰이고 있다고 여기게 된다. **잡은 방법** 백엔드 전체를 검색해 호출자가 없음을 확인했다. 이 진단이 「상시 규범 · 용어 사전 · 문장 사전」의 3층 검색 로드맵의 출발점이 됐다. 이 로드맵은 여덟 단계로 배포까지 갔다.

쓰는 사람의 한 줄 질문은 재지 않던 값을 재계 한다 — 몰랐으면 숫자로 답한다

대표는 제품의 첫 번째 실사용자이기도 하다. 「왜 전부 한 색이냐」, 「전부 다른 색인지 체크해 달라」 같은 질문은 사소해 보인다. 그러나 이 질문들은 테스트가 보지 않던 자리를 가리킨다. 이런 질문에는 눈대중이 아니라 계산과 데이터로 답한다. 화면 표시를 고치기 전에 데이터를 먼저 본다. 질문이 없었으면 아무도 재지 않았을 값이 여기서 나온다.

사례 · 한 색 그래프

무슨 일이 지식 그래프를 띄우자 대표가 「왜 전부 한 색이냐」고 물었다. 색은 세무 항목 분류(세목)를 뜻했다. 화면을 의심하기 전에 데이터를 봤다. 대화를 복원하는 코드 한 줄이 세목을 늘 「미분류」로 박아 넣었고, 그 값이 적재까지 흘러가 지식 521건 전부가 미분류였다. **왜 몰랐나** 화면에는 「미분류」 묶음 하나가 정상처럼 보였다. 세목으로 묶어 보여 주는 화면이 생기기 전까지 그 값을 읽는 사람이 없었다. **잡은 방법** 서버가 적재 시점에 실제 값으로 덮어쓰게 하고, 413건을 소급 재분류했다. 미분류는 100%에서 9%가 됐다. 세부 처리는 → 4장.

사례 · 「다 다른 색인가」

무슨 일이 재분류 뒤 대표가 「전부 다른 색인지 체크해 달라」고 했다. 실제로 계산하니 18개 라벨 중 색상각이 1~5도밖에 안 떨어진 쌍이 다섯이었다. 출장비와 미분류는 1도 차이였다. **왜 몰랐나** 세목이 전부 미분류일 때는 색이 하나뿐이라 충돌할 일이 없었다. 눈으로는 「대충 다르다」로 보였다. **잡은 방법** 색을 미리 배정하자 최소 간격이 12도가 됐다. 요청이 없었으면 재지 않았을 값이다.

사례 · 검색 단위

무슨 일이 대표가 「지금 지식도 코사인으로 찾아, 백서 형식에도 통하나」고 물었다. 코사인 검색은 글을 숫자 벡터로 바꾸는 임베딩끼리 방향의 가까움을 재는 방식이다. 코드를 열어 보니 지식은 제목과 본문 전체를 통째로 한번 임베딩하고 있었다. 긴 글을 통째로 임베딩하면 정보가 희석된다. **왜 몰랐나** 검색이 어쨌든 결과를 내니 결함이 드러나지 않았다. **잡은 방법** 이 질문에서 지식을 문장 단위로 다시 쓰는 작업이 시작됐다. 이후 이 문서에서는 이것을 「문장 단위로 다시 쓴 지식 사전」이라고 부른다.

사례 · 낡은 그래프

무슨 일이 그래프의 연결선은 5분마다 한 번 다시 계산됐다. 대표가 「새 지식이 들어올 때마다 재계산이 필요하지 않겠냐」고 물었다. **왜 몰랐나** 주기 작업이 돌고 있으니 언젠가는 맞는다. 낡은 구간은 짧아서 테스트에서 안 보인다. **잡은 방법** 수백 건 규모에서 전체 재계산이 1초 미만임을 재고, 지식을 바꾸는 네 경로 모두에서 바로 재계산하게 했다. 주기 작업은 안전망으로 남겼다.

데이터의 정체는 기억과 정황이 아니라 기록과 원문 전수로 정한다

대표의 기억은 강한 근거처럼 들린다. 정황이 기억에 들어맞으면 더 그렇다. 그러나 기억과 정황이 서로를 받치면 가설은 의심받지 않는다. 데이터를 지울지, 비싼 재작업을 할지 정하기 전에 같은 질문을 전에 조사한 문서가 있는지 찾고, 원문을 전수로 다시 본다. 정체가 확정돼야 지울지 말지를 논한다. 일반형은 → 1.4에서 다룬다.

사례 · 논문 데이터라는 기억

무슨 일이 관리자 「완료」 화면에 7월 중순 옛새치 미확정 검수 항목이 쌓여 있었다. 대표는 이것을 학회 논문 실험 데이터로 기억했다. 실제로는 제품 개발 작업(100건 일괄 감사, RAG 재검증)이 남긴 검수 대기열이었다. 논문이 DB에 남긴 흔적은 지식 168건의 상태 변경 하나뿐이었고, 그것도 이미 복구돼 있었다. **왜 몰랐나** 같은 주에 RAG 켜고 끄기 비교 리포트가 연달아 나왔다. 그 리포트들을 논문 실험 기록으로 묶은 커밋도 있었다. **잡은 방법** 이전 세션이 같은 질문을 조사해 남긴 문서를 찾아 읽었다. 미확정의 진짜 원인(접근 정책 버그)도 7월 기록에서 확인했다.

사례 · 재적재 직전

무슨 일이 이전 단계 기록은 「다른 사안의 수치가 답에 섞이는 원인은 지식 묶음 속 옛 AI 답변」이라고 적었다. 그 처방으로 지식 전체를 다시 적재하려 했다. 대표가 재적재를 정하기 전에 혼입 원인부터 특정하라고 했다. **왜 몰랐나** 근거로 삼은 다섯 건은 그 결론과 모순되지 않았다. 다른 설명을 시험하지 않았을 뿐이다. **잡은 방법** 섹션별로 전수 검사하니 같은 수치가 질문 칸에도 있었다. 남의 사안 수치는 세 섹션에 흩어져 있었다. AI 답변 제거는 무효였고 재적재도 필요 없었다.

2.2 무엇을 먼저, 무엇을 지금 안 할지

순서를 정하는 일은 대표의 몫이다. 순서가 틀리면 아무것도 깨지지 않는다. 일이 진행되는 것처럼 보일 뿐이다. 켜기만 하면 되는 작업, 배선만 하면 되는 작업은 진척처럼 보인다. 반대로 「지금 안 한다」는 결정은 기록하지 않으면 「영원히 안 한다」나 「잊었다」와 구별되지 않는다. 이 절은 순서와 보류를 정하는 원칙이다.

리드타임이 가장 긴 작업을 먼저 시작한다

다음 작업은 「끝나야 다른 일을 시작할 수 있고 오래 걸리는 것」부터 고른다. 데이터 작업은 대개 수집 → 검수 → 적재로 이어져 가장 길다. 바깥에서 데이터가 오기를 기다리는 계획이라면, 우리가 직접 만들 수 있는지부터 따진다.

사례 · 켜기나 수집이나

무슨 일이 남은 일이 둘이었다. 새 파이프라인 서버 유닛을 켜는 것과, 답을 쓰는 뒷단 모델이 쓸 판례·해석 원문 지식을 모으는 것이다. 수집을 먼저 했다. 수집 → 세무사 승인 → 적재의 리드타임이 가장 길고, 뒷단 모델 설계의 전제였다. 서버 유닛은 켜도 답변이 판정 없는 임시 자문이라 얻는 게 적었다. 같은 주에 앞단 모델용 학습 사례도, 원문 지식도 「밖에서 받는다」에서 「우리가 만든다」로 바뀌었다. **왜 몰랐나** 켜기만 하면 되는 작업이 진척처럼 보인다. **잡은 방법** 남은 일을 리드타임과 의존 관계로 줄 세웠다. 10월 초 프로덕션 챗은 이렇게 모은 원문 지식만으로 답하게 됐다.

데이터를 쓰는 쪽을 먼저 설계하고, 공급하는 배선은 그다음에 간다

배선을 먼저 하면 일은 진행돼 보인다. 그러나 나중에 데이터를 부르는 모양이 달라지면 다시 짤다. 데이터 경로를 깔기 전에 그 데이터를 쓰는 단계의 호출 모양부터 정한다. 같은 이유로, 새 데이터를 기존 테이블에 넣기 전에 그 테이블을 읽는 소비자를 모두 적어 본다.

사례 · 배선과 답변 단계

무슨 일이 원문 지식을 검색에 잇는 배선과, 그 지식으로 답을 쓰는 뒷단 단계의 설계 중 무엇을 먼저 할지 정해야 했다. 대표는 답변 단계 설계를 먼저 골랐다. 그 단계가 지식을 부르는 모양이 검색 갈래 구조를 정하기 때문이다. **왜 몰랐나** 배선부터 하면 그 주의 진척은 더 커 보인다. **잡은 방법** 설계를 먼저 하자 「판례를 따로 부르지 않는다」는 결정이 나왔다. 배선을 먼저 했다면 판례 전용 갈래를 깔았다가 걷어 냈을 것이다.

사례 · 같은 테이블에 넣기

무슨 일이 설계 문서는 새 원문 지식을 기존 지식 테이블에 출처 표시만 달리해 넣기로 돼 있었다. 그 테이블은 문장 사전 합성, 검수 선례 검색, 지식 그래프가 모두 읽고 있었다. **왜 몰랐나** 넣는 순간에는 아무것도 깨지지 않는다. 다른 갈래가 새 지식을 제 것처럼 쓰기 시작할 뿐이다. **잡은 방법** 실제 테이블 구조를 확인한 단계에서 대표가 별도 스키마로 정정했다. 세부는 → 4장.

지금 안 하는 일은 다시 열 조건을 숫자로 적어 둔다

「지금 안 하면 나중에 힘들다」는 말은 늘 그럴듯하다. 이 말이 맞는지는 교체 지점에 이미 경계가 있는지로 판단한다. 경계가 있으면 나중에 바뀌도 비용이 크게 늘지 않는다. 그다음은 그 교체가 제품의 논지와 같은 축인지 본다. 미루기로 했으면 다시 열 조건을 숫자로 남긴다. 그래야 「보류」가 「잊음」이 되지 않는다. 일반형은 → 1.7에서 다룬다.

사례 · 검색 엔진 교체 보류

무슨 일이 그래프 기반 검색 엔진으로의 교체를 비채택으로 유지했다. 검색 모듈은 이미 인터페이스 하나로 분리돼 있어서 나중에 바뀌도 파이프라인을 안 건드린다. 그래프가 풀어 줄 뻔했던 문제(중복 오답)는 다른 휴리스틱으로 풀렸다. 지식 413건 규모에서는 이득이 없었다. **왜 몰랐나** 「나중엔 늦는다」는 주장은 경계를 확인하기 전까지 반박되지 않는다. **잡은 방법** 다시 열 신호 셋을 적었다. 지식이 5,000~10,000건에 이르렀을 때, 지원 업종이 둘 이상이 됐을 때, 지금 화면으로 못 푸는 실패 사례가 나왔을 때다.

원칙에는 조건을 함께 적고, 조건이 차면 근거를 들어 바꾼다

많은 원칙이 사실은 「무엇이 입증될 때까지」의 조건부다. 조건을 빼고 무조건형 문장으로 적으면 영구 규칙처럼 읽힌다. 그러면 조건이 채워져도 아무도 바꾸지 못한다. 원칙을 적을 때 「언제까지, 무엇이 입증되면」을 같이 적는다. 바꿀 때는 통과선과 되돌리기 조건을 먼저 정하고, 되돌리기 쉬운 자리에서 바꾼다. 일반형은 → 1.4에서 다룬다.

사례 · 기본 검색 전환

무슨 일이 「기본 검색 방식은 어느 단계에서도 바꾸지 않는다」는 원칙이 있었다. 다시 보니 이 원칙은 「입증 전까지」였다. 여러 갈래 비교 벤치와 판정형 질문 192회로 조건이 채워졌다. **왜 몰랐나** 원칙이 무조건형 문장으로 적혀 있었다. **잡은 방법** 대표가 원칙의 원래 이유를 되짚었다. 통과선은 「여러 갈래를 합친 검색의 오염이 기존 검색 이하」로, 되돌리기 조건은 「스모크 이상이면 즉시, 이후는 실사용 대화로 재판정」으로 정했다. 코드 기본값은 그대로 두고 서버 설정 한 줄로 바꿨다. 로컬과 벤치는 영향을 받지 않았다.

자원과 비용은 코드의 실제 의존성과 실측으로 답한다

큰 자원을 주겠다는 제안은 「받아 두면 손해 없다」처럼 보인다. 그러나 실제 병목은 그 질문에 가려진다. 자원 신청은 의존성 목록과 실측 병목으로 근거를 댈다. 비용 계획은 무료 한도를 먼저 빼고 단가를 곱한다. 확정은 견적이 아니라 청구 실측으로 한다.

사례 · GPU 제안

무슨 일이 대회 측의 인프라 조사에 GPU는 신청하지 않고 스토리지 200GB만 답했다. 코드를 전수로 보니 LLM·임베딩·벡터 검색이 모두 외부 API와 관리형 DB였다. GPU 라이브러리는 하나도 없었다. 실제 병목은 GPU가 아니라 서버 메모리 1GB였다. **왜 몰랐나** GPU 질문이 메모리 문제를 가렸다. **잡은 방법** 의존성 파일과 로컬 파일 의존을 검색했다. 고도화 계획(에이전트형 검색, 그래프 검색)도 API 호출만으로 된다는 것을 방식별 표로 확인했다.

사례 · 10배 과대 견적

무슨 일이 서버 증설 계획은 월 37,000원이었다. 실제 청구는 컴퓨트 0달러, 부트 볼륨 2.76달러였다. 10배 과대였다. 새 사양(2코어, 12GB)이 클라우드 상시 무료 한도(4코어, 24GB) 안에 통째로 들어갔다. **왜 몰랐나** 단가표의 숫자는 맞았다. 무료 한도를 빼는 단계가 계획에 없었다. **잡은 방법** 생성 화면 견적에 컴퓨트 항목이 없는 것을 보고 사용량 실측으로 확인했다. 예산 알림은 정상 청구의 몇 배로 잡아 알림 자체가 이상 신호가 되게 했다.

2.3 실험의 끝을 미리 긋는다

LLM 실험은 끝나지 않는다. 회차마다 「조금만 더 고치면」이 가능해 보이고, 실제로 어떤 숫자는 조금씩 오른다. 채점기는 그럴듯한 일반론에 관대하다. 그래서 언제 멈출지는 실험 결과가 아니라 실험 전에 대표가 정해야 한다. 이 절은 유지와 후퇴, 완성과 계기판, 소스 확장을 가르는 기준을 다룬다.

실험을 시작하기 전에 유지·후퇴 기준을 숫자로 선언한다

모델이나 프롬프트를 개선하는 실험은 시작 전에 「어디를 넘으면 유지, 어디 밑이면 물러선다」를 숫자로 적는다. 결과가 나오면 그 기준에 대 보고 그대로 따른다. 물러설 때는 LLM이 못 하는 일을 결정적 로직으로 옮긴다. 결정적 로직이란 같은 입력에 늘 같은 결과를 내는 규칙이다. 그 대가도 명시하고 받아들인다. 일반형은 → 1.2에서, 실험 설계는 → 3장에서 다룬다.

사례 · 앞단 모델의 역할 축소

무슨 일이 이 제품에는 질문을 받아 사실을 뽑고 부족하면 되묻는 앞단 모델이 있다. 이 모델이 「정보가 충분한가」까지 판정할 수 있는지 재기 전에 기준을 정했다. 중간 vs 충분 구분이 72%(질문 길이만 보는 기준선)를 확실히 넘으면 유지, 세 가지 개선을 다 해도 우연 수준이면 역할 축소. 7개 방식 × 446건을 돌린 결과 남은 회차가 없었다. 체크리스트 방식 50.8%, 정답을 미리 준 천장 실험 68.6%, 다른 회사 상위 모델의 블라인드 판정 54.2%였다. **왜 몰랐나** 회차마다 개선이 보였다. 천장 실험 개정판은 항목 정확도를 50.7%에서 73.2%로 올렸다. 기준이 없었다면 개선 회차가 끝없이 이어졌을 것이다. **잡은 방법** 하루 전에 기준을 문서로 선언했고, 대표가 결과를 그 기준에 대 보고 축소를 확정했다. 앞단 모델은 추출과 되묻기만 맡는다. 충분 판정은 LLM 없는 상한(한 턴에 질문 2개, 최대 2턴)이 맡는다. 대가는 정보를 다 준 사용자도 한 턴은 질문을 받는다는 것이다. 원인은 모델이 아니라 과제 정의에 있었다. 모델을 바꿔서 풀리는 문제가 아니었다.

빌드에서 증명으로 넘어갈 때 완성 기준을 문장으로 박는다

숫자가 보이면 그 숫자를 목표로 삼게 된다. 기능을 다 만든 뒤에는 「무엇이 되면 끝인가」를 한 문장으로 적는다. 그래야 계기판의 숫자를 목표에서 내릴 수 있다. 표본이 없는 지표는 0%가 아니다. 표본이 없으면 패널을 만들지 않고, 표본이 20 미만이면 비율로 인용하지 않는다. 일반형은 → 1.2에서 다룬다.

사례 · 커버리지라는 계기판

무슨 일이 문장 사전의 기능 열아홉 개를 모두 배포한 뒤에도 커버리지를 올리는 작업이 이어지고 있었다. 커버리지는 질문 중 사전 문장이 붙은 비율이다. 자동 경로의 천장인 63~65%에 이미 도달해 있었다. **왜 몰랐나** 계기판과 목표의 구분이 없었다. 오르는 숫자는 늘 일처럼 보인다. **잡은 방법** 로드맵을 정리하며 완성 기준을 한 줄로 적었다. 「세무사가 하루 쓰면 사전이 그만큼 좋아져 있고, 다음 회차가 그것을 지우지 않으며, 지표가 편차 안에서 재현된다.」 이것을 재현성 · 축적성 · 임팩트 세 관문으로 나눴다. 커버리지는 계기판으로 내렸다.

목표 지표에 실패한 조치도 다른 지표로 다시 보고 살릴 것은 넘긴다

실험의 목표 지표만 보면 실패한 회차는 통째로 버려진다. 실험 기록에는 목표 지표 외에 지연·실패율·출력 길이를 같이 남긴다. 목표에 실패한 조치라도 운영 지표에 이득이 있으면 따로 제품에 넘긴다. 유지·후퇴 기준과 짝을 이루는 원칙이다. 물러서는 결정이 쓸모 있는 부산물까지 버리지 않게 한다.

사례 · 출력 길이 상한

무슨 일이 앞단 모델이 한 번에 나열하는 항목을 3개로 묶는 조치는 판정 정확도에 효과가 없었다(중간 vs 충분 49.2%). 그러나 출력이 중간에 잘리는 퇴화가 37건에서 0건이 됐고, 지연 중앙값은 1.0초였다. **왜 몰랐나** 목표 지표로는 「실패한 회차」였다. **잡은 방법** 회차마다 지연·잘림·나열 개수를 같이 기록해 뒀기 때문에 이득이 보였다. 출력 길이 상한을 제품 호출에 넘겼다.

지식 소스는 「무엇이 비었나」를 수치로 보인 뒤 하나씩 더한다

소스의 성격이 그 소스가 답할 수 있는 질문을 정한다. 판례는 분쟁의 기록이라 다툼이 드문 주제는 담지 않는다. 같은 출처 계열이라고 같은 성질을 가정하지 않는다. 소스를 더하기 전에 무엇이 비었는지 키워드별 적중률로 먼저 보이고, 소스마다 검색 방식과 적중률을 따로 잔다. 대표는 이 숫자로 다음 소스를 고른다. 새 지식을 넣을 때는 계측을 먼저 깬다. 계측 없이 넣으면 품질이 왜 바뀌었는지 영영 모른다.

사례 · 판례에 없는 공제

무슨 일이 판례 요지 487건을 모았다. 전체 키워드 적중은 63%였는데, 의료비·월세·교육비 공제 같은 키워드는 0/10에서 1/17이었다. **왜 몰랐나** 판례는 본문 검색이라 본문 한 줄로도 걸려 건수는 채워졌다. 제목 적중 0이 신호였다. **잡은 방법** 키워드별 적중률을 따로 썼다. 대표는 그 숫자로 두 번째 소스를 질의회신으로 정했다. 질의회신은 제목 검색이라 적중이 100%였다. 넓히고 나자 일반인 질문 표본에서 쓸모 있는 근거가 붙는 비율이 25.0%에서 54.2%로 올랐다.

사례 · 확장 6규칙

무슨 일이 근거 분포를 개수로만 남기면 「왜 그 근거가 들어왔나」를 되짚지 못했다. 응답에 근거 출처가 없어 사전증이 실제로 끼었는지 확인할 수 없던 공백도 있었다. **왜 몰랐나** 요약 수치만으로도 대시보드는 그럴듯하게 돈다. **잡은 방법** 외부 지식을 대량으로 넣을 때 지킬 여섯 규칙을 정했다. 계측 먼저 · 정밀도 감시 · 한 번에 한 소스 · 라이선스 · 권위 등급 · 소급 가능. 세부는 → 4장.

2.4 운영 규칙은 단순하게, 강제는 아래층에서

운영 규칙을 고를 때 더 유연한 선택지는 늘 매력적이다. 하지만 유연함의 비용은 곁에 드러나지 않는다. 기존 로직을 고치고, 새 검증을 만들고, 화면이 못 막는 경합을 떠안는다. 이 절은 규칙을 고르는 기준과, 정한 규칙을 어디서 강제할지, 그리고 그 규칙이 사용자에게 어떻게 보여야 하는지를 다룬다.

운영 규칙은 다른 로직을 안 건드리는 가장 단순한 쪽으로 정한다

운영 규칙을 고를 때 「이 선택이 건드리는 기존 로직」을 먼저 센다. 파생값은 입력받지 말고 계산만 하게 한다. 그러면 합계 검증 없이도 불변식이 지켜진다. 일반형은 → 1.7에서 다룬다.

사례 · 수정한 지식의 크레딧

무슨 일이 이 제품은 세무사가 기여한 지식이 살아 있는 기간에 따라 수익을 나눈다. 누군가 지식을 고치면 크레딧을 누구에게 줄지, 선택지가 넷이었다. 대표는 「원작성자 유지 + 수정 이력만 기록」을 골랐다. 정산 재원은 「총수익 × 분배 비율」로 계산만 하고 관리자가 덮어쓸 수 없게 했다. **왜 몰랐나** 크레딧 이동·분할이나 재원 직접 입력이 더 유연해 보였다. 그러려면 정산 로직을 고치고 분배 합 검증을 새로 만들어야 한다는 비용이 안 보였다. **잡은 방법** 대표가 선택지를 나란히 놓고 비교해 확정했다.

새 분류 체계는 이미 쓰이는 범주의 이름을 그대로 품는다

판정이나 집계에 이미 쓰이는 범주가 있으면 새 분류 체계는 그 이름을 그대로 품고 확장한다. 백지에서 짜면 두 경로가 다른 이름을 쓰게 되고, 경로를 잇는 자리마다 번역이 필요해진다. 추가 범주는 실제 데이터의 반복 주제와 실측 비율로 정한다.

사례 · 세목 17개

무슨 일이 지식을 묶을 세목을 17개로 정했다. 이 제품에는 경비 처리 여부를 규칙으로 판정하는 엔진이 있다. 그 엔진의 지출 유형 9종을 그대로 넣고, 자문형 8종을 더했다. 자문형은 엔진이 판정 못 하는 질문이 실측 55%였다는 데서 나왔다. 8종의 재료는 질문 407건을 50건 간격으로 훑어 찾은 반복 주제였다. **왜 몰랐나** 새 체계를 백지에서 짜면 판정형 질문과 지식 묶음이 서로 다른 이름을 쓴다. 그 비용은 한참 뒤에 나타난다. **잡은 방법** 대표가 「엔진 9종을 그대로 포함」을 원칙으로 확정했다. 이후 자동 분류와 소급 재분류로 미분류는 100%에서 9%가 됐다.

「한 번에 하나만」 같은 운영 규칙은 DB 제약에 맡긴다

경합은 드물다. 드물어서 화면 검사로는 안 나온다. 생기면 어느 쪽이 반영될지 아무도 모른다. 이런 운영 규칙은 앱 코드 대신 DB 제약에 맡긴다. 지식 변경은 대기열을 거쳐, 승인 전까지 검색 결과를 흔들지 않게 한다. 구현은 → 5장.

사례 · 수정 제안 대기열

무슨 일이 세무사의 지식 수정 제안을 대기열로 받았다. 승인 전까지는 원문이 그대로 검색되고, 관리자가 승인할 때 임베딩을 다시 만든다. 지식 한 건당 대기 중인 제안은 DB의 고유 제약으로 1건만 허용했다. **왜 몰랐나** 경합하는 제안 두 건은 데모 규모에서 생기지 않는다. **잡은 방법** 미리 막고, 실제 DB에서 제안 → 반려, 제안 → 승인 → 원복까지 돌려 원작성자 기록이 바뀌지 않음을 확인했다.

오래 걸리고 되돌리기 어려운 작업은 기본을 예약으로 둔다

버튼 한 번으로 큰 작업에 들어가면 안 된다. 되돌리기 어렵거나 오래 걸리는 작업은 기본값을 지연·예약으로, 즉시 실행은 한 번 더 묻는 형태로 둔다. 예약 시각은 사용자의 시간대로 계산한다.

사례 · 지식 재구조화

무슨 일이 문장 사전의 목차를 다시 짜는 재구조화는 수십 분이 걸리고 사전 한 세대를 통째로 갈아 끼운다. 대표의 기준은 「세무사가 버튼 한 번으로 큰 작업에 들어가면 안 된다」였다. **왜 몰랐나** 예방 원칙이다. 사고가 나기 전에는 즉시 실행 버튼이 편리할 뿐이다. **잡은 방법** 기본은 야간 예약, 즉시 실행은 확정형으로 했다. 합성 트리거는 관리자 전용, 큰 목차 재배치는 별도의 [확정] 버튼이다.

보호 장치가 멈춘 것을 「실패」라고 부르지 않는다

상태 이름은 시스템의 사실이 아니라 사용자에게 일어난 일로 짓는다. 가드가 작업을 멈췄다면 사용자의 지식은 지켜진 것이다. 그것을 빨간 「실패」로 그리면 사용자는 정반대로 읽는다. 통과한 회차라도 실패 건수가 0이 아니면 보여 준다. 화면 설계는 → 6장.

사례 · 중단된 회차

무슨 일이 분류 가드가 재구조화 회차를 멈추면 화면에 빨간 「실패」가 떴다. 재구조화 실패를 보면 세무사는 지식이 비었을까 봐 놀란다. 가드는 정확히 그 반대를 한 것이다. **왜 몰랐나** 작업 상태가 error라는 사실만 보면 실패 표시가 자연스럽다. **잡은 방법** 호박색 「중단됨 — 기존 세대를 지켰습니다」로 바꾸고 이번 값, 직전 값, 요구 값, 사유를 함께 보여 줬다.

2.5 사람이 확정하는 자리

이 제품의 지식은 사람의 판단으로 자란다. 그런데 자동화는 늘 사람의 판단을 덮어쓸 기회를 가진다. 중복을 지우고, 잔여물을 치우고, 목차를 다시 만들고, 문안을 요약한다. 하나하나를 깔끔해 보인다. 대표가 정해야 할 것은 어느 자리를 사람이 확정하고, 자동화는 그 확정을 어떻게 존중하는지다. 그리고 「사람이 승인한다」는 규칙이 무엇 위에서 있는지다.

정책이 없는 데이터는 자동 처리하지 말고 경고한 뒤 사람이 정한다

지우면 누군가의 몫이 사라지는 데이터가 있다. 그 몫을 어떻게 나눌지 정책이 정해지기 전에는 자동으로 지우거나 합치지 않는다. 대신 경고를 띄우고 사람이 판단하게 한다. 경고 기능이 실패해도 본 흐름은 막지 않는다.

사례 · 중복 적재된 코멘트

무슨 일이 같은 질문·답변에 검수 코멘트 3건이 중복 적재된 것을 발견했다. 중복 판정 키가 내용이 아니라 원본 ID 기준이라, 같은 내용이 따로따로 쌓였다. **왜 몰랐나** 자동 처리는 깔끔해 보인다. 그러나 누구의 기여를 지울지 정할 기여 정책이 그때는 없었다. **잡은 방법** 대표가 「기여 정책이 없으니 자동 처리는 위험하다」고 판단했다. 내용 기준 자동 거절 대신, 검수 단계에서 85% 이상 비슷하면 경고를 띄웠다. 소급 정리도 관리자가 화면에서 직접 확정했다 (활성 지식 526건 → 413건).

사람이 손댄 것은 자동 회차가 뒤집지도, 치우지도 않는다

사람이 손댄 구조는 자동 회차가 보존한다. 고침도 사람이 한다. 보호 장치를 만들면 그것이 모든 갱신 경로를 막는지 확인한다. 되돌릴 지식이 없고 사람이 언제든지 옮길 수 있는 영역은 가드로 막기보다 계속으로 지켜본다. 재생성 파이프라인의 일반형은 → 1.5에서 다룬다.

사례 · 한 회차 만의 원상복귀

무슨 일이 세무사가 「기타」에서 다른 목차로 옮긴 문장이 정확히 한 회차 만에 제자리로 돌아갔다. 목차를 매번 전부 다시 만들고 이전 세대를 보관함으로 보내는 구조였다. 사람이 고친 것을 지키는 플래그는 옛 경로 하나만 막고 있었다. 지식이 자라지 않고 매번 다시 만들어지고 있었다. **왜 몰랐나** 보관은 정상 동작이었다. 사람이 손댄 양이 작아 비용이 안 보였다. **잡은 방법** 옮긴 문장의 운명을 재구조화 직후 DB에서 추적했다. 이후 손댄 문장은 모두 보존하고 가장 가까운 새 목차에 다시 붙였다. 프로덕션 두 회차에서 보존 3/3, 재부착 2/3이었다. 다만 그 3건 중 적어도 둘은 세무사의 실제 작업이 아니라 검증 과정의 편집이었다. 실사용 표본으로 착각하지 않는다.

사례 · 하지 않기로 한 자동화

무슨 일이 넷을 정해야 했다. 이름이 같은 지워진 목차를 되살릴지, 「기타」류 후보를 코드로 자동 제외할지, 빈 목차를 지울지, 잘못 배정된 문장을 코드로 재배정할지. **왜 몰랐나** 넷 다 자동화하면 편해 보인다. 하나하나가 사람이 내린 판단을 코드가 덮어쓰는 자리다. **잡은 방법** 사람이 지운 목차는 되살리지 않고, 나머지 셋도 하지 않기로 했다. 다음 세대는 이전 세대의 오배정도 물려받는다. 대신 세무사가 고치면 그 고침이 이후로 이어진다. 한 목차에 문장이 몰리는 현상에는 선을 긋지 않고 계속만 한다.

잔여물은 버리지도 섞지도 말고, 사람 눈에만 보이는 칸에 둔다

어느 분류에도 안 맞는 잔여물이 남는다. 버리면 지식이 사라지고, 섞으면 검색이 오염된다. 잔여물은 사람 눈에 보이고 기계 검색에는 안 보이는 칸에 둔다. 가공하지 않은 원문 그대로 둔다. 그래야 나중에 사람이 옮겨도 출처가

진짜다.

사례 · 「기타」 칸

무슨 일이 「기타」 목차를 트리에는 보이되 검색에서는 빠지게 했다. 「기타」는 합성하지 않고 원문 1건을 문장 1개로 넣었다. **왜 몰랐나** 잔여물을 합성하면 조항처럼 생긴 가짜 문장이 생긴다. 사람이 그것을 다른 목차로 옮기고 나면 진짜와 구별되지 않는다. **잡은 방법** 가장 불리한 조건으로 확인했다. 「기타」 문장 자신의 임베딩으로 검색해도 상위 5건에 「기타」 소속이 0건이었다.

모든 답변에 들어가는 문안은 사람이 확정하고, 넘치면 멈춘다

모든 답변에 늘 주입되는 규범 문안은 생성하지 않는다. 사람이 확정한 고정 문안으로 둔다. LLM 요약은 매번 그럴듯하지만 매번 조금씩 다르다. 배포할 때마다 규범이 흔들린다. 크기 예산을 넘으면 잘라 넣지 말고 배포 점검에서 멈춘다. 조용히 잘린 규범보다 멈춘 배포가 낫다. 구현은 → 3장.

사례 · 상시 규범

무슨 일이 규범 원문은 17.4k자로, 시스템 프롬프트(1,028자)의 17배라 요약이 필요했다. 요약 초안은 개발 도구로 쓰던 외산 모델이 썼다. **왜 몰랐나** 요약은 그럴듯했다. 예산을 넘는 규범을 잘라 넣으면 잘린 채로 조용히 들어간다. **잡은 방법** 제품 모델로 다시 쓰지 않고, 세무사 검토본으로 「수정 없음, 확정」을 받았다. 이후 변경은 세무사 컨펌 → 문서 → 점검 → 배포 순서로 정했다. 빌드 함수는 예산을 넘으면 절단하지 않고 에러를 낸다. 덧붙여, 사안과 맞지 않는 역대 규칙은 모델이 무시했다. 문구도 사안에 맞아야 따른다.

거버넌스 규칙보다 신원을 먼저 세운다

「두 명이 승인하면 반영」 같은 규칙은 승인한 사람이 진짜 그 사람이라는 전제 위에 선다. 신원을 요청 본문으로 받으면 본문만 바꿔 위조할 수 있다. 본문 신원 위의 N명 승인은 장식이다. 순서는 인증 → 이의 기간 → 관리자 브레이크다. 단, 「누가 썼나」(기여 귀속)를 「누가 눌렀나」(인증 신원)로 덮지 않는다. 일반형은 → 1.5에서 다룬다.

사례 · 한 명이 바꾸는 시스템 프롬프트

무슨 일이 세무사 한 명의 확정이 모든 답변의 시스템 프롬프트를 바꾸게 됐다. 그런데 신원은 요청 본문으로 받고 있었다. **왜 몰랐나** 데모 계정만 쓰는 환경이라 위조가 실제로 일어나지 않았다. **잡은 방법** 대표가 여러 세무사가 확정하는 구조의 위험을 열린 과제로 제기하고 순서를 정했다. 쓰기 API 전체에 토큰 인증을 걸었다. 비로그인 방문자가 쓰는 챗은 제외했다. 본문에 다른 사람 ID와 내 토큰을 함께 보내자 서버는 토큰 쪽을 기록했다.

지우는 범위는 날짜가 아니라 상태로, 쓰기는 계측과 확인 뒤에 한다

「미확정 항목 정리」 같은 한 덩어리 요청 안에는 진행 단계가 다른 데이터가 섞여 있다. 삭제 범위는 날짜 구간이 아니라 상태와 하루 반영 여부로 정한다. 다른 곳에 독립 사본이 남는지, 이의 같은 절차가 걸려 있는지 확인한다. 애매한 상태는 남기고 별도 결정으로 넘긴다. 공유 DB에 쓰는 일은 기본을 시험 실행으로 두고, 결정은 쓰기 없는 계측으로 먼저 내린다. 쓰기는 대표가 확인한 뒤에 한다. 일반형은 → 1.6에서 다룬다.

사례 · 「제출됨」만

무슨 일이 확정된 세션 평가 39건은 지식 테이블에 독립 사본이 39건 모두 살아 있어서 지웠다. 검수실은 지식에 반영되지 않은 「제출됨」만 지웠다. 한 단계 더 나간 「검수 저장」(이의 가능) 2건은 범위에서 뺐다. **왜 몰랐나** 날짜 구간으로만 자르면 이의 절차가 걸린 데이터까지 함께 사라진다. **잡은 방법** 대표가 범위를 「검수실 '제출됨'만」으로 못 박았다. 구현 쪽은 사본 대조와 리뷰 상태로 근거를 확인했다.

사례 · 메모리 안에서 먼저 재기

무슨 일이 원문 지식을 적재하기 전에 프로덕션 DB를 건드리지 않고 메모리 안에서 158문항을 두 번 잼다. 유사도 컷, 범위, 근거의 서열이 그 숫자로 정해졌다. 적재 스크립트는 쓰기 옵션이 없으면 시험 실행만 하고, 같은 입력을 두 번 넣어도 결과가 같다. **왜 몰랐나** 쓰고 나서 재면 잘못된 결정이 이미 데이터에 박힌다. **잡은 방법** 계속 → 대표 확정 → 쓰기 순서를 지켰다. 메모리 안 숫자는 DB 경로로 다시 재기 전까지 잠정으로 적었다.

2.6 누가 무엇을 볼 수 있나

상담 챗봇에는 사장의 대화가 쌓인다. 세무사는 그 대화를 검수하고, 다른 세무사와 지식을 나눈다. 대표가 정하는 것은 누가 어디까지 읽을 수 있는지, 바깥으로 나가는 자료에 무엇이라고 적히는지, 제품 경로가 지켜야 할 규정의 경계가 어디인지다. 이 자리의 실수는 화면에서 보이지 않는다. 화면은 늘 정책대로 보여 준다. 정책이 실제로 강제되는지는 화면 밖에서만 드러난다.

열람 범위는 화면 관행이 아니라 강제여야 한다 — 역할별로 직접 센다

화면이 배경분만 보여 준다고 해서 나머지를 못 읽는 것은 아니다. 데이터 노출을 설계하기 전에 역할별 토큰으로 API를 직접 불러 실제 범위를 센다. 접근 제어는 DB 행 단위 보안 정책으로 강제한다. 이것을 RLS(Row Level Security)라고 한다. 열람 판정은 함수 하나에 모은다. 축소 조건을 쓸 때는 기존 화면 경로를 빼먹지 않는다. 일반형은 → 1.5에서, 구현은 → 5장에서 다룬다.

사례 · 218건 전부

무슨 일이 세무사 계정이 대화 218건 전부를 원문까지 읽을 수 있었다. 검수 이력이 없는 계정도 같았다. 화면이 배경분만 보여 주고 있었을 뿐이다. 축소 조건을 그대로 쓰면 역할을 가리지 않아, 사장님이 일감에 실린 남의 대화를 읽게 되는 구멍도 있었다. **왜 몰랐나** 브라우저 콘솔이나 API를 직접 부르지 않는 한 드러나지 않는다. 화면 관행이 강제처럼 보였다. **잡은 방법** 설계 전제를 확인하려고 세무사 토큰으로 DB API를 직접 불렀다. 대표는 「남겨 두면 비식별은 강제가 아니라 화면 관행일 뿐」이라며 열람 축소를 이번 공사에 넣었다.

권한은 DB가, 표시는 화면이 정한다 — 권한을 여는 데이터에는 닫는 절차를 둔다

보안과 정합의 불변식은 DB가 강제한다. 누가 무엇을 보여 줄지는 화면이 정한다. 두 층을 섞지 않는다. DB가 열어 둔 것도 화면은 일부러 그리지 않을 수 있다. 권한을 줄일 때는 정책만이 아니라 권한을 여는 데이터의 수명까지 본다. 기대 효과를 미리 계산해 두고 실측과 대조한다. 권한을 여는 데이터에는 닫히는 절차를 둔다. 닫기는 일회성 SQL도 자동 실행도 아닌, 손실 없고 되돌리기 쉬운 운영 버튼으로 한다. 일회성 SQL은 또 쌓이고, 자동 실행은 운영자가 못 막는다.

사례 · 사적 대화는 그리지 않는다

무슨 일이 세무사와 사장님의 인앱 상담방을 만들며 비식별 처리·동의·등재를 DB 함수 하나로 묶었다. 방은 수락 전이 안에서만 열리고, 보낸 사람·역할·시각은 DB가 호출자로 정한다. 클라이언트가 보낸 값은 무시해 사칭을 막았다. 반대로 관리자의 상담 본문 열람은 DB 권한이 열려 있어도 화면이 그리지 않게 했다. 사적 대화라는 대표의 판단이었다. **왜 몰랐나** 클라이언트가 규칙대로 부르는 한 어느 쪽이든 똑같이 동작한다. 차이는 누군가 규칙을 건너뛰면 때만 난다. **잡은 방법** 설계 단계에서 「원문을 빼낼 경로」와 「남의 이름으로 보낼 경로」를 하나씩 따져 막았다.

사례 · 218에서 206

무슨 일이 열람 축소 정책을 배포했는데 세무사가 읽을 수 있는 대화가 218건에서 206건으로만 줄었다. 마감이 7월에 지났는데 열린 채 남은 100건짜리 일감 2개(와 진행 중 4개)가 대부분을 열고 있었다. 일감에는 「마감이 지나면 닫는다」는 절차가 없었다. **왜 몰랐나** 정책은 맞게 바뀌었다. 문제는 정책이 참조하는 데이터였다. **잡은 방법** 정책 배포 때 그 조건을 빼고 다시 센 기대값(170~194)을 계산해 뒀다. 일감을 닫자 정확히 그 값(170/193/194)이 나왔다. 일감 닫기는 운영 버튼으로 만들었다.

밖으로 나가는 자료는 출처 표기를, 제품 경로는 규정의 경계를 따로 점검한다

내부에서 굳은 줄임말은 아무도 다시 보지 않는다. 그 줄임말이 사람과 AI의 출처를 뒤바꾸고 있으면, 바깥 자료에서 그것은 사실 왜곡이 된다. 외부로 나가는 자료는 출처 표기(사람인가 AI인가)를 따로 점검한다. 성격이 다른 검토 요청은 한 표로 섞지 않는다. 같은 방식으로, 대회 규정 같은 외부 경계는 「제품이 구동될 때 그 호출이 일어나는가」로 판정한다. 일반형은 → 1.5 · 1.7에서 다룬다.

사례 · 「세무사 코멘트」라는 표기

무슨 일이 내부 문서가 검수 코멘트의 출처를 「세무사 코멘트」라고 적고 있었다. 실제로는 평가자 계정 이름으로 들어간 감사 모드 AI의 산출물이었다. 사람인 세무사의 개입은 승인과 원문·인용 대조뿐이었다. 이 출처는 프로젝트 안에서 한 번 이상 잘못 읽혔다. 세무사에게 보낼 확인 자료에 이 표기가 그대로 들어갈 뻔했다. **왜 몰랐나** 내부에서 굳은 줄임말이라 아무도 다시 보지 않았다. **잡은 방법** 세무사용 PDF·엑셀을 만들며 표기를 점검했다. 외부본에는 「검수 코멘트(AI 평가 도구 작성, 참고용)」로 쓰고 내부 용어를 뺐다. 제품 확인용 31문장을 논문 맹검 표본 30건과 대조해 같은 문장이 없음을 확인했다. 읽는 자료(PDF)와 답을 적는 곳(회신표)도 나눴다.

사례 · 국내 모델 트랙

무슨 일이 이 제품은 「국내 모델만 쓴다」는 대회 트랙에 나갔다. 논문용 3방향 비교가 가능해지자 같은 세션에서 벤치마크 도구가 제안됐다. 대표가 논문 작업과 제품 수정을 한 세션에 맡기는 위험을 지적했다. 외산 모델로 채점하는 것이 규정 위반인지도 다시 따졌다. **왜 몰랐나** 같은 코드베이스에서는 오프라인 예외가 제품 코드로 번지기 쉽다. 「LLM이 채점하는가」로 판단하면 기준이 흐려진다. **잡은 방법** 판정 기준을 「제품 구동 경로 안의 추론 호출인가」로 정했다. 오프라인 예외는 연구 범위 명시, 제출물 아님, 런타임에서 불러오지 않음을 모두 만족할 때만 허용하고, 제품 세션에서는 아예 적용하지 않는다. 외산 호출을 막는 정적 검사가 없으면 방어선이 문서와 사람의 주의력뿐이라는 것도 적어 뒀다.

작업 전후 점검표

- ☐ 재검토하려는 로드맵 항목의 시작 동기를 한 문장으로 적어 확인받았는가? 항목 이름이 기술 이름이라면 특히 그렇게 했는가?
- ☐ 품질을 묻기 전에, 그 기능이 실제 제품 경로에서 호출되고 있는지 코드로 확인했는가?
- ☐ 실사용 중 던진 질문에 눈대중이 아니라 데이터와 계산으로 답했는가?
- ☐ 지우거나 다시 적재하기 전에 같은 질문을 조사한 이전 문서를 찾고, 근거가 된 결론을 원문 전수로 다시 봤는가?
- ☐ 다음 작업을 고를 때 리드타임과 의존 관계로 줄 세웠는가? 켜기만 하면 되는 작업을 진척으로 착각하지 않았는가?
- ☐ 데이터 배선 전에 그 데이터를 쓰는 단계의 호출 모양과, 기존 테이블을 읽는 소비자 목록을 정했는가?
- ☐ 「지금 안 한다」고 정한 일마다 다시 열 조건을 숫자로 적었는가? 원칙에는 「무엇이 입증될 때까지」를 붙였는가?
- ☐ 자원·비용 요청에 의존성 목록, 실측 병목, 무료 한도를 뺀 계산이 붙어 있는가?
- ☐ 실험을 시작하기 전에 유지·후퇴 기준을 숫자로 문서에 적었는가? 물러설 때 대가를 명시했는가?
- ☐ 지금 올리고 있는 숫자가 목표인가, 계기판인가? 완성 기준이 문장으로 있는가? 표본 20 미만의 비율을 인용하고 있지 않은가?
- ☐ 새 지식 소스를 더하기 전에 무엇이 비었는지 키워드별 적중률로 보였고, 계측을 먼저 깔았는가?
- ☐ 운영 규칙의 선택지마다 건드리는 기존 로직을 썼는가? 경합이 생길 수 있는 규칙을 DB 제약으로 막았는가?
- ☐ 오래 걸리거나 되돌리기 어려운 작업이 버튼 한 번으로 시작되지 않는가? 가드가 멈춘 상태가 「실패」로 그려지지 않는가?
- ☐ 자동 처리·자동 재생성이 사람이 손댄 것, 정책이 없는 누군가의 몫을 지우거나 되돌리지 않는가? 승인 규칙이 인증된 신원 위에 서 있는가?
- ☐ 역할별 토큰으로 API를 직접 불러 열람 범위를 썼는가? 밖으로 나가는 자료의 출처 표기가 사람과 시를 바르게 구분하는가?

3장

AI 엔지니어

모델에게 무엇을 말기고 무엇을 말기지 않을지 정해서 코드로 만드는 사람

이 장은 프롬프트를 쓰고, 도구 호출 스키마를 짜고, 모델 출력 뒤에 가드를 붙이고, 모델이 그 일을 해낼 수 있는지 재는 사람을 위한 것이다. 사례는 한 AI 세무 상담 챗봇에서 나왔다. 사장님이 세무 질문을 하면 LLM이 근거 지식을 찾아 답하고, 세무사가 그 답을 검수해 다시 지식으로 되돌리는 제품이다. 근거를 찾아 프롬프트에 붙여 주는 방식을 RAG(검색 증강 생성)라고 한다. 모델은 국내 LLM API 하나만 썼다. 세법 판정 규칙을 코드로 짰 결정적 판정기(이하 규칙 엔진)가 옆에 있었다. 이 자리의 실수는 오류를 내지 않는다. 모델은 개수 상한을 어겨도, id를 한 글자 지어내도, 수치를 지어내도 언제나 그럴듯한 문장을 돌려준다. API가 실패하면 폴백이 「미분류」 나 「인용 0」 같은 그럴듯한 판정을 대신 내놓는다. 정확도 74.3%는 「전부 불충분」으로 찍은 값과 같았는데도 좋은 숫자로 읽혔다. 이 장은 묻는다. **이것이 그럴듯하게 틀렸다면, 누가 언제 알아챌까?** 첫 태그가 AI 엔지니어인 교훈 38개를 모아 보니 원칙 26개, 뿌리는 여섯 갈래였다. 말로 부탁한 제약은 지켜지지 않는다. 모델과 코드의 일을 가른다. 실패를 판정으로 둔갑시키지 않는다. 프롬프트의 글자는 전부 지시다. 모델을 탓하기 전에 과제를 의심한다. 가드와 계측은 범위를 확인하고 붙인다. 모든 직무에 통하는 교훈은 1장이 주인이고, 여기서는 AI 엔지니어의 눈으로 필요한 만큼만 다시 다룬다.

이 장의 원칙

1. 개수·누락·유일 배정은 말로 부탁하지 말고 출력 형식으로 강제한다
2. 모델이 지어낼 수 있는 id는 받지 않는다 — 별칭·enum·교집합
3. 구조화 출력은 판정 필드를 앞에 두고, 목록과 토큰에 상한을 건다
4. 모델이 따르지 않는 금지 규칙은 입력에서 빼서 지킨다
5. 셀 수 있고 규칙이 분명한 일은 코드가 하고, 모델에게는 판단만 남긴다
6. 결정적 판정을 프롬프트 맨 위에 두고, 이상한 문구는 그 판정 출력부터 거슬러 본다
7. 근거가 없는 판정은 코드가 강등한다
8. 모델 출력을 고치는 규칙은 순수 함수로 떼어 제품과 평가가 함께 쓴다
9. 「실패」 「모름」 「그 밖」 을 서로 다른 칸에 둔다
10. 두 회차 결과가 같으면 그 단계가 실제로 돌았는지부터 본다
11. 모델 품질은 후처리 전 원응답에서, 주 경로와 폴백을 따로 센다
12. 프롬프트 속 예시 문장은 모델이 그대로 따라 쓴다고 가정한다
13. 범주를 더하면 예외를 맨 앞에 두고, 포괄 규칙이 삼키는지 프로브한다
14. 다르게 행동시키려면 표식을 붙이지 말고 항목마다 지시한다
15. 출처를 가르게 하려면 라벨과 사용 규칙을 한 쌍으로 넣는다
16. 빌려 쓰거나 물려받은 프롬프트는 맥락이 바뀌면 본문까지 다시 읽는다

17. 모든 답변에 넣는 규범은 사람이 확정한 고정 문안으로 두고, 넘치면 멈춘다
18. 유지·후퇴 기준을 먼저 적고, 천장과 상위 모델 기준선으로 과제 정의를 의심한다
19. 평가셋의 지름길은 비교선으로 드러내고 케이스 단위 분리로 막는다
20. 처방은 하나씩, 그리고 함께 잔다
21. 처방은 목표 지표 밖도 잔다 — 반대편의 손해와 다른 쪽의 이득
22. 결론이 맞아도 근거의 적용 대상이 맞는지 따로 본다
23. 날조 수치는 금지 규칙과 출력 가드를 함께 써서 막는다
24. 사용자 화면으로 가는 모든 LLM 문구에 출처 검사를 걸고, 가드가 안 보는 것을 적는다
25. 모든 LLM 호출에 상한을 걸고, 동시 호출과 자동 도구 선택을 피한다
26. 계측과 대조는 본 경로 옆에서 돌리고 답변을 막지 않게 한다

3.1 말로 부탁한 제약은 구조로 옮긴다

이 프로젝트에서 가장 자주 되풀이된 교훈은 하나다. 모델에게 말로 부탁한 제약은 지켜지지 않는다. 「최대 20개」 「빠짐없이」 「겹치는 건 내지 말 것」 「지어내지 마세요」가 모두 어겨졌다. 어겨도 오류가 나지 않는다. 처방은 셋이었다. 출력 형식(스키마)으로 옮기거나, 입력에서 빼거나, 출력 뒤에서 거른다. 이 장에서 「도구 호출」은 모델이 자유 문장 대신 정해진 JSON 모양(스키마)으로 답하게 하는 방식을, 「enum」은 그 스키마 안에서 고를 수 있는 값을 목록으로 못 박은 것을 말한다.

개수·누락·유일 배정은 말로 부탁하지 말고 출력 형식으로 강제한다

개수 상한, 「빠짐없이」, 「정확히 하나에」 같은 집합 제약은 프롬프트 문장으로는 지켜지지 않는다. 이런 제약은 실패가 불가능한 출력 형식으로 옮긴다. 개수는 스키마의 `minItems / maxItems` 나 코드의 슬라이스로 건다. 누락과 중복은 「목록을 먼저 확정하고, 그 목록을 enum으로만 고르게 하는」 2단 구조로 막는다. 프롬프트에는 「무엇이 좋은 묶음인가」 같은 판단만 남긴다. 스키마는 개수만 지키게 할 뿐 의미까지 지키게 하지는 못한다. 그래서 결과 분포는 계속 잔다. 이 원칙의 일반형은 → 1.4에서 다룬다.

사례 · 목차 만들기

상담 기록을 주제별로 묶어 문장 단위로 다시 쓴 지식 사전을 만들었다. 모델이 상담 묶음에서 주제 후보를 내면(맵), 다른 호출이 그것을 목차로 합쳤다(리듀스). **무슨 일이** 「최대 20개」를 부탁하자 149개가 나왔다. 「8~14개」에는 후보 155개 중 128개를 그대로 돌려줬다. 맵 단계의 「배치당 3~6개」에는 배치당 18~32개가 나왔다. **왜 몰랐나** 모델이 산문 지시를 어겨도 에러가 나지 않는다. 결과는 그저 「목차가 좀 길다」로 보였다. **잡은 방법** 실행 결과에서 개수를 췄다. 도구 스키마에 `minItems / maxItems`를 박자 지켜졌다. 맵에 같은 상한을 넣자 완주 시간도 1013초에서 664초로 줄었다. 다만 30칸을 채우라고 하면 비슷한 말을 쪼개서라도 30개를 만들었다. 스키마는 개수를 지키게 할 뿐 의미 병합은 못 시킨다.

사례 · 대분류 묶기

무슨 일이 세목 29개를 대분류(대목)로 묶으면서 「모든 세목은 정확히 하나의 대목에, 대목은 3~8개」를 프롬프트로 부탁했다. 같은 29세목, temperature 0(무작위성을 최소로 둔 설정)으로 조건을 갈라 졌다. 기존 구현은 누락 9·중복 1이었다. 문구를 바꾸자 누락 2·중복 3이었다. 스키마를 「세목마다 대목 하나」로 뒤집자 누락·중복은 0이 됐지만 이번엔 개수 제약을 어겨 대목을 17개로 쪼갬다. **왜 몰랐나** 기존 구현은 원래부터 9건을 빠뜨리고 있었지만, 화면에는 「미분류」로만 보였다. **잡은 방법** 2단 스키마로 바꿨다. 첫 호출이 `minItems / maxItems`로 대목 목록을 확정하고, 둘째 호출은 그 목록을 `enum`으로만 배정한다. 누락·중복이 0이 됐다. 그래도 같은 입력, temperature 0에서 한 대목에 몰리는 세목 수가 10·11·14·17로 흔들렸다. 그래서 분포는 계속 켜다.

모델이 제어할 수 있는 id는 받지 않는다 — 별칭·enum·교집합

모델이 참조 id를 돌려주게 하면 언젠가 한 글자 틀린 id가 온다. 36자 uuid를 그대로 보여 주지 말고 `p1`, `p2` 같은 짧은 별칭이나 실존 id의 `enum`으로 준다. 돌려받은 id는 이번에 넣은 집합과의 교집합만 저장한다. 별칭을 되돌릴 때 모르는 값은 조용히 버리지 않는다. 따로 세어야 환각 계측이 살아 있다.

사례 · 근거 id

무슨 일이 모델이 문장을 합성하며 근거 문단 id 배열을 돌려줬는데, 그중 하나를 한 글자 다르게 지어냈다. 그 값이 uuid 배열 컬럼에 들어가며 insert가 실패했다. 작업은 37분 동안 돌다가 20개 카테고리 중 11번째에서야 터졌다. 같은 버그가 옛 경로에도 숨어 있었다. **왜 몰랐나** 대부분의 id는 맞았다. 틀린 id 하나가 긴 작업의 중간에서야 DB 제약에 걸렸다. **잡은 방법** 프롬프트에는 짧은 별칭 `p1..pn`을 보여 주고 출력에서 uuid로 되돌렸다. 대조 실험에서 uuid 조건에서만 환각 id가 나왔고(4·2·1건) 별칭 조건은 전부 0이었다. 프로덕션에서도 환각 id가 15에서 0이 됐다. 모르는 값은 호출 쪽 교집합이 떨어뜨리되 개수를 따로 센다.

구조화 출력은 판정 필드를 앞에 두고, 목록과 토큰에 상한을 건다

모델은 스키마의 필드를 앞에서부터 쓴다. 긴 목록 필드가 판정 필드보다 앞에 있으면, 출력이 길이 상한에서 잘릴 때 판정값 자체가 사라진다. 핵심 판정 필드를 맨 앞에 둔다. 목록에는 최대 개수를, 호출에는 `max_tokens`를 건다. 잘린 출력은 별도 범주로 세고, 「잘림을 이쪽으로 본다면」 민감도 줄을 함께 적는다.

사례 · 충분성 판정

질문을 받아 사실을 뽑고 되묻는 앞단 모델에게 「이 질문으로 답할 만큼 사실이 다 나왔나」(충분성)를 판정시켰다. **무슨 일이** 모델이 빠진 사실 목록을 판정 필드보다 앞에 썼다. 같은 말을 되풀이하는 퇴화 생성으로 600토큰에서 잘리면 판정값이 아예 없었다(14~37건). 빠진 사실은 중앙값 4~5개로, 정답 1~3개보다 많았다. **왜 몰랐나** 잘린 건은 오류가 아니라 빈 판정으로 남아 집계에 섞였다. **잡은 방법** 「잘림 = 불충분」으로 본 민감도 줄을 병기해 결론이 같은지 봤다. 목록 최대 3개와 토큰 상한을 걸자 잘림이 37에서 0, 나열 중앙값이 5에서 3이 됐다.

모델이 따르지 않는 금지 규칙은 입력에서 빼서 지킨다

「이것은 쓰지 말 것」이라는 규칙과 「빠짐없이 다뤄라」라는 규칙이 부딪히면 모델은 다루는 쪽을 택한다. 쓰지 말아야 할 내용은 입력에서 뺀다. 결과물을 손으로 고쳐도 원인은 남는다. 다음에 다시 생성하면 같은 주장이 다시 생긴다. 그래서 생성 입력을 고친다.

사례 · 틀린 답변의 재생산

지식 사전은 상담 기록(질문, AI 답변, 검수 코멘트 묶음)을 합성해 만들었다. 검수 코멘트가 「이 답변은 틀렸다」 고 지적한 AI 답변도 묶음에 함께 있었다. **무슨 일이** 합성 프롬프트에 「오류로 지적된 AI 답변을 옮기지 말 것」을 넣어도 20%가 옮겨졌다. 살아 있는 문장 중 이런 옮김이 최소 30건이었다. **왜 몰랐나** 옮겨진 문장은 검수를 거친 선례처럼 보였다. 판정형 답변이 그 틀린 문장을 「검수 선례」로 인용하고서야 드러났다. **잡은 방법** 변형 다섯을 나란히 돌렸다. 지적된 답변을 합성 입력에서 빼고 규칙 문구를 더하자 옮김이 30.9%에서 1.1%가 됐다. 규칙만 넣은 조건은 20.0%였다.

3.2 모델에게 맡길 것, 코드가 할 것

이 자리의 일은 모델을 고르는 것보다 모델에게 어떤 일을 맡길지 정하는 것에 가깝다. 코드가 공짜로 아는 것(무엇이 이미 있는지, 몇 번 나왔는지, 근거 본문에 이 조문이 있는지)을 모델에게 물으면 그럴듯한 추측이 돌아온다. 이 절의 원칙은 모두 「분명한 것은 코드가, 판단만 모델이」라는 뿌리를 공유한다. 끝에는 판정 자체를 모델에게 넘긴 뒤에도 코드가 쥐고 있어야 하는 것을 다룬다.

셀 수 있고 규칙이 분명한 일은 코드가 하고, 모델에게는 판단만 남긴다

「이미 덮인 것은 빠라」 「빈도순으로 정리하라」처럼 규칙이 분명하거나 세면 되는 일을 모델에게 시키지 않는다. 모델은 그 정보를 모르거나, 알아도 틀리게 센다. 코드가 걸러서 뺀 목록을 주고, 빈도는 파이썬이 세서 숫자로 남긴다. 모델에게는 「넓은 주제로 묶기」처럼 코드가 못 하는 판단만 남긴다.

사례 · 빈칸 채우기

무슨 일이 목차에서 비슷한 이름을 접어 빈칸이 생기면 모델이 새 주제를 제안해 채우게 했다. 프롬프트에는 「확정 목차와 겹치는 건 내지 말 것」을 줬다. 모델은 1.8초 만에 후보 목록의 머리를 그대로 베꼈고, 11개 전부가 기존 이름과 겹쳐 거부됐다. **왜 몰랐나** 오류 없이 코드 쪽 채우기(폴백)로 넘어갔다. 결과 목차는 30칸이 다 차 있었다. **잡은 방법** 모델 채우기와 코드 채우기를 따로 센 계측이 「모델 0 / 코드 8」을 보여 줬다. 코드가 덮이지 않은 후보만 걸러 주자(72 → 20) 모델 채우기 8·거부 0이 됐다. 정직하게 적은 것도 있다. 남은 후보가 전부 1회 관찰이라, 모델은 그 목록을 거의 순서대로 골랐다. 결과는 코드 채우기와 같았다. 이 사실을 함수 설명에 적었다.

사례 · 관찰 빈도

무슨 일이 맵 단계는 배치마다 독립이라, 여러 배치에서 반복 관찰된 주제가 넓은 주제다. 그런데 리듀스에는 평평한 후보 목록을 주고 「빈도순으로 정리하라」고 했다. 모델이 알 수 없는 것을 요구한 셈이다. **잡은 방법** 후보를 이름으로 접어 관찰 횟수와 함께 넘기는 집계 함수를 만들었다(업무용승용차·복리후생비 5회, 질문 하나짜리는 1회). 리듀스가 실패했을 때의 폴백도 첫 등장 순에서 빈도순으로 바꿨다.

결정적 판정을 프롬프트 맨 위에 두고, 이상한 문구는 그 판정 출력부터 거슬러 본다

RAG 근거에는 다른 질문자의 사안이 섞여 온다. 모델은 남의 인원·금액·업종을 사용자의 사실처럼 옮긴다. 이것을 이 책에서는 맥락 혼입이라고 부른다. 규칙 엔진이 사실관계를 붙잡아 판정을 프롬프트 위에 박고, 근거는 법리 보강으로만 넣는 경로는 혼입에 강했다. 대신 그 엔진 출력도 프롬프트에 들어가는 입력이다. 답변의 이상한 문구는 근거 코퍼스만이 아니라 엔진 출력에서도 온다. 혼입·날조 측정은 경로별로 따로 한다. 거슬러 보기의 일반형은 → 1.3에서 다룬다.

사례 · 경로별 혼입

무슨 일이 근거만으로 답하는 자문 경로에서는 남의 사안 사실이 섞이는 혼입이 32회 중 25회였다. 엔진 판정이 프롬프트 위에 있는 판정형 경로에서는 32회 중 0~1회였다. **왜 몰랐나** 이전 혼입 측정이 전부 자문 경로였다. 경로별 차이가 있다는 것 자체를 몰랐다. **잡은 방법** 판정에 필요한 사실을 다 말한 질문 3개로 엔진 판정을 고정하고 192회를 돌렸다. 한 경로의 결과를 다른 경로로 일반화하지 않는다.

사례 · 엔진이 낸 잔재

무슨 일이 복리후생비 답변에 「회원권 명의」가 32회 중 21회·17회씩 나와 근거 혼입처럼 보였다. 실제로는 엔진의 필요증빙 목록에 남은 헬스장 사안의 잔재였다. 판정 카드는 「복리후생비 인정」인데 본문은 「급여성 상여」라고 반박한 일도 엔진 근거 문구가 원인이었다. **왜 몰랐나** 엔진은 결정적이고 권위가 있어 의심 대상에서 빠졌다. **잡은 방법** 문구의 출처를 거슬러 엔진 출력에서 찾았다. 감사 모드 LLM이 단 검수 코멘트에 이미 「환각 증빙」 지적이 있었다. 맞는 쪽(판정·금액)을 남기고 엔진 문구를 고치자 본문에 「전액 인정」을 명시한 답이 21회에서 53회가 됐다.

근거가 없는 판정은 코드가 강등한다

설계가 바뀌어 판정 권위가 규칙 엔진에서 답을 쓰는 뒷단 모델로 넘어가도, 「근거가 있을 때만 판정한다」는 조건은 코드가 쥔다. 모델이 판정과 그 근거(조문·문서번호)를 함께 내게 하고, 코드가 그 근거를 실제로 넣어 준 본문과 대조한다. 본문에 없으면 판정을 「판정 보류」로 내리고 결론 문장을 지운다. 근거에 빈틈이 있는데 확정으로 냈으면 「조건부」로 내린다. 근거 본문에 없는 「제N조」 문장도 지운다. 대조는 식별력 있는 토큰이 든 인용만 쎌다. 숫자 없는 법 이름은 아무 본문에나 부분일치한다.

사례 · 강등 함수

무슨 일이 답을 쓰는 모델이 {판정, 판정 근거, 문장들}을 도구 출력으로 내고, 강등 함수가 근거를 검색 본문과 대조했다. 그런데 「소득세법」처럼 숫자 없는 근거는 아무 본문에나 부분일치해 강등을 피했다. **왜 몰랐나** 부분일치는 늘 성공하니 대조가 매번 통과로 보였다. **잡은 방법** 강등 규칙을 짜며 경우를 따지다 드러났다. 조문 번호·문서번호처럼 숫자가 든 인용만 증거로 세게 고쳤다. 스모크 두 문항에서 월세 세액공제는 「조건부」로, 승용차 리스는 「인정」으로 나왔다.

모델 출력을 고치는 규칙은 순수 함수로 떼어 제품과 평가가 함께 쓴다

강등·삭제·대조처럼 모델 출력을 결정적으로 고치는 규칙은 모델 호출 함수 안에 섞지 않는다. 입력과 출력만 받는 순수 함수로 떼어 낸다. 그러면 평가 하니스가 모델을 부르지 않고도 같은 규칙을 그대로 쓴다. 제품과 평가의 규칙이 두 벌이 되면, 평가에서 쎌 품질은 제품의 품질이 아니다.

사례 · 강등 함수 분리

무슨 일이 앞의 강등 함수를 처음부터 모델 호출과 분리해 켜다. 다음에 만들 평가 하니스가 같은 강등 규칙을 그대로 재사용하게 하려는 구현 판단이었다. 사고가 나서 얻은 교훈이 아니라 설계 원칙이다. 3.3에서 본 것처럼 제품 함수와 다른 함수로 재면 측정이 틀린다. 그 자리를 처음부터 막아 둔 것이다.

3.3 실패를 판정으로 둔갑시키지 않는다

LLM 호출은 자주 실패한다. 레이트 리밋, 타임아웃, 도구를 부르지 않은 응답. 이 실패를 `except: return 기본값` 으로 감싸면 화면은 멀쩡하다. 대신 그 기본값이 「모델의 판단」인 척 통계에 섞인다. 「미분류」는 분류기가 고른 답처럼, 「인용 0」은 합성기가 고른 답처럼 보인다. 이 절은 실패와 판단을 끝까지 다른 칸에 두는 원칙들이다.

「실패」「모름」「그 밖」을 서로 다른 칸에 둔다

분류 결과에는 세 칸이 필요하다. 모델이 적극적으로 고른 잔여 범주(「기타」), 모델이 확신하지 못해 보류한 칸(「미분류」), 그리고 물어보지도 못한 실패다. 불확실하면 억지로 배정하지 말고 「미분류」를 돌려주게 한다. 틀린 라벨은 맞는 라벨과 구별되지 않기 때문이다. 그리고 제품 함수는 (결과, 실패사유)를 돌려준다. 모델이 진짜로 「미분류」를 고른 경우만 사유가 비어 있다. 도구를 안 부른 응답도 실패로 센다. 정답 라벨이 없으면 「분포와 사례를 봤다」와 「정확도를 잴다」를 구분해 적는다(→ 1.1 · 1.2).

사례 · 세목 분류

무슨 일이 상담 기록을 17개 세목으로 나누는 분류기를 함수 호출 방식으로 만들며, 실패나 불확실이면 「미분류」를 돌려주게 했다. 「기타」와 「미분류」는 합치지 않았다. 재분류 결과 17개 세목이 모두 1건 이상 쓰였고 미분류는 9%였다. **왜 몰랐나** 분포가 그럴듯하면 정확도까지 확인된 것처럼 읽힌다. 튜닝 전후 비교는 사례 4건뿐이었다. **잡은 방법** 평가 문서가 「확인하지 못한 것」으로 따로 적었다. 미분류 9%가 「분류 불가」인지 「분류기 실패」인지는 나눠 볼 숙제로 남겼다. 반대 방향의 함정도 있었다. 다른 분류기에서 「안 맞으면 미분류, 억지로 끼워 맞추지 마세요」를 모델이 너무 성실히 따라 웬만한 상담을 미분류로 보냈다. 「미분류」를 최후의 수단으로 낮추자 미분류가 45%에서 28%로 줄었다.

사례 · 삼켜진 레이트 리밋

무슨 일이 분류 함수의 `except: return "미분류"`가 레이트 리밋 오류를 「미분류」로 바꿨다. 이 숫자가 「회차 붕괴(미분류 40/38/75%)」와 「미분류 재시도는 효과가 없다(33 → 33 → 32%)」라는 결론을 만들었다. 합성 함수의 `except: return []`는 타임아웃을 「인용 0」으로 바꿔 「인용률 하락」을 만들었다. **왜 몰랐나** 예외를 셀 수 없는 함수로 썼으니 「예외 0건」이었다. **잡은 방법** 8위커 재현 로그에 레이트 리밋 16건이 찍혔다. 실패를 따로 세는 사본으로 다시 썼다. 타임아웃 세목을 빼자 회차 간 상대 편차가 7.9%에서 0.9%로 줄었다. 두 함수 모두 실패 사유를 함께 돌려주게 고쳤다.

두 회차 결과가 같으면 그 단계가 실제로 돌았는지부터 본다

같은 결과는 「그 단계는 원인이 아니다」의 증거가 아니다. 두 회차가 모두 같은 폴백을 탔을 수 있다. 결론을 내거나 재실행을 권하기 전에 단계별 실행 여부를 확인한다. 소요 시간이 타임아웃 상한의 정수배면 타임아웃부터 본다. 상한은 호출 한 곳이 아니라 전부에 건다(→ 1.1).

사례 · 한 번도 안 돈 합치기

무슨 일이 목차를 합치는 리듀스 단계가 며칠 동안 한 번도 실행되지 않았다. 매번 타임아웃 뒤 파이썬 폴백(첫 등장 순 중복 제거)이 목차를 정했다. 실제로 목차를 정한 것은 배치 순서였다. **왜 몰랐나** 폴백 결과도 그럴듯한 목록이었다. 상한을 바꾼 두 날의 목록이 같자 「합치기 탓이 아니다」로 오독했다. 그 철회도 다음 날 다시 뒤집혔다. **잡은 방법** 오프라인 실행 소요 1201초가 정확히 600×2 라는 데서 잡았다. SDK 기본값(600초 타임아웃 + 재시도)에 물려 있던 것이다. 같은 기본값에 맵 단계가 21분 동안 무진전으로 물린 일도 있었다. 상한 없는 LLM 호출 10곳에 명시적 상한을 걸자 합치기가 1201초에서 19초가 됐다.

모델 품질은 후처리 전 원응답에서, 주 경로와 폴백을 따로 센다

누락·중복·환각 같은 모델 품질 지표는 후처리를 거치기 전의 원응답에서 센다. 후처리가 지운 결함은 후처리 뒤에서 보이지 않는다. 0은 기대하던 값이라 의심받지 않는다. 폴백이 있는 경로는 주 경로가 낸 것과 폴백이 낸 것을 따로 센다. 합계만 보면 폴백이 실패를 삼킨다.

사례 · 언제나 0인 중복

무슨 일이 대목 배정 드라이언에서 「중복 배정 수」를 최종 매핑(딕셔너리)으로 세자 항상 0이 나왔다. 딕셔너리가 이미 중복을 지운 결과였다. **왜 몰랐나** 0은 바라던 값이었다. **잡은 방법** 계측 정의를 다시 읽다 찾았다. 원응답에서 따로 세는 칸을 두었다. 앞의 빈칸 채우기에서 「모델 0 / 코드 8」을 드러낸 것도 주 경로와 폴백을 따로 센 계측이었다.

3.4 프롬프트의 글자는 전부 지시다

프롬프트를 쓰는 사람은 문장마다 역할을 나눈다. 이것은 규칙, 이것은 어조 안내, 이것은 예시, 이것은 표식. 모델은 그렇게 나누지 않는다. 예시는 따라 쓸 문장이 되고, 넓은 규칙은 새로 더한 예외를 덮고, 남에게서 물려받은 머리말은 거짓이 된 뒤에도 계속 지시한다. 이 절은 프롬프트의 모든 글자를 지시로 읽는 원칙들이다.

프롬프트 속 예시 문장은 모델이 그대로 따라 쓴다고 가정한다

어조를 안내하려고 넣은 예시 문장도 모델에게는 「이렇게 써라」다. 예시에는 그 문장을 써도 되는 조건을 붙인다. 생성 프롬프트에서는 주제를 끌고 갈 수 있는 구체 예시를 뺀다.

사례 · 거짓 세무사 귀속

무슨 일이 근거에 출처 라벨을 붙이는 작업의 1회차에서, 일반 법리 사전만 근거인데도 「세무사들은 ~로 보았습니다」라고 쓰는 거짓 귀속이 오히려 늘었다(5 → 6). 시스템 프롬프트의 어조 예시 「유사 사례에서 세무사들은 ~로 보았습니다」가 그 문장을 직접 지시하고 있었다. 검색용 가상 질문을 만드는 프롬프트의 「헬스장 회원권」 예시도 모든 질문을 그 주제로 끌 수 있었다. **왜 몰랐나** 예시는 어조 안내로 넣은 것이라 지시라고 생각하지 않았다. **잡은 방법** 걸린 문장의 모양이 예시와 똑같았다. 예시에 「검수 선례가 근거일 때만」을 붙이자 2회차의 거짓 귀속이 2건으로 줄었다.

범주를 더하면 예외를 맨 앞에 두고, 포괄 규칙이 삼키는지 프로브한다

enum에 새 값을 넣었다고 모델이 그 값을 쓰지는 않는다. 「어떤 답이든 이쪽」 같은 기존의 넓은 규칙이 새 범주를 덮는다. 예외 범주를 enum과 정의의 맨 앞에 두고, 포괄 규칙에서 명시적으로 뺀다. 그리고 경계에 놓인

짧은 문장 몇 개(프로브)로 실제로 갈리는지 본다. 사람에게 너무 당연한 범주도 적는다. 범주는 이름만이 아니라 한 줄 정의와 함께 주고, 그 글을 화면 설명으로도 쓴다(→ 1.7). 한쪽 범주를 넓히면 반대쪽 오판이 늘어나는지도 함께 잔다.

사례 · 삼켜진 「모르겠어요」

무슨 일이 사실 추출 프롬프트에 「모름(unknown)」 값을 추가만 했다. 그러자 기존 「어떤 답이든 기재됨(stated)」 문구 때문에 「원가는 모르겠어요」가 기재됨으로 나왔다. **왜 몰랐나** 새 enum 값을 넣었으니 동작할 거라 여겼다. **잡은 방법** 프로브 4문장을 돌렸다. 「모름」을 enum과 정의 맨 앞에 두고 포괄 규칙을 「모른다는 말 말고 어떤 답이든」으로 좁히자 4문장 중 3이 맞았다. 문장 끝의 「...모르겠어요」는 아직 틀려 측정 대상으로 남겼다.

사례 · 부정형 답

무슨 일이 정답 사실 목록을 주어 준 판정 실험에서, 실패의 대부분이 「전입신고 한 번도 안 했어요」를 「빠진 사실」로 본 것이었다. 정답이 「충분」인 질문의 사실 중 55%가 부정형이었다. 「부정형도 기재된 것」이라고 한 줄 더하자 항목 정확도는 50.7%에서 73.2%로 올랐다. 그러나 없는 사실까지 기재됨으로 보기 시작해, 진짜 지표는 거의 그대로(68.6%)였다. **왜 몰랐나** 사람에게 부정형은 명백한 답이라 프롬프트에 적을 생각을 못 했다. **잡은 방법** 실패 건을 항목 단위로 읽었다.

다르게 행동시키려면 표식을 붙이지 말고 항목마다 지시한다

입력에 [다시 묻기] 같은 표식만 붙이면 모델이 맥락을 알아서 다르게 행동하리라 기대하게 된다. 모델은 같은 행동을 되풀이한다. 항목마다 무엇을 다르게 할지 적는다. 출력 개수가 요청보다 적으면 코드가 결정적 문안으로 채운다.

사례 · 되묻기 문안

앞단 모델은 사용자가 「모르겠어요」라고 하면 다른 말로 한 번 더 묻는다. **무슨 일이** [다시 묻기] 표식만 붙이자 같은 질문을 그대로 되풀이했다. 사실 2개를 물어야 하는데 1개만 묻는 출력도 있었다. **왜 몰랐나** 표식이 붙었으니 모델이 사정을 알 거라 여겼다. **잡은 방법** 「모르겠어요」「부분 답변」 시나리오 출력을 직접 읽었다. 항목마다 무엇을 다르게 물을지 지시하자 「대략 얼마 정도로 생각하시나요?」가 나왔다. 모자란 항목은 코드의 고정 문안으로 채웠다.

출처를 가르게 하려면 라벨과 사용 규칙을 한 쌍으로 넣는다

성격이 다른 근거(검수를 거친 선례, 일반 법리 사전)를 평평하게 이어 붙이면 모델은 출처를 구분하지 않는다. 근거 쪽에 출처 라벨을 붙이고, 그 라벨을 어떻게 쓰라는 규칙(우선순위, 「선례의 금액·인원을 사용자 사실로 옮기지 말 것」)을 함께 넣는다. 둘 중 하나만 넣으면 「조치했다」는 느낌만 남는다. 처방은 요소별로 분해해 효과를 잔다.

사례 · 맥락 혼입

무슨 일이 「직원을 3명 더 뽑았는데 고용증대세액공제를 받을 수 있나요?」에 답변이 사용자가 말하지 않은 「신규 채용 4명 중 청년 2명」을 넣었다. 남의 사안 사실이 섞인 것이다. **왜 몰랐나** 라벨만 붙였을 때도, 규칙만 넣었을 때도 처방을 했다는 사실 자체가 안심을 줬다. **잡은 방법** 검색을 한 번 고정하고 답변 생성만 32회씩 반복했다. 혼입은 처방 전 30, 라벨만 29, 규칙만 29, 둘 다 9였다. 다만 근거 다섯 건이 모두 그 사안 한 건의 묶음일 때는 규칙으로도 못 막았다(26/32).

빌려 쓰거나 물려받은 프롬프트는 맥락이 바뀌면 본문까지 다시 읽는다

LLM 함수는 이름과 시그니처가 맞아도 재사용이 맞다는 보장이 없다. 판단 기준은 시스템 프롬프트 본문에 있다. 재사용 검토는 프롬프트 본문까지 읽는다. 공용 함수의 머리말이나 「그 밖의 전부」에 붙는 출처 문구는 새 갈래가 생기면 조용히 물려받는다. 설계의 전제가 바뀌면 공용 문구를 새 전제로 다시 읽는다. 출처 문구는 근거의 종류마다 조립하고, 출처를 부풀리지 않게 한다(→ 1.4).

사례 · 빌린 분류기

무슨 일이 목차를 상담 기록에서 새로 뽑는 동적 목차로 분류하면서, 옛 고정 17세목의 이름과 설명이 박힌 분류 프롬프트를 빌려 쓰고 있었다. 모델은 「업무용승용차·임차료·접대성지출…」을 설명받고, 전혀 다른 목록 중에서 고르라는 지시를 받았다. **왜 몰랐나** 호출은 성공했다. 함수 이름과 시그니처만 보면 맞는 재사용이었다. **잡은 방법** 미분류 원인을 2×2로 쪼개다 프롬프트 불일치가 원인 하나임을 확인했다. 전용 프롬프트를 새로 썼다.

사례 · 거짓이 된 머리말

무슨 일이 자문 답변의 선두 문구 「세무사들이 남긴 검수 의견을 근거로」는 근거가 참고 사전만 아니면 무조건 붙었다. 판례·국세청 해석 원문을 새 근거로 붙이면 그것까지 「세무사 검수」라 부르게 될 판이었다. 공통 규범 머리말의 「판정은 규칙엔진의 권위다」도, 판정 권위가 답을 쓰는 모델로 옮긴 새 설계에서는 거짓이었다. **왜 몰랐나** 출처 문구가 「사전이 아니면」이라는 부정 조건이라 새 갈래가 자동으로 물려받았다. 머리말은 공용 함수 안에 있어 눈에 띄지 않았다. **잡은 방법** 새 답변 단계를 짜며 드러났다. 출처 문구는 근거 종류마다 조립하고, 규범은 본문만 빌려 머리말은 새로 썼다. 덧붙여, 이 제품의 「검수 선례」라는 이름도 실은 감사 모드 LLM의 검수 코멘트에서 나왔다. 사람 세무사가 한 일은 승인과 원문·인용 대조뿐이다. 문구가 출처를 부풀리지 않아야 하는 이유가 여기 있다.

모든 답변에 넣는 규범은 사람이 확정한 고정 문안으로 두고, 넘치면 멈춘다

모든 답변의 프롬프트에 상시로 들어가는 규범(답변 절차, 해석 원칙, 자주 틀리는 패턴)은 런타임에 LLM이 요약하게 두지 않는다. LLM 요약은 매번 그럴듯하지만 비결정적이라, 배포할 때마다 규범이 흔들린다. 사람이 확정한 고정 문안으로 둔다. 크기 예산을 넘으면 잘라 넣지 말고 배포 점검에서 멈춘다. 조용히 잘린 규범보다 멈춘 배포가 낫다. 규범도 사안과 맞아야 모델이 따른다.

사례 · 상시 규범

무슨 일이 규범 원본 전문은 17.4k자로, 당시 시스템 프롬프트(1,028자)의 17배였다. 요약이 필요했다. **왜 몰랐나** 요약을 제품 모델에 맡기거나 예산을 넘는 부분을 잘라 넣었다면, 아무 오류 없이 날마다 조금씩 다른 규범이 들어갔을 것이다. **잡은 방법** 요약 초안은 개발 도구로 쓴 LLM이 썼고, 세무사 검토본으로 「수정 없음, 확정」을 받았다. 합계 1,314자, 예산 2,800자다. 빌드 함수는 예산을 넘으면 자르지 않고 에러를 낸다. 실패율로 효과도 봤다. 규범 파일에 한 줄(가사관련비)을 더하자 그 경고가 답변에 나타났다. 반대로 사안과 안 맞는 역지 규칙(「비용 인정엔 엄격해석 금지」)은 모델이 따르지 않았다. 「지어내지 마라」 규범만으로는 조문 날조도 못 막았다(→ 3.6).

3.5 모델을 타하기 전에 과제를 의심한다

모델이 기대만큼 못 하면 먼저 떠오르는 처방은 둘이다. 프롬프트를 한 번 더 고치거나, 모델을 바꾸거나. 회차마다 「조금만 더」가 가능해 보여서 개선 회차는 끝없이 이어진다. 이 절은 그 고리를 끊는 원칙들이다. 물러설 기준을 먼저 적고, 정답을 쥐여 준 천장과 더 큰 모델의 기준선을 채고, 평가셋이 주는 지름길을 막는다. 처방의 효과는 하나씩, 그리고 함께, 목표 밖 지표까지 쟁다. 이 장에서 「재현율」은 정답이 그 범주인 것 중 모델이 맞힌 비율을, 「few-shot」은 프롬프트에 예시 문답 몇 개를 넣는 방식을 말한다.

유지·후퇴 기준을 먼저 적고, 천장과 상위 모델 기준선으로 과제 정의를 의심한다

모델·프롬프트 개선 실험을 시작하기 전에 「어디를 넘으면 유지, 어디 밑이면 물러선다」를 수치로 적는다(→ 1.2). 「모델 타」이라는 결론 전에는 두 값을 쟁다. 하나는 정답을 쥐여 준 천장(오라클), 다른 하나는 더 큰 모델의 기준선이다. 둘 다 기준에 못 미치면 원인은 과제 정의다. 모델 교체로 풀리지 않는다. 그때는 열린 판단(「다 말했나?」)을 닫힌 대조(「이 사실이 나와 있나?」)로 바꿀 수 있는 자리만 모델에게 남긴다. 모델이 못 하는 일은 결정적 로직으로 옮기고, 그 대가를 적어서 받아들인다.

사례 · 충분성 판정의 포기

무슨 일이 앞단 모델이 「이 질문으로 판정할 만큼 사실이 다 나왔나」를 가릴 수 있는지 재기 전에 기준을 적었다. 「중간 질문 대 충분 질문」 정확도가 72%(질문 길이만으로 가르는 규칙의 값)를 확실히 넘으면 유지, 세 가지를 다 해도 우연 수준이면 역할을 줄인다. 7개 모드 × 446건 결과, 넘은 회차가 없었다. 유사 케이스의 체크리스트를 붙인 방식은 50.8%, 정답 목록을 쥐여 준 오라클은 68.6%였다. **왜 몰랐나** 회차마다 「조금만 더」가 가능해 보였다. 오라클 개선판은 항목 정확도를 50.7%에서 73.2%로 올렸다. 게다가 제품 모델은 「충분」을 거의 말하지 않았고(불충분 재현율 99%), 더 큰 외산 모델은 중간 질문의 78%를 「충분」으로 봤다. 두 모델이 반대 방향으로 틀려서 「모델이 약하다」로 읽혔다. **잡은 방법** 오라클과, 오프라인 진단용으로 한 번 돌린 상위 모델 블라인드 채점(54.2%)을 같은 하니스로 쟁다. 원인은 라벨이었다. 「충분」은 그 판례가 실제로 갈린 사실 1~3개에 달려 있어서, 질문만 보고는 어느 모델도 알 수 없다. 앞단 모델에게는 사실 추출과 되묻기 문안만 맡기고, 「충분」판정은 LLM 없는 상한(턴당 2개·최대 2턴)에 넘겼다. 대가는 적어서 받아들였다. 정보를 다 준 사용자도 한 턴은 질문을 받는다.

평가셋의 지름길은 비교선으로 드러내고 케이스 단위 분리로 막는다

누출로 오른 점수는 개선과 구별되지 않는다. 모두 모델이 「잘 맞히는」 모습으로 나타나기 때문이다. 불균형 평가셋에서는 다수 클래스로 전부 찍은 값을 표에 비교선으로 둔다(→ 1.2). 길이 같은 표면 지름길은 그 규칙만으로 찍은 값을 비교선으로 둔다. 한 케이스에서 파생한 문항들은 평가·few-shot·검색을 원 케이스 단위로 가르다. 블라인드 채점에는 의미 없는 id를 쓰고, 같은 케이스의 형제 문항은 서로 다른 채점자에게 보낸다.

사례 · 74.3%

무슨 일이 few-shot을 넣은 판정의 전체 정확도 74.3%가 그럴듯해 보였다. 평가셋의 「불충분」 비율이 정확히 74.3%였다. 「충분」 재현율은 1.8%였다. **왜 몰랐나** 전체 정확도 하나로는 「전부 불충분」으로 찍은 모델과 구별되지 않는다. few-shot이 충분 재현율을 오히려 낮춘 것도 묻혔다. **잡은 방법** 평가셋 분포와 대조했다. 진짜 지표를 「중간 대 충분」 부분집합 정확도로 정했다.

사례 · 누출 경로 셋

무슨 일이 평가 데이터를 파생하며 누출 경로 셋이 보였다. 충분 질문이 길었다(중앙 230자 대 원본 111자). 같은 케이스의 3형제 문항이 정규화된 질문을 공유했다. 원 id의 접미사(`_suf / _part`)가 라벨을 드러냈다. **잡은 방법** 데이터 파생 단계에서 통계를 보고, 블라인드 설계 때 id를 점검했다. 길이만으로 찍는 값(72%)을 비교선으로 두고, 검색은 자기 케이스를 빼고(leave-case-out) 돌렸다.

처방은 하나씩, 그리고 함께詹다

원인은 대조군으로 하나씩 분리해 확인한다. 그러나 처방은 단독과 조합을 모두詹다. 맞물린 결함은 하나씩 고치면 각각 「효과 없음」으로 보여, 좋은 처방을 버리게 된다. 한쪽만 고치면 손실이 다른 단계로 옮겨갈 뿐인지도 확인한다.

사례 · 인용 누수

무슨 일이 문장 합성에서 넣어 준 근거의 일부만 인용되는 누수가 있었다. 「모든 근거를 최소 한 번 인용」 규칙, 짧은 id 별칭, 입력 예산 축소를 처방으로 골랐다. 같은 크기 대조군(N=27)에서 처방 전 59%, 규칙만 75%, 별칭만 74%였다. 셋을 합치자 세 번 모두 96%가 됐다. 프로덕션 인용률은 65%에서 93%가 됐다. **왜 몰랐나** 하나씩만 시험하면 부분 효과뿐이라 버릴 뻔했다. 반대로 「맵 프롬프트가 원인」이라는 단일 원인설은 2×2 대조군에서 뒤집혔다. 맵 프롬프트만 고치면 유실 지점이 「미분류」에서 「미투입」으로 옮겨갈 뿐이었다. **잡은 방법** 크기 축× 처방 조합 표로詹다. 맵 프롬프트는 합성 쪽 분할과 함께 넣었다.

처방은 목표 지표 밖도詹다 — 반대편의 손해와 다른 쪽의 이득

한 지표를 겨냥한 처방은 그 반대편 지표를 망가뜨릴 수 있다. 모델 입력에서 정보를 가리는 처방은 그 정보가 필요한 답변에서 날조를 부르는지 함께詹다. 반대로 목표 지표에 실패한 처방도 지연·실패율·출력 길이에서 이득이 있을 수 있다. 실험 하니스는 목표 지표 밖의 값도 늘 남기고, 살릴 것은 제품으로 넘긴다.

사례 · 가린 수치

무슨 일이 맥락 혼입을 막으려고 근거의 모든 섹션에서 수치를 가렸다. 그러자 「경조사비 20만원 기준」 같은 법정 기준액까지 가려졌다. 모델은 30만원, 3만원을 지어냈다. 정답 20만원을 언급한 답이 32회 중 22회에서 0회가 됐다. **왜 몰랐나** 혼입 지표로는 혼입이 줄어 보였다. **잡은 방법** 답변 속 기준액을 값별 분포표로 만들었다. 정규식은 「남의 사안 수치」와 「법정 기준 수치」를 가를 수 없다. 이 처방은 버렸다.

사례 · 실패한 회차의 쓸모

무슨 일이 목록 최대 3개와 토큰 상한을 거는 처방은 판정 정확도에는 효과가 없었다(중간 대 충분 49.2%). 그러나 퇴화로 인한 잘림이 37에서 0, 응답 지연 중앙값이 1.0초가 됐다. **왜 몰랐나** 목표 지표만 보면 「실패한 회차」로 버려진다. **잡은 방법** 회차마다 정확도 외에 지연·잘림·나열 개수를 함께 기록해 두었다. 이 조치를 제품의 앞단 모델 호출(토큰 상한 600)에 넘겼다.

결론이 맞아도 근거의 적용 대상이 맞는지 따로 본다

판정 일치율은 결론만 비교한다. 결론이 맞아도 근거가 다른 납세자 유형(개인과 법인), 다른 세목, 다른 시기에서 왔을 수 있다. 결론 일치 지표 하나로 답변 품질을 보지 않는다. 근거의 적용 대상이 질문과 맞는지 따로 본다.

사례 · 개인과 법인

무슨 일이 병원 원장(개인사업자)의 질문에 법인 대상 질의회신만 검색되어, 답이 법인세법 시행령을 근거로 들었다. 엔진 판정과 결론은 같았다. **왜 몰랐나** 엔진 대조 지표는 결론만 비교한다. 근거가 어느 납세자 유형에서 왔는지는 보지 않는다. **잡은 방법** 스모크 답변을 사람이 직접 읽었다. 열린 과제로 남겼다. 검색 쪽 처방은 → 4장.

3.6 가드와 계측은 범위를 확인하고 붙인다

말로 부탁해 안 되는 것은 출력 뒤에서 거른다. 그러나 가드는 자기가 보는 것만 본다. 「날조는 가드가 잡는다」는 문장이 설계 문서에 한 번 적히면, 그 가드가 실제로 무엇을 토큰으로 보는지는 아무도 다시 확인하지 않는다. 가드가 지키는 자리 밖으로 나가는 문구도 있다. 계측과 대조 장치는 반대로 너무 열심히 붙어 사용자 응답을 막을 수 있다. 이 절은 가드와 계측의 범위를 다룬다.

날조 수치는 금지 규칙과 출력 가드를 함께 써서 막는다

「수치를 지어내지 마세요」는 줄여 줄 뿐 막지 못한다. 출력 뒤에 가드를 붙인다. 가드는 「질문·엔진 출력·근거·규범 어디에도 없는 수」를 찾는다. 숫자만 지우면 문장이 깨지므로 문장 단위로 빼고, 하나라도 뺐으면 확인 안내를 붙인다. 값을 고치지 않는다. 어느 값이 맞는지는 전문가 몫이다. 출처 목록에는 이전 턴의 **사용자** 발화만 넣는다. 앞선 답변을 넣으면 그 답의 날조가 다음 턴의 근거로 되살아난다. 정당한 법정 수치가 필요하면 사람이 승인한 규범 층에 넣어 가드가 통과시키게 한다(→ 1.4).

사례 · 접대비 한도

무슨 일이 자문 경로에 「수치를 지어내지 마세요」가 있었는데도 96회 중 11회가 수치를 날조했다. 판정형 답변은 접대비 한도를 64회 중 23회, 회차마다 다른 값으로 말했다. **왜 몰랐나** 날조된 한도는 그럴듯한 숫자였고, 판정 카드는 맞았다. **잡은 방법** 「어디에도 없는 수」를 세는 지표를 만들었다. 판정형 날조 답변은 192회 중 29회에서, 규칙만 고쳐 10회, 규칙과 가드를 함께 써서 0회가 됐다. 자문 경로도 11/96에서 0이 됐다.

사용자 화면으로 가는 모든 LLM 문구에 출처 검사를 걸고, 가드가 안 보는 것을 적는다

날조 가드는 보통 「답변」에만 붙는다. 그런데 사용자 화면으로 가는 LLM 문구는 답변만이 아니다. 되묻기 질문, 요약, 라벨도 나간다. 이런 중간 산출물은 답변 가드가 지키는 자리 밖이다. 사용자에게 가는 모든 경로에 수치 출처 검사를 걸고, 출처 없는 수치가 든 항목은 버린다. 그 때문에 꼭 필요한 항목을 잃는 빈도를 따로 쟀다. 「다른

가드가 잡는다」고 책임을 넘기는 문장을 썼다면, 그 가드의 코드에서 해당 입력이 실제로 걸리는지 확인한다(→ 1.4). 가드가 안 보는 토큰은 열린 과제로 적는다.

사례 · 되묻기 속 기준액

무슨 일이 앞단 모델이 빠진 사실을 「골프 비용이 1인당 5만원 이하인지 여부」처럼 틀린 기준액을 붙여 냈다. 이 문구가 그대로 되묻기 질문이 되면, 사용자에게 틀린 기준을 단정하는 셈이 된다. **왜 몰랐나** 중간 산출물(빠진 사실 목록)이라 답변의 날조 수치 가드가 지키는 자리 밖이었다. **잡은 방법** 로컬 시나리오 출력을 읽다 발견했다. 출처 없는 수치가 든 항목은 버리고 버린 수를 센다. 같은 이유로, 상담 기록을 비식별화할 때도 LLM이 다시 쓰게 하는 안을 기각했다. 날조 수치가 생기고, 세무사가 원문이 아닌 것을 검수하게 되기 때문이다.

사례 · 아무도 안 잡던 날조 조문

무슨 일이 설계 문서는 「날조 조문은 날조 수치 가드가 잡는다」고 적었다. 그런데 그 가드는 설계상 조문 번호를 수치 토큰에서 뺀다. 날조 조문은 아무도 잡지 않고 있었다. **왜 몰랐나** 설계 문서가 책임을 다른 가드에 넘겼고, 넘겨받은 쪽의 실제 토큰 규칙은 아무도 확인하지 않았다. **잡은 방법** 답을 쓰는 모델의 가드를 짜며 그 가드의 범위를 확인했다. 근거 본문에 없는 「제N조」 문장을 지우는 조문 가드를 따로 만들었다. 같은 가드는 「N월」 기간도 보지 않았다. 「연말정산 시점(다음 해 2월~5월)」 같은 날조가 통과했다. 이것은 열린 과제로 적어 두었다.

모든 LLM 호출에 상한을 걸고, 동시 호출과 자동 도구 선택을 피한다

SDK 기본 타임아웃은 사람의 인내보다 훨씬 길다. 호출 하나하나에 명시적 상한을 걸고, 한 곳이 아니라 전부를 감사한다(→ 1.1). 폴백이 있는 자리라면 상한은 정확도를 깎지 않고 최악의 시간만 자른다. LLM API는 단건으로는 빠르다가 동시 호출과 자동 도구 선택에서 지연이 폭발한다. 배치 평가는 순차로, 짧은 타임아웃으로, 도구를 강제 지정해서, 토큰 상한을 걸고 돌린다. 열린 생성보다 닫힌 대조가 빠르고 안정적이라는 점도 설계 선택에 반영한다. 운영 쪽 대기열과 혼잡 안내는 → 5장.

사례 · 평가 하니스의 지연

무슨 일이 동시 3건으로 호출하자 응답 지연 중앙값이 62초였다(단건 1~2초). 순차여도 간헐적으로 30초 멈춤이 있었고, 상당수가 같은 말을 되풀이하는 퇴화 생성이었다. 모델이 도구를 고르게 둔 단건 호출(`tool_choice="auto"`)은 120초 타임아웃이 한 번 났고, 도구를 강제 지정하면 2초였다. 정해진 목록과 대조만 하는 닫힌 방식은 중앙값 0.75~1.0초에 퇴화가 없었다. **왜 몰랐나** 단건 스모크는 늘 빨랐다. 동시성과 자동 선택에서만 터진다. **잡은 방법** 하니스를 돌리며 지연 분포를 기록했다. 배치 평가는 순차·20초 타임아웃·도구 강제로 돌렸다.

계측과 대조는 본 경로 옆에서 돌리고 답변을 막지 않게 한다

판정 품질을 재는 대조 장치나 계측은 사용자 응답 경로 위에 있지 않는다. 본 경로와 병렬로 돌리고, 실패는 삼키고, 기다리는 시간에 상한을 둔다. 지표 때문에 사용자가 답을 못 받는 일은 없어야 한다. 다만 삼킨 실패도 3.3의 원칙대로 따로 세어야, 지표가 대조에 성공한 건만으로 조용히 계산되지 않는다.

사례 · 엔진 대조

무슨 일이 판정 권위가 답을 쓰는 모델로 넘어간 뒤, 그 판정 품질을 꽤 유일한 객관 지표로 규칙 엔진 대조를 붙였다. 엔진은 원래 판정 경로와 같은 관문(사실 추출, 유형, 필수값, 결정 변수 검증)을 통과할 때만 부른다. **잡은 방법** 사고에서 나온 교훈이 아니라 설게 판단이다. 대조는 답변 생성과 병렬로 돌고, 실패는 전부 삼키며, 답이 나온 뒤 5초까지만 기다린다. 스모크에서 승용차 리스 문항은 「일치」, 월세 세액공제 문항은 「엔진 적용 범위 밖」으로 기록됐다.

작업 전후 점검표

- ☐ 프롬프트에 「최대 n개」 「빠짐없이」 「정확히 하나」 를 문장으로만 적지 않았는가? 스키마의 개수 제약이나 「목록 확정 → enum 배정」 으로 옮겼는가?
- ☐ 모델이 돌려주는 id가 실존 enum이나 짧은 별칭인가? 넣은 집합과의 교집합만 쓰고, 버린 값을 세는가?
- ☐ 구조화 출력에서 판정 필드가 목록 필드보다 앞에 있는가? 목록 최대 개수와 `max_tokens` 가 걸려 있는가?
- ☐ 「이것은 쓰지 말 것」 규칙이 있다면, 그 내용을 입력에서 빼는 쪽이 가능하지 않은가?
- ☐ 코드가 공짜로 아는 것(이미 있는 것, 빈도, 본문 포함 여부)을 모델에게 묻고 있지 않은가?
- ☐ LLM 함수가 실패를 기본값으로 돌려주지 않는가? (결과, 실패사유) 로 「실패」 「모름」 「그 밖」 이 갈리는가? 도구를 안 부른 응답도 실패로 세는가?
- ☐ 두 회차 결과가 같거나 소요 시간이 타임아웃의 정수배라면, 그 단계가 실제로 돌았는지 확인했는가?
- ☐ 누락·중복·환각을 후처리 전 원응답에서 세는가? 주 경로와 풀백을 따로 세는가?
- ☐ 프롬프트의 예시 문장에 쓰는 조건이 붙어 있는가? 새 범주를 넣었다면 포괄 규칙이 삼키지 않는지 프로브 문장으로 봤는가?
- ☐ 빌려 쓰는 LLM 함수와 공용 머리말의 프롬프트 본문을 지금의 전제로 다시 읽었는가? 출처 문구가 근거를 부풀리지 않는가?
- ☐ 개선 실험 전에 유지·후퇴 기준을 수치로 적었는가? 오라클 천장과 상위 모델 기준선, 다수 클래스 찍기 값, 표면 규칙 값을 같은 표에 두었는가?
- ☐ 평가·few-shot·검색이 원 케이스 단위로 갈려 있는가? 길이·형제 문항·id 접미사로 라벨이 새지 않는가?
- ☐ 처방을 단독과 조합으로 모두 켜는가? 목표 지표 밖(정확성, 날조, 지연, 잘림)도 함께 기록했는가?
- ☐ 결론 일치만이 아니라 근거의 적용 대상(개인/법인, 세목)이 질문과 맞는지 봤는가?
- ☐ 사용자 화면으로 가는 모든 LLM 문구(답변, 되묻기, 요약, 라벨)에 수치·조문 출처 검사가 걸려 있는가? 「다른 가드가 잡는다」 는 문장의 가드 코드를 직접 확인했는가?

4 장

지식 엔지니어

근거 지식을 모으고, 자르고, 분류하고, 다시 만들고, 찾게 하는 사람

이 장은 LLM이 답할 때 기대는 지식을 맡은 사람을 위한 것이다. 외부 문서를 수집하는 사람, 문서를 검색 단위로 자르고 분류하는 사람, 검색의 임계값과 여러 지식 갈래의 합치는 방식을 정하는 사람, 지식을 주기적으로 다시 만드는 파이프라인을 돌리는 사람이 여기에 든다. 사례는 한 AI 세무 상담 챗봇에서 나왔다. 사장님이 세무 질문을 하면 LLM이 근거 지식을 찾아 답하고, 세무사가 답을 검수해 그 결과를 다시 지식으로 되돌리는 제품이다. 근거를 찾아 붙여 답하는 이 방식을 RAG(검색 증강 생성)라고 한다. 모델은 국내 LLM API 하나만 썼다. 이 자리의 실수는 거의 오류를 내지 않는다. 검색은 언제나 무언가를 돌려주고, 돌려준 것은 언제나 그럴듯하게 관련 있어 보인다. 파이프라인은 에러 없이 완주하면서 원문의 80%를 버린다. 수집기는 「결과 없음」이라는 정상 응답으로 실패를 감춘다. 사람이 고친 문장은 다음 자동 회차에 조용히 원래대로 돌아간다. 그래서 이 장의 질문은 늘 같다. **이 지식이 그럴듯하게 틀렸다면, 누가 언제 알아챌까?** 지식 엔지니어가 주 직무인 교훈은 39개였다. 그중 모든 직무에 통하는 16개는 1장이 말고, 여기서는 필요한 자리에서만 짧게 짚는다. 남은 23개를 같은 뿌리끼리 묶으니 원칙 20개, 여섯 갈래가 되었다. 지식의 단위, 임계값, 커버리지 계측, 층과 융합, 재생성 파이프라인, 새 소스의 수집이다.

이 장의 원칙

1. 지식을 보여 주거나 고치기 전에 저장 단위와 관계를 코드로 확인한다
2. 검색 단위는 문서가 아니라 홀로 서는 문장·조항으로 낮춘다
3. 분류기에는 판정 신호만 넣고, 지식은 검색과 합성에 준다
4. 임계값은 숫자가 아니라 그 선 위에서 하는 일과 한 쌍이다
5. 검색 컷은 컷 없이 돌린 점수대별 정답 등급 분포로 굿는다
6. 검색 결과를 판정 신호로 쓰기 전에 정답 클래스별 분포부터 갈라 본다
7. 에러 없는 완주를 믿지 말고 깔때기의 각 지점을 따로 센다
8. 여러 코퍼스를 한 갈래에 섞었으면 코퍼스별 등장 횟수를 센다
9. 평가 질문은 평가 대상 코퍼스를 보지 않고 사용자 쪽에서 만든다
10. 어떤 질문에도 적용되는 규칙은 검색하지 말고 상시 주입한다
11. 코퍼스마다 임계값을 손으로 맞추고 있다면 융합기부터 만든다
12. 순위 융합의 이득은 갈래들이 겹칠 때만 난다
13. 갈래별 쿼터는 독식만 막고, 빈 자리는 다른 갈래로 채운다
14. 세대교체는 상태 값으로 가르고, 건너야 할 것을 목록으로 점검한다
15. 표시(레이블)와 정체성(id)을 가르고, 그릇의 성격대로 지우는 법을 정한다
16. 분류 안 되는 잔여물은 사람에게에는 보이고 검색에는 안 보이는 칸에 둔다

17. 소스의 성격이 답할 수 있는 질문을 정한다 — 주제별 적중률로 먼저 쟀다
18. 외부 API의 「없음」은 실패일 수 있다 — 빈 결과 비율을 지표로 둔다
19. 수집기의 정렬·상한·인코딩·식별자는 추측하지 말고 명시한다
20. 대화를 지식 풀로 들이는 비식별은 실데이터 감사와 화면 확인까지 해야 끝난다

4.1 지식의 단위와 실제 모양

검색은 임베딩으로 돈다. 임베딩은 글 한 덩이를 수천 차원의 숫자 벡터 하나로 바꿔, 뜻이 가까운 글끼리 공간에서 가깝게 놓는 변환이다. 질문도 벡터로 바꾼 뒤 두 벡터의 방향이 얼마나 같은지(코사인 유사도)로 가까운 지식을 고른다. 이때 「한 덩이」를 무엇으로 잡느냐가 이후 모든 것을 정한다. 이 제품의 첫 지식베이스는 [질문 / AI 답변 / 검수 코멘트] 세 칸짜리 묶음(이하 번들) 하나를 통째로 벡터 하나로 만들었다. 이 절의 원칙은 모두 「지식 한 행이 실제로 무엇을 담고, 그 벡터가 무엇에 끌려가는가」를 먼저 본다는 뿌리를 공유한다.

지식을 보여 주거나 고치기 전에 저장 단위와 관계를 코드로 확인한다

지식을 보여 주는 화면, 지식을 정리하는 도구, 지식을 지우는 기능은 모두 지식의 실제 구조 위에 선다. 저장 단위(무엇이 한 행인가), 관계 유무(행끼리 이어져 있는가), 검색 방식(어떤 인덱스로, 어떤 필터로 찾는가)을 먼저 표로 적고 거기에 맞춰 설계한다. 「지식망」 같은 이름이 주는 인상으로 구조를 짐작하지 않는다. 메타데이터는 그것을 읽는 화면이 생기기 전에 분포를 한 번 찍어 본다. 아무도 읽지 않는 칸은 틀려도 아무도 모른다 → 1.3.

사례 · 지식 지도 화면

무슨 일이 「세무사가 지식베이스를 거미줄처럼 보고 직접 고치는 화면」을 요청받았다. 화면부터 그리지 않고 스키마와 적재·검색 코드를 먼저 읽었다. 지식베이스는 그래프가 아니라 연결(edge)이 하나도 없는 단일 테이블이었다. 임베딩 단위는 번들 하나였다. 벡터가 4096차원이라 근사 최근접 검색 인덱스의 상한(2000차원)을 넘어서, 매 질문마다 활성 526건 전체를 하나씩 비교하고 있었다. 업종 필터는 값이 「의원」으로 하드코딩되어 사실상 모든 행을 통과시켰다. **왜 몰랐나** 「인덱스 없음」은 마이그레이션 주석에 이미 적혀 있었다. 기능이 동작하니 아무도 읽지 않았다. **잡은 방법** 구조를 표로 적은 뒤, 화면은 저장된 분류로 묶은 클러스터를 기본으로 하고 노드를 누를 때만 그 자리에서 유사도를 계산하게 정했다. 「번들 하나가 벡터 하나」라는 사실은 곧이어 중복 정리가 틀어진 원인의 열쇠가 됐다.

사례 · 중복 정리

무슨 일이 같은 내용이 세 번 적재된 것을 보고, 코사인 0.85 이상을 한 무리로 묶어 무리마다 가장 오래된 것만 남기려 했다. 그 기준은 526건 중 74%(390건)를 묶었고, 286건이 지워질 뻔했다. **왜 몰랐나** 의심하던 3건이 정확히 잡혀 방법이 맞아 보였다. 하지만 번들의 벡터는 질문 텍스트에 끌려간다. 같은 대화의 다른 구간, 같은 시나리오 틀로 만든 다른 대화(감사 모드 LLM이 만든 합성 시나리오)가 한 무리로 엮였다. **잡은 방법** 무리 하나를 대화 id와 구간 id로 손으로 대조했다. 진짜 중복은 같은 구간에 160ms 간격으로 들어간 2건뿐이었다. 자동 후보를 「같은 대화·같은 구간」이라는 구조적 신호로 바꾸자 286건이 45건이 됐다. 의미 유사도는 사람이 훑어볼 목록에만 쓴다 → 1.3 · 1.6.

검색 단위는 문서가 아니라 홀로 서는 문장·조항으로 낮춘다

긴 글을 벡터 하나로 만들면 그 벡터는 글 안의 여러 주제의 평균이 된다. 글이 길수록 정보가 희석된다. 검색 단위를 문장이나 조항으로 낮추고, 검색·기여 크레딧·수정 이력의 최소 단위를 그 단위 하나로 맞춘다. 셋이 다른

단위면 「이 답의 근거는 누구의 어느 문장인가」에 답할 수 없다. 합성한 문장은 앞 문장에 기대지 않는 독립 완결형으로 만든다. 단위를 낮추면 점수 분포도 바뀌므로 임계값은 새로 쥔다. 다른 소스의 컷을 물려받으면 잘린 것은 결과에 보이지 않는다 → 1.2.

사례 · 문장 단위 사전

무슨 일이 대표가 「지금 지식베이스도 코사인으로 찾나, 백서 같은 형식에도 통하나」를 물었다. 코드를 열어 보니 제목과 본문 전체를 통째로 한 번 임베딩하고 있었다. 그래서 번들을 원재료로 삼아 LLM이 문장(조항) 단위로 응축한 별도 사전을 만들었다. 이 장에서는 이것을 「문장 사전」이라고 부른다. 원 번들 검색 경로는 건드리지 않고 옆에 나란히 엮었다. **왜 몰랐나** 통째 임베딩으로도 검색은 어쨌든 무언가를 찾아왔다. **그 뒤** 응축 문장은 짧아서 코사인 분포가 낮았다. 관련 있는 매치 하나가 0.4985로, 번들용으로 정한 컷 0.50에 소리 없이 잘렸다. 단위를 바꾼 날 임계값도 함께 바뀌어야 했던 것이다.

분류기에는 판정 신호만 넣고, 지식은 검색과 합성에 준다

「지식이 있는 곳」과 「판정 신호가 있는 곳」은 다르다. 주제를 판정하는 분류기에는 주제를 정하는 텍스트만 준다. 답변과 검수 코멘트는 지식이 담긴 곳이라 넣는 편이 나아 보이지만, 그 답변이 질문과 다른 주제로 흘렀다면 분류기는 엉뚱한 주제를 읽는다. 검색과 합성에는 전체를 준다. 입력 조건을 바꿀 때는 기존 조건이 이전 값을 그대로 재현하는지도 함께 봐서 계측이 정상인지 확인한다.

사례 · 주제 분류

무슨 일이 문장 사전을 만드는 첫 단계는 원 번들 413건을 주제(세목)로 나누는 분류였다. 분류기는 질문·AI 답변·검수 코멘트가 든 번들 전체(평균 601자)를 읽었다. **왜 몰랐나** 평균 배정률은 그럴듯했다. 원인은 어느 목차로도 갈 곳이 없던 건들의 원문을 직접 읽고서야 나왔다. 번들에 실린 AI 답변이 질문과 다른 주제였다 → 1.3. **잡은 방법** 입력 조건 셋을 각 3회 켜다. 번들 전체 52.2%, 질문+코멘트 57.1%, 질문만(55자) 64.4%였다. 전체 조건이 기존 값을 재현해 계측도 정상이었다. 읽히는 텍스트를 열 분의 일로 줄였더니 배정률이 12.2%p 올랐다.

4.2 임계값은 쓰임에 묶인다

검색에는 거의 언제나 컷이 붙는다. 코사인 점수가 몇 이상인 것만 근거로 쓰느냐는 숫자 하나다. 같은 임베딩, 같은 지식베이스, 같은 코사인이면 같은 숫자를 다시 쓰고 싶어진다. 그런데 컷 위에서 하는 일이 다르면 같은 숫자가 정반대로 틀린다. 그리고 컷이 잘라 낸 것은 결과에 나타나지 않으므로, 틀린 컷은 답을 틀리게 만들지 않고 답을 조용히 가난하게 만든다.

임계값은 숫자가 아니라 그 선 위에서 하는 일과 한 쌍이다

「관련 있다」와 「사실상 같다」는 다른 질문이다. 최근접 하나를 고르는 구조와 모든 쌍을 이어 묶는 구조도 다르다. 앞의 것은 선을 넘은 하나만 쓰지만, 뒤의 것은 오탐 한 쌍이 무관한 무리를 통째로 잇는다. 그래서 쓰임이 바뀌면 같은 데이터라도 선을 다시 쥔다. 임계값마다 「무엇을 가리는 값인지」를 이름과 주석에 적는다. 진짜 동의어가 선 아래에 있으면 선을 내리지 말고 다른 신호와 합집합으로 쓴다. 오탐과 정탐이 같은 구간에 섞이면 선을 긋지 않고 알려진 한계로 둔다. 실측 편차와 전수 판독 위에 선을 긋는 일반 원칙은 → 1.2에서 다룬다.

사례 · 중복 경고

무슨 일이 세무사가 검수 결과를 지식으로 저장하기 전에 「이미 비슷한 지식이 있다」고 경고하는 사전검토를 붙였다. 검색 컷이 0.50이었지만 경고 임계값은 85%로 따로 잡았다. **왜 몰랐나** 같은 코사인이니 한 값을 쓰고 싶어진다. 그러면 검색에 걸릴 만한 모든 것이 중복으로 경고된다. **잡은 방법** 설계 때 분리했다. 실제 중복 3건은 94%대로 잡혔다. 사전검토가 비교하는 텍스트는 실제 적재 텍스트를 만드는 함수를 그대로 불러서 만든다 → 1.6.

사례 · 목차 접기

무슨 일이 문장 사전은 회차마다 AI가 목차를 새로 짓는다. 이전 회차의 세목에 사람이 옮겨 둔 문장을 새 세목에 다시 붙일 때, 세목 이름 임베딩의 코사인 0.70은 전수 판독에서 오탐이 없었다. 같은 0.70을 한 회차 안에서 비슷한 세목끼리 접는 데 쓰자, 오탐 한 쌍(0.7166)이 무관한 세목 셋을 한 덩어리로 뭉쳤다. **왜 몰랐나** 같은 지식베이스, 같은 임베딩, 같은 이름 축이라 같은 값이 통할 것처럼 보였다. **잡은 방법** 새 회차 세목 30개의 내부 쌍을 전수로 다시 뽑았다. 0.75와 0.80은 결과가 같은 안정 구간이었다. 선 아래에 있던 진짜 동의어(0.6954, 0.6326)는 선을 내려 살리지 않고, 문자열 규칙(겹치는 글자 3자 이상)과 합집합으로 잡았다.

검색 컷은 컷 없이 돌린 점수대별 정답 등급 분포로 굿는다

평균 점수 하나로는 어느 점수대부터 잡음인지 보이지 않는다. 먼저 컷 없이 돌려 결과마다 정답 등급(쓸모없음 / 일부 / 쓸모 있음)을 매기고, 점수대별로 표를 만든다. 잡음만 있는 점수대가 끝나는 자리가 컷 후보다. 후보끼리는 같은 문항, 같은 채점기로 비교하고, 채점의 흔들림보다 큰 차이만 믿는다.

사례 · 질의회신 컷

무슨 일이 국세청 질의회신을 새 코퍼스(검색 대상 문서 묶음)로 넣으면서 컷을 정해야 했다. **잡은 방법** 컷 없는 첫 회차에서 일반 사용자 말투의 문항을 점수대별 등급(0/1/2)으로 켜다. 0.30대는 13/0/0, 0.40대는 49/5/14였다. 0.40 미만은 거의 잡음이었다. 두 번째 회차에서 후보를 비교하니 「쓸모 있음」 비율이 컷 0.40에서 54.2%, 0.45에서 43.8%였다. 이 채점기의 흔들림은 $\pm 2\%$ p였으므로 차이를 믿을 수 있었다.

검색 결과를 판정 신호로 쓰기 전에 정답 클래스별 분포부터 갈라 본다

검색이 「비슷한 사례」를 잘 찾아오는 것과, 그 사례가 판정에 쓸 차이를 담고 있는 것은 다른 일이다. 검색 결과를 어떤 판정의 신호로 쓰려면, 정답 클래스별로 그 신호의 분포가 다른지를 먼저 본다. 같으면 몇 개를 가져오든(k) 구조가 같으니 더 돌리지 않는다.

사례 · 유사 사례 체크리스트

무슨 일이 질문을 받아 사실을 뽑고 부족하면 되묻는 앞단 모델이 있다. 이 모델이 「사실이 충분인가」를 더 잘 가리게 하려고, 비슷한 과거 사례에서 빠져 있던 사실 목록을 가져와 체크리스트로 붙였다. **왜 몰랐나** 가져온 이웃의 67%가 같은 세목이었다. 검색은 잘 되는 것처럼 보였다. **잡은 방법** 정답이 「충분」인 질문과 「불충분」인 질문으로 나눠 이웃 항목의 상태를 켜다. 이웃이 「빠진 사실」을 갖는 비율이 두 쪽 모두 45%로 같았다. 항목의 53%는 이번 질문과 상관없는 것이었다. 세무에서 결정적 사실은 사례마다 다르다. 이 체크리스트를 붙인 회차는 50.8%로 기준선 50%에서 움직이지 않았다.

4.3 커버리지는 갈때기로 재고 계기판으로 쓴다

문장 사전은 한 번 만들고 끝나는 물건이 아니라 회차마다 다시 만드는 물건이다. 한 회차(재구조화)는 원 번들 413건을 받아 SI가 목차를 짓고, 번들을 세목에 배정하고, 세목마다 문장을 합성한다. 커버리지는 원 번들 중 몇 건이 문장의 근거로 인용됐는가다. 숫자 하나라 올리고 싶어진다. 대표는 이것을 「목표가 아니라 계기판」이라고 못 박았다. 계기판은 어디서 새는지 보여 줄 때만 쓸모가 있다. 고정 표본으로 재기, 조건당 3회 반복, 제품과 같은 동시성, 평균 다음엔 건 단위로 겹쳐 보기, 원시값 남기기 같은 측정의 일반 원칙은 → 1.1 · 1.2 · 1.3에서 다룬다. 이 절은 지식 파이프라인에 고유한 것만 남긴다.

에러 없는 완주를 믿지 말고 갈때기의 각 지점을 따로 센다

원문이 지식이 되기까지 거치는 단계를 갈때기로 그리고, 지점마다 몇 건이 빠졌는지를 회차 결과에 남긴다. 이 제품에서는 배정 → 투입 → 인용 세 지점이었다. 지점이 다르면 처방이 정반대다. 계측을 먼저 넣고 나서 고친다. 계측 없이 고치면 유실 지점이 옮겨 갈 뿐인데 그것도 모른다. 지표가 좋아 보이는 실패 모드도 함께 막는다. 이를테면 목차를 합치는 단계가 같은 이름 30개를 돌려주면 선택지가 하나뿐이라 배정률이 오히려 오른다 → 1.2.

사례 · 커버리지 20%

무슨 일이 문장 사전의 첫 완주판이 원 번들 413건 중 83건(20%)만 문장으로 만들었다. 원문 80%가 버려졌다. **왜 몰랐나** 파이프라인은 에러 없이 끝났다. 분류 결과를 저장하지 않아 미분류가 화면 어디에도 없었다. **잡은 방법** 문장의 출처 번들을 집계하고, 갈때기 세 지점을 화면에 나란히 띄웠다. 미분류가 크면 목차가 좁은 것이고, 미투입이 크면 세목이 커서 합성 예산에 밀린 것이다. 원인은 하나가 아니었다. 분류를 묶음으로 처리해 표본 미분류가 45%에서 84%로 뛰어 있었고, 목차를 합치는 단계는 타임아웃 때문에 한 번도 실행된 적이 없었고, 합성 단계에서 인용이 새고 있었다. 하나씩 고치자 커버리지가 37.3% → 54.0% → 63.9%로 올랐다. 이후 회차마다 DB 교차 확인으로 궤적을 따라갈 수 있었다. 자동 경로의 천장은 63~65%로 봤다.

여러 코퍼스를 한 갈래에 섞었으면 코퍼스별 등장 횟수를 센다

길이와 문체가 다른 코퍼스를 한 검색 갈래에 섞으면 짧은 쪽의 코사인이 낮게 나와 긴 쪽에 밀린다. 합친 갈래의 점수가 좋아 보여도, 어느 코퍼스가 실제로 났는지는 따로 세어야 드러난다. 짧은 쪽을 살려야 하면 갈래를 나누거나 코퍼스별 컷을 둔다.

사례 · 판례 요지

무슨 일이 국세 판례의 요지와 국세청 질의회신을 한 갈래로 넣었다. 판례 요지는 컷 0.40에서 158문항 중 10회만 났다. 0.45에서는 한 번도 뜨지 않았다. **왜 몰랐나** 갈래 전체의 점수는 좋아 보였다. **잡은 방법** 코퍼스별 등장 횟수를 계측했다. 판례 요지는 중앙값이 120자로 짧아서 코사인이 낮았고, 본문이 긴 질의회신에 자리를 내줬다.

평가 질문은 평가 대상 코퍼스를 보지 않고 사용자 쪽에서 만든다

문서를 보고 만든 질문은 그 문서가 잘 찾아지는 질문이 된다. 점수는 오르지만 실제 사용자를 대변하지 못한다. 평가 질문(벤치)은 코퍼스를 열기 전에 사용자의 분야와 말투에서 만든다. 비교하는 다른 갈래의 설정은 프로덕션과 똑같이 맞춘다. 벤치의 설계 원칙 전반은 → 7장.

사례 · 시민 문항

무슨 일이 판례·질의회신 코퍼스용으로 일반 사용자 말투의 질문 48문항을 새로 뒀다. 코퍼스 문서는 보지 않고 설계 문서의 분야별로만 만들었다. 함께 비교하는 사전 갈래의 컷도 프로덕션 값(0.45)으로 맞췄다. 앞 절의 컷 표와 「쓸모 있음」 54.2%는 이 문항에서 나온 숫자다.

4.4 층과 융합 — 성격이 다른 지식을 한 답에 모으기

지식에는 성격이 있다. 어떤 질문에든 지켜야 하는 규범, 용어와 제도를 풀어 주는 사전, 검수 경험에서 나온 문장, 원상담 번들은 서로 다른 일을 한다. 이 제품은 이것을 세 층으로 나눴다. L0는 규범으로, 검색하지 않고 매 질문의 지시문에 늘 넣는다. L1은 사전, L2는 검수 경험(문장 사전)으로, 둘은 질문마다 검색한다. 여러 갈래의 검색 결과는 RRF로 합쳤다. RRF(Reciprocal Rank Fusion)는 갈래마다 점수 척도가 달라도 쓸 수 있게, 점수 대신 각 갈래 안의 순위로 합산하는 방법이다. 층을 나누는 일은 그럴듯한 기법 이름보다 「갈래들이 실제로 어떻게 생겼나」에 달려 있었다.

어떤 질문에든 적용되는 규칙은 검색하지 말고 상시 주입한다

검색은 질문과 가까운 상위 몇 개(top-k)만 가져온다. 모든 답에 지켜야 할 판단 규칙을 검색 대상에 두면, 질문과 말이 안 닿은 날 그 규칙은 빠진다. 지식 문서를 층과 용도로 나눌 때는 파일 이름이나 형식이 아니라 문서가 스스로 밝힌 역할부터 읽는다.

사례 · 규범 문서

무슨 일이 처음에 한 시드 문서를 다른 시드들과 함께 사전 층(검색 대상)으로 분류했다. 그 문서는 머리말에 「본 지식 베이스는 당신의 시스템 프롬프트다」라고 적고 있었다. 상시 주입하는 규범 층이 맞았다. **왜 몰랐나** 파일 이름과 형식이 다른 시드와 같아 한 종류로 묶었다. **잡은 방법** 층 배치를 확정하면서 본문을 다시 읽고 정정했다. 문서의 역할은 내용이 아니라 문서가 자기 독자에게 거는 말에서 드러났다.

코퍼스마다 임계값을 손으로 맞추고 있다면 융합기부터 만든다

「이 코퍼스가 비면 저 코퍼스로 넘어간다」는 하드 폴백은 융합이 아니라 경쟁이다. 두 컷의 상대값이 승자를 정하므로, 코퍼스를 하나 더할 때마다 손 튜닝이 늘어난다. 성격이 다른 지식은 한 코퍼스로 뭉치지 말고 층으로 나눠 순위로 합친다. 새 코퍼스를 넣기 전에 그릇(융합기)을 먼저 만든다. 그릇 없이 넣은 새 코퍼스는 기존 코퍼스와 경쟁만 한다. 같은 이유로, 새 코퍼스를 기존 테이블에 넣기 전에 그 테이블을 읽는 갈래를 전부 쏜다. 이 제품에서는 공용 테이블을 문장 사전의 합성 원재료, 검수 선례 검색, 지식 지도 화면이 모두 읽고 있어서 새 코퍼스는 별도 스키마로 갔다 → 1.5.

사례 · 하드 폴백

무슨 일이 원 번들 갈래와 문장 사전 갈래는 한쪽이 비면 다른 쪽으로 넘어가는 구조였고, 컷을 갈래마다 따로 맞추고 있었다. **왜 몰랐나** 갈래가 둘일 때는 손 튜닝으로 버틸 수 있었다. **잡은 방법** 세 층을 설계하다가 폴백이 경쟁이라는 것을 짚었다. 로드맵 순서를 바꿔 사전 층보다 융합기를 먼저 만들었다. 검색 엔진을 그래프 기반으로 통째 바꾸는 안은 이 경계가 이미 있어서 미뤘고, 다시 열 조건을 숫자로 적어 두었다 → 1.7.

순위 융합의 이득은 갈래들이 겹칠 때만 난다

RRF가 점수를 끌어올리는 것은 같은 문서가 여러 갈래에 동시에 나와 순위가 합산될 때다. 같은 코퍼스를 두 방식(예: 벡터와 키워드)으로 찾을 때 이득이 난다. 서로 다른 코퍼스끼리는 겹칠 문서가 없으므로 RRF는 갈래를 번갈아 끼워 넣는 교차 배치로 퇴화한다. 기대치를 거기에 맞추고, 교차 배치가 좋은 근거를 밀어내는 비용을 따로 낸다.

사례 · 4갈래 벤치

무슨 일이 융합 검색기를 단일 갈래들과 나란히 벤치에 돌렸다. 융합은 사실상 교차 배치로 동작했다. 진 2건은 문장 사전의 4~5위에 있던 좋은 문장이 원 번들 갈래의 항목에 밀린 경우였다. **왜 몰랐나** RRF는 효과가 널리 알려진 기법이라 그대로 이득이 날 것처럼 보였다. **잡은 방법** 진 문항에서 어느 갈래의 몇 위가 밀렸는지 확인했다. 그 뒤로 교차 배치의 비용을 따로 계측했다.

갈래별 쿼터는 독식만 막고, 빈 자리는 다른 갈래로 채운다

「갈래마다 최대 몇 개」라는 쿼터는 공정해 보이는 장치라 부작용이 드러나지 않는다. 하드 상한으로 두면 한 갈래가 빈 질문에서 근거 수가 그냥 준다. 쿼터는 한 갈래가 자리를 다 차지하는 것만 막고, 남은 자리는 다른 갈래로 채운다(백필). 상한을 둘 때는 한 갈래가 비는 경우를 반드시 시험한다.

사례 · 3/3 쿼터

무슨 일이 융합 검색기에 원 번들 3개, 문장 사전 3개의 쿼터를 걸었다. 문장 사전이 아무것도 못 찾는 질문에서는 근거가 3개뿐이어서, 원 번들 갈래 하나만 쓸 때(5개)보다 오히려 적었다. **잡은 방법** 빈 갈래 사례를 따져 백필을 넣었다. 쿼터 변형도 켜다. 2/4는 조금 나뉘고 4/2는 같았다.

4.5 재생성 파이프라인과 사람의 손

문장 사전은 회차마다 목차와 문장을 새로 짓고, 이전 회차(세대)는 보관으로 넘긴다. 전체를 다시 만드는 구조는 개선이 빠르다. 대신 사람이 고친 것은 다음 자동 회차에 조용히 지워질 수 있다. 지식베이스가 「자라는」 것이 아니라 「매번 다시 만들어지는」 것이 된다. 가장 먼저 겪은 일은 이랬다. 세무사 편집 화면에서 「기타」로부터 옮긴 문장이 정확히 한 회차 만에 원래 자리로 돌아갔다. 보호 플래그는 옛 재합성 경로 하나만 막고 있었다. 손댄 문장을 이력으로 판정해 보존하고 새 세목에 다시 붙이자 프로덕션 2회 모두 보존 3/3, 재부착 2/3이 됐다. 다만 그 3건 중 적어도 둘은 실제 세무사 작업이 아니라 검증 중의 편집이었다. 보호 장치가 모든 갱신 경로를 막는지 보라는 일반 원칙은 → 1.5에서 다룬다.

세대교체는 상태 값으로 가르고, 건너야 할 것을 목록으로 점검한다

재생성 파이프라인에는 「다음 세대로 건너야 할 것」의 목록을 명시적으로 둔다. 본문, 분류상의 배치, 사람이 손댄 것이다. 하나를 고쳤다고 「승계는 해결됐다」고 여기지 말고 목록을 하나씩 점검한다. 특히 사람의 손댐은 다음 자동 회차가 지우지 않는지 실제 회차를 돌려 본다. 세대의 경계는 생성 시각의 간격이 아니라 시스템이 직접 적는 상태(활성 여부·세대 id)로 가르다. 시각은 운영 패턴이 바뀌면 깨지는 추정이다.

사례 · 대목 소실

문장 사전의 목차는 두 단이다. AI가 짓는 세목 위에, 사람이 세목을 묶어 두는 대목이 있다. **무슨 일이** 재구조화가 새 세대 세목을 대목 없이 만들어서, 대목 배치가 회차마다 100% 사라졌다. 세무사가 손으로 배치해도 그날 밤 예약 회차가 지웠다. 화면에서 세목이 전부 「미분류」 아래로 보인 원인이었다. **왜 몰랐나** 바로 전에 문장 승계를 고쳐서 세대 승계는 끝났다고 여겼다. 대목은 이름만 남아 화면에 계속 보였다. **잡은 방법** DB를 실측하니 대목이 붙은 16건이 전부 보관된 옛 세대였다. 대목 승계를 넣은 첫 회차에서 승계된 세목 19/30이 이전과 같은 대목에 앉았다.

사례 · 세대 구분

무슨 일이 분석 스크립트가 생성 시각 간격 1시간으로 세대를 갈랐다. 붙어서 돈 두 회차가 합쳐져 「최신 세대」를 세목 60개로 잡았다. **왜 몰랐나** 회차는 보통 멀리 떨어져 돌아서 시각 간격이 늘 통했다. **잡은 방법** 세대 내부 쌍을 다시 뽑는 출발점에서 개수가 이상해 걸렸다. 활성 상태 값으로 직접 뽑아 30개를 얻었다.

표시(레이블)와 정체성(id)을 가르고, 그릇의 성격대로 지우는 법을 정한다

바뀔 수 있는 표시값을 정체성으로 쓰지 않는다. 레이블이 안 바뀌는 동안은 문자열 키도 잘 동작하지만, AI가 이름을 매번 새로 짓는 구조에서는 이름이 바뀌는 순간 그 아래 지식이 고아가 된다. 지우는 법은 그릇에 무엇이 들었는지로 정한다. 지식·이력·기여가 든 그릇은 지우지 않고 연결을 끊는다. 끊긴 부모 아래의 자식은 「안 보임」보다 「미분류로 보임」을 택한다. 끊는 사유는 컬럼이 아니라 이벤트 기록으로 쌓는다. 컬럼이면 마지막 사유만 남는다.

사례 · 세목과 대목

무슨 일이 세목 이름 문자열의 유일성 제약이 사실상 기본키 노릇을 했다. 이름이 바뀌면 그 세목의 문서가 고아가 됐다. **왜 몰랐나** 목차를 사람이 정할 때는 이름이 거의 안 바뀌었다. **잡은 방법** 목차를 AI가 회차마다 새로 짓는 설계로 가면서 이름이 자주 바뀔 것이 확실해졌다. 안정 UUID 테이블을 들었다. 지우기도 둘로 갈랐다. 세목은 문장·이력·기여가 든 그릇이라 사유를 적어야 하는 「연결 끊기」로, 대목은 지식이 없는 정리 계층이라 그냥 삭제로 했다.

분류 안 되는 잔여물은 사람에게는 보이고 검색에는 안 보이는 칸에 둔다

어느 범주에도 안 맞는 잔여물은 버리면 지식이 줄고, 섞으면 검색이 오염된다. 사람의 트리에는 보이되 기계 검색에서는 빠지는 칸에 격리한다. 그리고 가공하지 않은 채로 둔다. 잔여물을 합성하면 조항처럼 생긴 가짜 문장이 생기고, 사람이 그것을 다른 세목으로 옮긴 뒤에는 진짜 문장과 구별되지 않는다. 분류기에도 「모르겠다」를 돌려줄 칸을 따로 두는 쪽이 같은 원리다 → 3장.

사례 · 「기타」 세목

무슨 일이 어느 목차에도 안 맞는 번들이 남았다. 「기타」 세목을 만들어 트리에는 보이고 검색에서는 빠지게 했다. 「기타」는 합성하지 않고 원문 1건을 문장 1개로 그대로 넣었다. 커버리지 계산에도 더하지 않았다. 검색에 안 잡히는 문장이 답변을 덮는 것처럼 세면 안 되기 때문이다. **잡은 방법** 가장 불리한 조건으로 확인했다. 「기타」 문장 자신의 벡터로 검색해도 상위 5건에 「기타」 소속이 0건이었다.

4.6 새 소스를 들일 때 — 수집 경로를 실측하고 한 번에 하나씩

자체 검수 경험만으로는 답할 수 있는 범위가 좁아서, 공개된 국세 판례의 요지와 국세청 질의회신을 새 코퍼스로 들였다. 외부 소스는 그럴듯하게 틀리기 가장 좋은 자리다. 건수는 채워지고, 파일은 열리고, 응답은 「정상」이다. 이 팀은 외부 지식을 넣을 때 여섯 규칙을 정했다. 계측 먼저, 정밀도 감시, 한 번에 한 소스, 라이선스, 권위 등급, 소급 가능하다 → 1.1. 이 절은 그 규칙을 실제로 지키며 배운 것이다.

소스의 성격이 답할 수 있는 질문을 정한다 — 주제별 적중률로 먼저 쟀다

판례는 분쟁의 기록이다. 다툼이 드문 주제는 판례에 나오지 않는다. 소스를 고를 때는 「이 소스가 어떤 질문을 담는가」를 주제 키워드별 적중률로 먼저 쟀다. 소스를 하나 더할 때는 「무엇이 비었나」를 먼저 수치로 보이고, 소스별 검색 방식과 적중률을 따로 쟀다. 같은 출처 계열이라고 같은 성질을 가정하지 않는다.

사례 · 판례와 질의회신

무슨 일이 판례 요지 487건을 본수집했다. 전체 키워드 적중은 63%였지만 의료비·월세·교육비 공제 같은 키워드는 0/10에서 1/17이었다. **왜 몰랐나** 판례 검색은 본문 한 줄만 걸려도 결과에 넣어서 건수는 채워졌다. 제목 적중 0이 신호였다. **잡은 방법** 키워드별 적중률을 따로 쟀다. 대표가 그 숫자를 보고 두 번째 소스로 질의회신을 정했다. 질의회신은 제목 검색이라 적중이 100%였다. 넓히고 나자 일반 사용자 문항의 「쓸모 있음」이 25.0%에서 54.2%로 올랐다. 두 소스는 같은 출처 계열이었지만 성격은 정반대였다.

외부 API의 「없음」은 실패일 수 있다 — 빈 결과 비율을 지표로 둔다

「결과 없음」은 오류가 아니라 정상 응답으로 온다. 수집기는 빈 결과 비율을 지표로 둔다. 비율이 높으면 다른 경로(웹 페이지, 다른 기관)로 같은 문서를 받아 교차 확인한다. 경로를 바꿀 때는 그 사이트의 robots 규칙을 확인해 허용된 쪽만 쓴다.

사례 · 판례 요지 공백

무슨 일이 공공 법령 API의 상세 조회가 국세 판례에 「일치하는 판례가 없습니다」를 돌려줬다. 기존 수집본에 요지가 740건 중 57건뿐이던 원인이었다. **왜 몰랐나** 오류가 아니라 정상적인 빈 응답이었다. **잡은 방법** 요지 공백률을 보고 경로를 실측했다. 법령 사이트의 웹 판례 페이지가 302 리다이렉트로 국세청 법령 사이트의 문서 id를 넘겨준다는 고리를 찾았다. 검색은 법령 사이트에서 하고, 국세청 사이트는 robots가 허용하는 상세 조회만 썼다.

수집기의 정렬·상한·인코딩·식별자는 추측하지 말고 명시한다

수집기의 기본값은 조용히 틀린다. 정렬은 「제목 적중 우선」처럼 직접 정한다. 상한은 「이번 실행」이 아니라 누적 기준으로 건다. 인코딩은 추정에 맡기지 않고 적는다. 문서 대조는 URL이 아니라 사건번호 같은 안정 식별자로 한다. 실패 재시도 로직을 짜는 대신 다시 돌리면 이어서 받도록 만든다. 외부 데이터의 필드는 이름이 아니라 실제 값 몇 건을 열어 뜻을 확인한 뒤 쓴다 → 1.4.

사례 · 판례 수집기

무슨 일이 네 가지가 한꺼번에 어긋나 있었다. 검색이 선고일자순이라 본문 한 줄로 걸린 무관한 사건이 앞에 왔다. 상한이 「이번 실행」 기준이라 다시 돌릴 때마다 더 받았다. 인코딩 자동 추정이 짧은 UTF-8 JSON을 키릴 문자로 읽었다. 사용자가 건넨 URL 조각은 그가 클릭한 문서가 아니었다. 그 사이트는 화면 상태를 URL에 남기지 않았다. **왜 몰랐나** 건수는 채워지고 파일은 열렸다. **잡은 방법** 파일럿 20건을 손으로 대조했고, 다시 돌렸을 때 새로 받는 건이 0인지 확인했다. 질의회신을 넣을 때는 「전문」이라는 이름의 필드에 회신만 있고 요지가 없었다. 본문을 「요지 + 회신」으로 직접 조립했다.

대화를 지식 풀로 들이는 비식별은 실데이터 감사와 화면 확인까지 해야 끝난다

사용자의 상담 대화를 비식별해 세무사들이 함께 보는 풀로 들일 때, 규칙 기반 마스킹의 단위시험은 우리가 상상한 문장만 시험한다. 실데이터 전수 감사와 실제 화면 확인을 완료 조건에 넣는다. 실데이터에 개인정보가 거의 없다면 걸린 것은 대부분 오탐이다. 과잉 마스킹과 누락이 교환될 때는 누락을 막는 쪽으로 정하고, 남는 한계는 화면에 고지한다. 화면 쪽 이야기는 → 6장.

사례 · 마스킹 규칙

무슨 일이 마스킹 DB 시험 87/87(양성 22·음성 21 포함)을 통과한 뒤에도 결함이 여섯 나왔다. 「공간이 원장」이 이름으로 잡혔다. 직함이 앞에 오는 「원장 ○○○입니다」는 놓쳤다. 고친 규칙이 「세무사 연결해 주세요」를 「세무사 [이름] 주세요」로 바꿨다. 제목 사본 때문에 번호 하나가 「전화 3건」으로 켜졌다. 풀 카드에 계정 표시명이 그대로 났다. 문장 속 쉼표 앞 전화번호를 놓쳤다. **왜 몰랐나** 단위시험은 상상한 문장만 봤다. 표시명 노출은 정규식 시험으로는 원리상 잡을 수 없다. **잡은 방법** 실데이터 218건 감사, 직접 쓴 문장으로 처음부터 끝까지 돌린 시험, 브라우저 화면 확인을 차례로 했다. 조사 「은」을 규칙에서 빼면 「지은·하은」 같은 이름이 새어서, 과잉 쪽을 감수했다.

작업 전후 점검표

- ☐ 지식 한 행이 무엇을 담는지, 무엇 단위로 임베딩되는지, 행끼리 관계가 있는지 표로 적어 보았는가?
- ☐ 벡터가 긴 글의 평균이 되고 있지 않은가? 검색·크레딧·수정 이력의 단위가 같은가?
- ☐ 분류기에 넣는 텍스트가 판정 신호뿐인가? 다른 주제로 흐른 답변이 섞여 있지 않은가?
- ☐ 이 임계값은 무엇을 가리는 값인가? 다른 소스나 다른 쓰임에서 물려받은 숫자는 아닌가?
- ☐ 선을 긋기 전에 컷 없이 돌려 점수대별 정답 등급표를 만들었는가? 차이가 채점 흔들림보다 큰가?
- ☐ 검색 결과를 판정 신호로 쓰려 한다면, 정답 클래스별로 그 신호의 분포가 다른가?
- ☐ 파이프라인이 에러 없이 끝났을 때, 갈때기의 지점마다 몇 건이 빠졌는지 회차 결과에 남는가?
- ☐ 여러 코퍼스를 한 갈래에 섞었다면 코퍼스별 등장 횟수를 썼는가? 짧은 쪽이 밀려나고 있지 않은가?
- ☐ 평가 질문을 코퍼스를 보기 전에 만들었는가? 비교 갈래의 설정이 프로덕션과 같은가?
- ☐ 모든 답에 지켜야 할 규칙이 검색 대상에 들어가 있지 않은가? 문서가 스스로 밝힌 역할을 읽었는가?
- ☐ 융합하는 갈래들이 실제로 겹치는가? 쿼터를 걸었다면 한 갈래가 비는 질문을 시험했는가?
- ☐ 새 코퍼스를 넣을 테이블을 읽는 소비자를 전부 적었는가?
- ☐ 재생성 파이프라인에 「다음 세대로 건너야 할 것」 목록이 있는가? 사람이 손댄 것이 다음 자동 회차를 실제로 살아남는 것을 보았는가?
- ☐ 바뀔 수 있는 이름이 키 노트를 하고 있지 않은가? 잔여물이 검색을 오염시키지도, 버려지지도 않는가?
- ☐ 수집기의 빈 결과 비율, 주제별 적중률, 정렬·상한·인코딩·식별자를 확인했는가? 필드 이름이 아니라 실제 값을 열어 보았는가?

5장

백엔드·인프라 엔지니어

DB 규칙·권한·외부 LLM 호출·배포·서버를 맡는 사람

이 장은 DB 스키마와 권한 정책을 짜는 사람, 외부 LLM API를 부르는 서버를 운영하는 사람, 배포 경로와 서버 한 대를 책임지는 사람을 위한 것이다. 사례는 한 AI 세무 상담 챗봇에서 나왔다. 사장님이 세무 질문을 하면 LLM이 근거 지식을 찾아 답한다. 이렇게 검색한 근거를 프롬프트에 넣어 답하게 하는 방식을 RAG(검색 증강 생성)라고 한다. 세무사는 그 답을 검수하고, 검수 결과는 다시 지식으로 들어간다. 모델은 국내 LLM API 하나만 썼다. 프론트는 호스팅 서비스에, 파이썬 백엔드는 클라우드 VM 한 대에, 데이터와 인증은 관리형 Postgres(Supabase)에 있다. 이 자리의 실수는 대개 성공 응답을 낸다. 권한에 막힌 쓰기는 오류 대신 「0행 갱신」으로 끝난다. 권한 없는 삭제 API는 200을 돌려준다. 프론트만 배포된 날에도 화면은 잘 뜬다. 두 번 재시도한 호출은 「느린 성공 한 건」으로 기록된다. 모두 그럴듯하다. 그래서 이 장은 매번 같은 질문으로 돌아간다. **이것이 그럴듯하게 틀렸다면, 누가 언제 알아챌까?** 첫 태그가 백엔드인 교훈 48개 가운데 모든 직무 공통인 21개는 1장이 주인이다. 여기서는 나머지 27개를 일곱 갈래 25개 원칙으로 묶고, 공통 교훈은 필요한 자리에서만 짧게 짚는다. 일곱 갈래는 DB가 강제하는 규칙, 데이터를 바꾸는 경로와 상태값, 오류 없이 비는 SQL, 외부 LLM API 운영, 둘 이상의 배포 자리, 인프라의 수요와 비용, 장시간 작업의 환경이다.

이 장의 원칙

1. 「한 번에 하나만」 같은 운영 규칙은 앱 코드가 아니라 DB 제약에 맡긴다
2. 보안·정합 불변식은 DB 함수와 트리거 하나로 몰고, 표시는 화면이 정한다
3. 권한 축소는 정책만이 아니라 권한을 여는 데이터의 수명까지 본다
4. 권한 정책이 부르는 함수는 비로그인에서도 실행되게 두고 거짓을 돌려준다
5. 쓰기·삭제의 성공은 응답 코드가 아니라 대상의 실제 상태로 판정한다
6. 파생 데이터는 모든 쓰기 경로에서 즉시 갱신하고, 주기 작업은 안전망으로 둔다
7. 데이터를 바꾸는 경로는 늘리지 않는다 — 조회는 넓게, 기본 선택은 좁게
8. 상태값을 새로 만들면 그 값을 걸러야 할 쿼리를 전부 함께 본다
9. 연결 칼럼을 믿지 말고 실제 채움 상태와 대상 밖 겹침을 센다
10. 순번은 그것을 매기는 쪽이 하나일 때만 순서를 말한다
11. 상태 전이는 마이그레이션 승인 전에 실DB 롤백 트랜잭션에서 끝까지 돌린다
12. 암묵 상태에 기대지 말고 문장을 나누거나 즉시 변수로 잡는다
13. 외부 LLM 호출은 호출 단위 번호표 게이트로 막고, 혼잡은 정상 응답으로 알린다
14. 스레드를 붙이면 SDK가 늦게 불러오는 모듈을 미리 불러 둔다
15. 처리 경로는 실행한 컴포넌트에서 기록하고, 전환 전후는 시각으로 가른다
16. 새 경로는 스위치로 끌 수 있게 하고, 끄면 기존 경로와 똑같이 만든다

17. 계약 변경은 서버가 옛·새 클라이언트를 함께 받는 중간 단계를 거친다
18. 배포·설정 자리가 둘 이상이면 「조용한 반쪽」을 점검 목록에 넣는다
19. 브랜치를 올릴 때는 딸려 가는 커밋과 그것이 부를 배포를 먼저 본다
20. 같은 앱을 한 번 더 띄우기 전에 프로세스 안 싱글톤을 목록으로 뽑는다
21. 스모크가 실패하면 요청이 서버 로직에 닿았는지부터 본다
22. 인프라 수요는 코드의 실제 의존성과 실측 병목으로 답한다
23. 클라우드 비용은 무료 한도를 먼저 빼고, 견적이 아니라 실측으로 확정한다
24. 다른 설정이 참조하는 값을 먼저 확정하고, 콘솔 표시보다 실상태를 믿는다
25. 장시간 작업은 낮에 돌리고, 진행은 작업 전체를 합산해 센다

5.1 규칙은 DB가 강제한다

이 제품에는 사장님·세무사·관리자 세 역할이 있고, 화면마다 보이는 범위가 다르다. 그런데 화면이 보여 주지 않는 것과 DB가 막는 것은 다르다. Supabase는 브라우저가 DB에 거의 직접 붙는 구조라서, 화면을 건너뛰고 API를 부르면 DB가 허락한 만큼 다 보인다. 그래서 행마다 「이 사용자가 이 행을 보거나 바꿀 수 있는가」를 DB가 판정하는 행 수준 보안(RLS, Row Level Security)이 실제 경계가 된다. 이 절의 원칙은 하나의 뿌리에서 나온다. 클라이언트가 규칙대로 부르는 동안에는 어느 층에 규칙을 두든 똑같이 동작한다. 차이는 누군가 규칙을 건너뛸 때만 난다. 접근 범위를 역할별 토큰으로 직접 불러 세는 일 자체는 모든 직무의 원칙이다→ 1.5.

「한 번에 하나만」 같은 운영 규칙은 앱 코드가 아니라 DB 제약에 맡긴다

운영 규칙 가운데 「대기 중인 제안은 하나만」「같은 대화에 확정은 한 번만」처럼 개수와 유일성을 말하는 것은 DB 제약으로 적을 수 있다. 앱 코드로 막으면 두 요청이 거의 동시에 들어오는 순간 둘 다 통과한다. 이런 경합은 드물어서 화면 검사로는 안 나온다. 생기면 어느 쪽이 반영될지 아무도 모른다. 부분 유일 인덱스(조건을 만족하는 행 사이에서만 유일성을 거는 인덱스)는 이런 규칙에 잘 맞는다. 지식을 바꾸는 일은 큐를 거치게 하고, 승인 전까지는 원문이 그대로 검색되게 둔다.

사례 · 지식 수정 제안

세무사는 지식 조각(검색 단위가 되는 짧은 글. 이하 「지식 조각」)을 화면에서 보다가 고칠 점을 제안할 수 있다. 무슨 일이 제안을 대기 큐 테이블로 받고, 관리자가 승인할 때 다시 임베딩(글을 검색용 숫자 벡터로 바꾸는 일)하게 했다. 같은 지식 조각에 대기 제안이 둘 쌓이면 어느 쪽이 이길지 정해져 있지 않았다. 왜 몰랐나 두 세무사가 같은 조각을 동시에 고치는 일은 데모에서 한 번도 일어나지 않았다. **잡은 방법** 구현 단계에서 미리 막았다. 부분 유일 인덱스로 지식 조각당 대기 제안을 1건으로 묶었다. 실DB에서 제안 → 반려, 제안 → 승인(실제 재임베딩) → 원복까지 돌려, 검토자와 원작성자 기록이 그대로임을 확인했다.

보안·정합 불변식은 DB 함수와 트리거 하나로 몰고, 표시는 화면이 정한다

「이 메시지를 보낸 사람은 누구인가」「이 방은 수락 뒤에만 열린다」 같은 불변식은 클라이언트가 보낸 값을 믿지 않는 자리에 둔다. 보낸 사람·역할·시각은 트리거가 호출자에게서 정하고, 클라이언트 값은 무시한다. 여러 단계가 함께 일어나야 하는 일(비식별 마스킹, 동의, 공용 풀 등재)은 DB 함수 하나로 묶어 중간에 꿰길 자리를 없앤다. 내부 함수는 사용자가 직접 부를 수 없게 막는다. 반대로 「무엇을 그릴지」는 화면이 정한다. 권한은

DB가, 표시는 화면이 말고 두 층을 섞지 않는다. 그리고 판정은 함수 하나에 모은다. 열람 판정이 정책마다 흩어져 있으면 한 곳만 고쳐진다.

사례 · 인앱 상담 방

사장님이 세무사에게 직접 상담을 신청하면, 세무사가 수락한 뒤 둘만의 대화 방이 열린다. 대화는 비식별을 거쳐 다른 세무사도 참고할 수 있는 공용 풀에 올라갈 수 있다. **무슨 일이** 설계 단계에서 「원문을 풀로 빼낼 경로」와 「다른 사람 이름으로 메시지를 보낼 경로」를 하나씩 따졌다. 클라이언트가 보낸 사람 id를 적어 보내게 두면 누구든 남을 사칭할 수 있었다. **왜 몰랐나** 우리 화면은 늘 자기 id를 보낸다. 규칙을 지키는 클라이언트만 있으면 어느 설계를 똑같이 동작한다. **잡은 방법** 마스킹·동의·등재를 DB 함수 하나로 묶었다. 방 개설은 수락 전이 안에서만 일어나는 함수로만 하게 하고, 동시 방 수 상한을 넘으면 수락까지 되돌리게 했다. 보낸 사람·역할·시각은 트리거가 정한다. DB 시험에 「사칭 불가」와 「내부 함수 직접 실행 불가」를 넣었다. 한편 관리자에게는 RLS상 본문 열람이 열려 있지만, 사적 대화라는 대표의 판단으로 화면이 본문을 그리지 않게 했다. 권한은 DB, 표시는 화면이라는 분업의 실례다.

권한 축소는 정책만이 아니라 권한을 여는 데이터의 수명까지 본다

RLS 정책은 대개 「이 사용자가 이 데이터와 어떤 관계인가」를 다른 테이블에서 찾는다. 그러면 권한의 실제 범위는 정책 문장과 그 관계 데이터의 곱이 된다. 정책을 좁혀도, 관계 데이터가 닫히지 않고 쌓여 있으면 범위는 그대로다. 축소 전에 기대 효과를 숫자로 계산해 두고, 적용 뒤 실측과 맞춘다. 숫자가 덜 줄었으면 정책이 아니라 데이터를 본다. 권한을 여는 데이터에는 닫히는 절차를 둔다. 닫기는 일회성 SQL(또 쌓인다)도 자동 실행(운영자가 못 막는다)도 아닌, 데이터 손실이 없고 되돌리기 쉬운 운영 버튼으로 만든다.

사례 · 대화 열람 축소

세무사에게는 검수할 대화를 묶음으로 배정하는데, 이 묶음을 「일감」이라고 불렀다. **무슨 일이** 세무사 토큰으로 API를 직접 불러 보니 대화 218건 전부를 본문까지 읽을 수 있었다→ 1.5. 정책을 좁혀 「검수 이력이 있거나, 열린 일감에 실린 대화」만 보이게 했다. 그런데 실측은 218 → 206건으로 거의 그대로였다. **왜 몰랐나** 정책은 맞게 바뀌었다. 문제는 정책이 참조하는 데이터였다. 마감이 7월에 지났는데 열린 채 남은 100건짜리 일감 2개(와 진행 중 4개)가 「열린 일감」 조건으로 대부분을 열어 주고 있었다. 일감에는 「마감이 지나면 닫는다」는 절차가 아예 없었다. **잡은 방법** 축소 때 「일감 조건을 빼면 170~194건」이라는 기대값을 계산해 둔 덕에 차이를 바로 알아봤다. 일감을 닫는 운영 버튼을 만들어 닫자 정확히 그 값(170/193/194)이 나왔다.

권한 정책이 부르는 함수는 비로그인에서도 실행되게 두고 거짓을 돌려준다

Supabase Realtime은 DB 행이 바뀔 때 구독한 클라이언트에게 이벤트를 밀어 주는 기능이다. 이벤트를 누구에게 보낼지는 RLS로 거른다. 이때 정책은 구독을 건 시점의 토큰으로 평가된다. 그래서 로그인 전 비로그인(anon) 상태로 붙은 채널은 로그인한 뒤에도 그 사람의 행을 받지 못한다. 더 나쁜 함정은 반대쪽에 있다. 정책이 부르는 함수에서 anon 실행 권한을 빼면, anon 구독자에 대한 평가가 권한 오류를 낸다. 그러면 그 이벤트가 그 구독자뿐 아니라 모두에게서 통째로 사라진다. 「권한을 줄이는 쪽이 안전하다」는 직관이 다른 구독자 전체를 끊는다. 정책 함수는 anon 실행 권한을 남기고, 호출자가 없으면 거짓을 돌려주게 짤다. 인증 상태가 바뀔 때마다 채널을 새 토큰으로 갈아 끼운다.

사례 · 실시간 메시지

무슨 일이 상담 방에서 사장님이 보낸 메시지가 세무사 화면에 오지 않았다. 모든 페이지가 로그인 전에 anon으로 한번 구독하는데, 메시지 정책이 부르는 「방 구성원인가」 함수에서 anon 실행 권한을 빼 두었다. **왜 몰랐나** 오류가 화면 어디에도 나오지 않았다. 서버 쪽 Node 클라이언트 하나로 시험하면 이벤트가 잘 왔다. **잡은 방법** anon 구독자를 하나 붙였을 때와 안 붙였을 때를 나란히 돌려, anon 하나가 붙으면 로그인한 세무사도 못 받는다는 것을 확정했다. 함수 권한을 되돌리고, 함수 안에서 호출자가 없으면 거짓을 돌려주게 했다. 구독 상태 신호가 「연결됨」이어도 실제 수신이 시작되지 않는 문제 같은 화면 쪽 이야기는 → 6장.

쓰기·삭제의 성공은 응답 코드가 아니라 대상의 실제 상태로 판정한다

권한 층은 막을 때 오류를 내지 않는 경우가 많다. RLS 아래의 UPDATE는 「0행 갱신」으로 끝나고 → 1.6, 어떤 스토리지 API는 권한이 없어도 200과 빈 배열을 돌려준다. 그래서 권한 정책은 SQL이 아니라 실제 사용자가 지나는 API 경로로, 역할별로 실측한다. 삭제가 성공했는지는 응답이 아니라 대상이 사라졌는지로 본다. 파일을 교체 업로드할 때는 새 파일 저장이 확정된 뒤에만 옛 파일을 지운다.

사례 · 프로필 사진 저장소

무슨 일이 세무사 프로필 사진을 저장하는 버킷에 「본인 폴더만 쓰고 지운다」는 정책을 걸었다. 그런데 스토리지의 일괄 삭제 API는 권한이 없어도 200과 빈 배열을 돌려줬다. 저장소 객체 테이블에 SQL DELETE를 하면 트리거가 막아서, 정책을 SQL로 검증할 수도 없었다. **왜 몰랐나** 응답 코드가 성공이었다. 남의 사진을 지우려다 실패해도, 내 사진을 지우는 데 성공해도 똑같이 보였다. **잡은 방법** 역할별로 실제 스토리지 API 경로를 불러 7항목을 실측했다 (남의 폴더 업로드 403, 본인 삭제 200 뒤 실제로 사라짐 등). 교체 업로드는 새 객체가 저장된 뒤에 옛 객체를 지우게 했다.

5.2 데이터를 바꾸는 경로와 상태값

이 제품의 지식은 계속 바뀐다. 세무사의 코멘트가 확정되면 지식 조각이 생기고, 철회되면 빠지고, 수정이 승인되면 다시 임베딩된다. 그 위에 지식 사이의 연결 그래프, 기여 집계, 분류 같은 파생 데이터가 얹힌다. 데이터를 바꾸는 경로가 여럿이면 후속 처리가 한 경로에서만 빠진다. 새 상태값을 만들면 그 값을 몰라야 할 쿼리가 다른 파일에서 그 값을 집어 간다. 이 절의 실수는 만든 자리에서는 잘 동작한다는 공통점이 있다. 틀린 것은 늘 다른 자리에 있다.

파생 데이터는 모든 쓰기 경로에서 즉시 갱신하고, 주기 작업은 안전망으로 둔다

파생 데이터를 주기 작업만으로 갱신하면 「언젠가는 맞는다」. 낡은 구간이 짧아서 테스트에서는 안 보이지만, 방금 확정한 지식이 화면에 안 보이는 순간 사용자는 기능이 고장 났다고 읽는다. 원본을 바꾸는 쓰기 경로를 전부 나열하고, 각 경로의 응답 직전에 갱신을 부른다. 이 갱신은 실패해도 본 동작을 막지 않는다(best-effort). 수동 DB 조작처럼 경로 밖에서 일어난 변경은 주기 작업이 잡는다. 둘은 대체 관계가 아니라 역할 분담이다.

사례 · 지식 연결 그래프

세무사가 지식 전체를 거미줄 그래프로 보고 직접 고칠 수 있게, 지식 조각 사이의 연결선을 DB에 저장했다. **무슨 일이** 연결선은 DB의 예약 작업 기능(pg_cron)으로 5분마다만 다시 계산했다. 그래서 지식이 적재된 직후 최대 5분 동안 그래프가 낡았다. **왜 몰랐나** 주기 작업이 돌고 있으니 결국 맞았다. 테스트는 5분을 기다리지 않았다. **잡은 방법** 대표가 「새 지식이 붙을 때마다 다시 계산해야 하지 않느냐」고 물었다. 수백 건 규모에서 전체 재계산이 1초도 안 걸리는 것을 확인하고, 지식 상태를 바꾸는 네 경로(코멘트 적재, 총평 적재, 철회, 수정 승인) 모두에서 응답 직전에 재계산을 부르게 했다. 5분 주기 작업은 안전망으로 남겼다.

데이터를 바꾸는 경로는 늘리지 않는다 — 조회는 넓게, 기본 선택은 좁게

일괄 정리 같은 새 작업에 전용 삭제 API를 만들면 처음에는 편하다. 그러나 기존 경로에 붙어 있던 후속 처리(그래프 재계산, 기여 집계)를 빠뜨리기 쉽다. 데이터를 바꾸는 경로를 최소로 유지해 후속 처리가 한 곳에 붙게 한다. 위험한 일괄 작업은 서버가 후보를 넓게 보여 주고, 화면의 기본 선택만 보수적으로 둔다. 사람이 「비슷하지만 정당한」 경우까지 훑어볼 수 있어야 하기 때문이다. 같은 일을 하는 로직이 두 경로에 따로 있으면 한쪽이 조용히 데이터를 버린다는 일반 원칙은 → 1.5에서 다룬다.

사례 · 중복 지식 정리

무슨 일이 같은 내용의 지식 조각이 여러 번 쌓인 것을 소급해 정리해야 했다. 새 삭제 경로를 만들지 않고 기존 철회 API를 재사용했다. 백엔드의 중복 후보 조회는 의미 유사도로 넓게 두고, 프론트의 기본 체크만 「같은 대화의 같은 구간을 다시 제출한 것」으로 좁혔다. **왜 몰랐나** 전용 API를 만들어도 삭제 자체는 잘 됐을 것이다. 빠지는 것은 연결 그래프와 기여 집계뿐이고, 그것은 다른 화면에서 한참 뒤에 드러난다. **잡은 방법** 구현 판단으로 막았다. 관리자는 넓은 후보를 훑어보고, 기본 체크된 것만 확정했다.

상태값을 새로 만들면 그 값을 걸러야 할 쿼리를 전부 함께 본다

새 status 값은 만든 자리에서는 잘 동작한다. 그 값을 걸러야 할 WHERE는 다른 파일에 있다. 상태값을 새로 만들거나 기존 값을 재사용할 때는 세대교체·복구·상태 전환 쿼리를 전부 함께 읽는다. 기존 값에 이미 다른 뜻이 있으면 재사용하지 않는다. 재사용하면 그 뜻에 걸린 모든 동작이 새 데이터에도 걸린다. 화면 키도 뜻이 다르다면 나눈다.

사례 · 지식 사전의 세대교체

한때 검수 지식을 세목별로 묶어 문장 단위로 다시 쓴 「지식 사전」을 따로 두었다. 이 사전은 재구조화할 때마다 통째로 새로 만들고, 옛 세대는 「보관」 상태로 내린다. **무슨 일이** 어느 세목에도 들어가지 않는 잔여물을 담은 「기타」를 새로 만들어야 했다. 세대교체 함수는 「활성」 상태만 보관으로 내린다. 그래서 「기타」를 그 함수에 넣지 않으면 회차마다 죽은 「기타」가 쌓일 판이었다. 반대로 「기타」에 기존의 「끊김(사람이 내린)」 상태를 재사용하면, 세목 재연결 버튼 한 번에 182건이 검색에 들어오고 「사람이 끊었다」는 이력 배지가 기계가 만든 문서로 오염될 판이었다. **왜 몰랐나** 두 실수 모두 「기타」를 만드는 코드 안에서는 보이지 않는다. **잡은 방법** 설계 단계에서 상태 전환 쿼리를 훑었다. 분류 쪽에서 같은 모양의 잔해(활성 분류 31개 대 실제 17개)를 이미 실측으로 잡은 적이 있었다. 「기타」에는 새 상태값을 주고, 세대교체 함수에 넣었다.

연결 칼럼을 믿지 말고 실제 채움 상태와 대상 밖 겹침을 센다

스키마에 외래키 칼럼이 있으면 그 칼럼으로 연쇄 삭제하면 될 것처럼 보인다. 그런데 그 칼럼이 실제로 채워져 있는지는 스키마가 말해 주지 않는다. 값이 전부 NULL이면 연쇄 삭제는 조용히 0건을 지운다. 그래서 다른 키 조합으로 매칭하면, 이번에는 그 조합이 대상 밖 레코드와 겹쳐 남의 데이터를 함께 지운다. 연쇄 삭제 전에 연결 칼럼의 채움 상태를 실측하고, 대체 키로 매칭할 때는 대상 밖 겹침을 따로 센다. 프로덕션 삭제의 전체 절차(영향 재확인 → 백업 → 행 수 검증 → 커밋)는 모든 직무의 원칙이다→ 1.6.

사례 · 미확정 검수 삭제

무슨 일이 프로덕션에서 미확정 검수 기록을 지우면서, 검수 한 건에 딸린 문장별 코멘트를 함께 지워야 했다. 코멘트 테이블에는 검수 id 칼럼이 있었다. 그런데 567건 전부가 NULL이었다. 설계상 코멘트는 「대화 id + 평가자 id」로만 검수와 이어져 있었다. **왜 몰랐나** 칼럼 이름이 연결을 약속하고 있었다. 그 칼럼으로 지웠다면 에러 없이 0건이 지워졌을 것이다. **잡은 방법** 삭제 전 사전 안전 확인 네 가지 중 하나를 칼럼 값을 실측했다. 대상 검수들의 (평가자, 대화) 쌍으로 매칭하고, 그 쌍이 대상 밖 검수와 겹치는 경우가 0건임을 따로 센 뒤에 지웠다.

순번은 그것을 매기는 쪽이 하나일 때만 순서를 말한다

메시지 순번처럼 정렬에 쓰는 값을 두 쪽이 따로 매기면 언젠가 같은 값이 나온다. 그때 「순번이 가장 큰 것이 직전 메시지」라는 가정이 깨진다. 안정 정렬(같은 키끼리는 들어온 순서를 지키는 정렬)이 우연히 순서를 지켜 주는 동안에는 드러나지 않는다. 여러 쪽이 매기는 순번은 믿지 말고 목록 위치로 순서를 찾는다. 이런 결함은 2턴짜리 시험에서는 안 보이니, 여러 턴에 걸친 기능은 3턴 이상 시나리오로 시험한다.

사례 · 되묻기 상태

새 상담 흐름에서는 질문을 받은 앞단 모델이 판단에 필요한 사실이 빠졌으면 사장님에게 되묻는다. 「지금 무엇을 되묻는 중인가」는 직전 메시지에 실어 둔다. **무슨 일이** 프론트는 사용자 메시지에 대화 길이를, 서버는 답변 메시지에 「최대 순번 + 1」을 매겼다. 그래서 2턴부터 두 메시지가 같은 순번이 됐다. 「순번 최대값 = 직전 메시지」로 상태를 찾다가 3턴째에 되묻기 상태가 사라졌다. **왜 몰랐나** 기존 흐름도 순번으로 정렬했지만 안정 정렬이라 우연히 맞았다. 2턴 시험은 통과했다. **잡은 방법** 3턴 시나리오에서 상태가 사라지는 것을 보고 찾았다. 직전 메시지는 순번이 아니라 목록의 마지막 위치로 찾게 바꿨다.

5.3 오류 없이 비는 SQL — 마이그레이션과 상태 전이

마이그레이션(스키마를 바꾸는 SQL 묶음)과 상태 전이 로직의 실수는 대개 문법 오류가 아니다. 문장은 실행되고, 갱신 대상만 빈다. 단위 테스트와 가짜 저장소로는 실제 스키마·제약과의 충돌이 안 보인다. 그렇다고 프로덕션에서 시험하면 시험이 곧 운영 데이터 변경이 된다. 이 절의 원칙은 그 사이에 실DB 롤백 트랜잭션이라는 시험대를 두는 것이다. 쓰기만 막고 진짜 함수를 부르는 드라이런, 이름과 날짜 상수를 카탈로그로 조회하는 습관은 모든 직무의 원칙이다→ 1.4 · 1.6.

상태 전이는 마이그레이션 승인 전에 실DB 롤백 트랜잭션에서 끝까지 돌린다

상태를 바꾸는 기능은 시험이 곧 운영 데이터 변경이다. 그렇다고 가짜 저장소로만 시험하면 실제 제약·트리거·스냅샷 규칙과의 충돌을 놓친다. 데이터 접근 층의 트랜잭션·커넥션 함수를 단일 커넥션과 세이브포인트로 바꿔

끼운다. 그 위에서 마이그레이션과 로직을 실DB에 적용하고, 시나리오를 끝까지 돌린 뒤 되돌린다. 끝나면 트랜잭션 밖에 변화가 0인지 확인한다. 마이그레이션 승인은 이 시험을 통과한 뒤에 받는다.

사례 · 규범 확정

모든 답변의 시스템 프롬프트에 상시로 들어가는 짧은 규칙 묶음을 「규범」이라고 불렀다. 세무사가 초안을 내고, 공개 기간 동안 이의를 받고, 승인이 모이면 확정되며, 관리자는 롤백할 수 있다. **무슨 일이** 확정·이의 기간·관리자 롤백은 모두 상태를 바꾸는 기능이었다. **왜 몰랐나** 단위 테스트는 가짜 저장소 위에서 전부 통과했다. 실제 스키마와 부딪히는 자리는 거기서 보이지 않았다. **잡은 방법** 트랜잭션 함수를 세이프포인트 하나로 바꿔 끼우고, 마이그레이션과 로직을 실DB에서 돌린 뒤 되돌렸다. 이의 기간 22항목, 관리자 브레이크 12항목, 확정 시나리오 전체를 돌렸고 트랜잭션 밖 변화 0을 확인했다. 이 시험대에서 다음 원칙의 결함 하나가 잡혔다.

암묵 상태에 기대지 말고 문장을 나누거나 즉시 변수로 잡는다

SQL에는 코드에 드러나지 않는 상태가 있다. PostgreSQL에서 한 문장 안의 CTE(`WITH`로 잇는 하위 쿼리)는 모두 같은 스냅샷을 본다. 그래서 앞 CTE가 넣은 행을 같은 문장의 본 쿼리가 읽지 못한다. PL/pgSQL의 `FOUND`는 「직전 쿼리가 행을 찾았나」를 담는데, 그 뒤에 어떤 쿼리든 돌면 덮어써진다. 둘 다 문법 오류 없이 실행되고 결과만 빈다. 앞서 넣은 행을 읽어야 하는 갱신은 문장을 나눈다. 암묵 상태 변수는 조회 바로 다음 줄에서 지역 변수로 옮겨 두고 쓴다.

사례 · CTE 스냅샷

무슨 일이 규범 확정 함수가 한 문장에서 새 문서를 넣고, 새 버전을 넣고, 문서의 「현재 버전」 칸을 그 버전으로 갱신했다(`with d as (insert...), v as (insert...) update ... from v`). 현재 버전 칸이 걸리지 않았다. **왜 몰랐나** 문법 오류 없이 실행되고 갱신 대상만 비었다. **잡은 방법** 앞의 롤백 트랜잭션 시험에서 결과가 비는 것을 보고 찾았다. 문장을 나눴다.

사례 · FOUND

무슨 일이 상담 방을 여는 함수가 방을 조회한 뒤 개수를 세는 `count(*)`를 한 번 더 돌렸다. 그 쿼리가 `FOUND`를 참으로 덮어써, 없는 방을 「다시 열기」 갈래로 보냈다. **왜 몰랐나** 코드에서는 `FOUND` 검사 줄이 조회 바로 뒤에 있는 것처럼 읽혔다. **잡은 방법** 드라이버 DB 시험에서 드러났다. 조회 직후 결과를 지역 변수에 담게 고쳤다.

5.4 외부 LLM API를 운영한다

이 제품의 답변·임베딩·분류는 전부 외부 LLM API 호출이다. 서버는 그 호출을 기다리는 시간이 대부분이다. 외부 API는 우리가 고칠 수 없는 이유로 느려지고, 같은 키에 요청이 몰리면 더 느려진다. SDK는 우리 모르게 재시도한다. 모든 LLM 호출에 명시적 상한을 거는 일, 재시도와 계측을 같은 층에 두는 일, 긴 지연을 분포 모양으로 판독하는 일은 모든 직무의 원칙이다→ 1.1. 동시 호출과 자동 도구 선택에서 지연이 폭발하는 모델 쪽 이야기는 → 3장. 이 절은 그 위에서 서버가 해야 할 일을 다룬다.

외부 LLM 호출은 호출 단위 번호표 게이트로 막고, 혼잡은 정상 응답으로 알린다

동시 접속이 몰릴 때 외부 API를 보호하는 게이트는 사용자 턴이 아니라 외부 호출 하나하나에 건다. 한 턴이 여러 번 호출하기 때문이다. 게이트 슬롯은 SDK 재시도까지 감싼다. 멈춘 호출도 자리를 차지하기 때문이다.

세마포어는 순서를 보장하지 않으니, 먼저 온 호출이 먼저 들어가는 번호표(FIFO)를 쓴다. 대기 상한은 호출마다가 아니라 턴 합계로 둔다. 상한을 넘으면 오류가 아니라 「지금 붐빈다」는 평범한 안내를 HTTP 200으로 돌려준다. 500으로 내면 프론트가 오류 화면으로 처리한다. 혼잡 예외가 중간의 폴백 코드에 삼켜지지 않게 다시 던진다. 게이트가 프로세스 안에 있으면 실제 동시 상한은 $k \times \text{워커 수}$ 다. 끝으로, 「멈춤 0」이 게이트 덕인지 시간대 차이인지는 표본만으로 가를 수 없다고 적어 둔다.

사례 · 전시회 대비

전시회에서는 비로그인 방문자가 한꺼번에 챗을 쓴다. 모두 같은 API 키를 쓴다. **무슨 일이** 배포 전 동시 4 조건에서 48턴 중 1턴이 120초 동안 멈췄다. 그 전 주에 이미 긴 지연이 120초 타임아웃 + 재시도 1회 모양으로 군집한다는 것, 그리고 순차 64회에서는 멈춤이 0이라 동시 호출의 산물이라는 것을 확인해 두었다. **왜 몰랐나** 개발자 혼자 쓰는 조건에서는 한 번도 멈추지 않는다. **잡은 방법** 프로세스 안에 번호표 게이트($k=3$)를 두고, 모든 외부 LLM 호출이 지나는 한 자리에 걸었다. 첫 한 턴의 대기 합계가 30초를 넘으면 HTTP 200 혼잡 안내를 돌려준다. 배포 뒤 동시 $4 \cdot 6 \cdot 8 \times 12$ 턴, 모두 216턴에서 멈춤 0, 혼잡 안내 0이었다.

스레드를 붙이면 SDK가 늦게 불러오는 모듈을 미리 불러 둔다

요즘 SDK는 시작 속도를 위해 하위 모듈을 처음 쓰는 순간에 불러온다(지연 import). 단일 스레드에서는 처음 건드리는 쪽이 하나뿐이라 문제가 없다. 그런데 두 스레드가 서로 다른 하위 모듈을 동시에 처음 건드리면, 파이썬의 import 잠금이 서로 엇갈려 교착 오류가 날 수 있다. 서버에서는 기동 직후 첫 요청에서만 날 수 있어 드물고, 그래서 재현이 어렵다. 병렬을 붙일 때는 쓰는 하위 모듈을 모듈 로드 때 미리 import한다. 콜드스타트 경로는 따로 스모크한다.

사례 · 대조 스레드

새 답변 단계에서는 LLM이 판정을 내리고, 기존 규칙 엔진은 대조용으로 옆에서 함께 돈다. **무슨 일이** 엔진 대조 스레드와 본 스레드가 채팅 호출 모듈과 임베딩 호출 모듈을 동시에 처음 건드리자 `_DeadlockError`가 났다. **왜 몰랐나** 단일 스레드 경로에서는 한 번도 나지 않았다. 서버에서는 기동 직후에만 날 수 있다. **잡은 방법** 로컬 스모크에서 드러났다. 두 하위 모듈을 모듈 로드 때 미리 불러 두게 했다.

처리 경로는 실행한 컴포넌트에서 기록하고, 전환 전후는 시각으로 가른다

「이 답변은 어느 지식 경로로 만들었나」는 나중에 품질을 비교할 때 가장 먼저 쓰이는 칼럼이다. 이 값을 결과의 모양에서 거꾸로 추정하면, 경로가 하나일 때는 늘 맞다가 경로가 섞이는 순간 틀린다. 실행한 컴포넌트(어떤 검색기를 돌렸나)에서 기록한다. 같은 이유로 「무엇을 요청했나」칼럼은 디폴트 전환을 보여 주지 못한다. 요청값을 비워 둔 사용자는 전환 전후 모두 비어 있기 때문이다. 디폴트를 바꾸는 날은 전환 시각을 기록해 두고, 실제 처리 경로 칼럼과 시각으로 가른다. 계정 이름 같은 식별자도 짐작하지 말고 조회해서 쓴다.

사례 · 융합 검색

무슨 일이 답변 기록의 「사용한 지식 경로」칸을, 검색 결과 1위 지식 조각의 출처 종류로 채우고 있었다. 여러 지식 묶음을 한꺼번에 검색해 순위를 합치는 융합 검색을 붙이면, 1위가 어느 묶음에서 왔느냐에 따라 경로가 잘못 기록될 자리였다. **왜 몰랐나** 지식 묶음이 하나뿐일 때는 결과에서 추정해도 늘 맞았다. **잡은 방법** 융합 검색 착수 전에 문서를 정정하다가 미리 찾았다. 검색기 종류에서 기록하게 바꿨다.

사례 · 디폴트 전환 계속

무슨 일이 프로덕션 기본 검색 경로를 융합 검색으로 바꾼 뒤 전후를 비교하려 했다. 계측의 「요청한 경로」 칸은 명시 요청값만 담아서, 전환 전후 둘 다 `null` 이었다. 비교할 데모 계정 이름도 짐작한 것과 실체가 달랐다. **왜 몰랐나** 요청 칸이 있으니 경로를 가를 수 있을 것처럼 보였다. **잡은 방법** 실제 처리 경로 칸과 전환 시각으로 같았다. 계정은 목록을 조회해 확인했다.

새 경로는 스위치로 끌 수 있게 하고, 끄면 기존 경로와 똑같이 만든다

실험 경로가 기존 경로에 섞이면 회귀가 조용히 들어온다. 새 경로는 설정 스위치나 파라미터로 붙이고, 값이 없거나 모르는 값이면 기존 코드를 그대로 타게 한다. 꺼진 상태에서 기존 프로덕션 경로의 코드가 한 줄도 안 바뀌었는지 diff로 확인하고, 같은 입력으로 결과가 같은지 증명한다. 이 원칙의 일반형은 → 1.6에서 다룬다. 스위치가 있어야 운영 중에 되돌릴 수 있다. 10월에 프로덕션 챗을 새 지식 묶음 단독으로 바꿀 때도 되돌리기는 설정값 하나였다.

사례 · 새 지식 묶음의 쿼터

세무 심판례와 질의회신 원문으로 새 지식 묶음을 만들어 융합 검색에 붙였다. **무슨 일이** 새 묶음을 적재하기 전부터 새 검색 경로 코드는 배포되어야 했다. **왜 몰랐나** 해당 없음. 예방 설계다. **잡은 방법** 새 묶음의 쿼터가 0이면 그 갈래를 아예 붙이지 않게 했다. 쿼터 0이 곧 「적재 전 상태」다. 기존 융합 검색 분기는 diff로 한 줄도 바뀌지 않았음을 확인했다.

5.5 배포 — 둘 이상의 자리가 따로 성공한다

이 제품은 배포 자리가 둘이다. 프론트는 메인 브랜치에 올리면 호스팅 서비스가 자동으로 빌드한다. 백엔드는 배포용 브랜치에 올린 뒤 서버에서 배포 스크립트를 돌려야 반영된다. 한쪽만 하면 반쪽만 바뀐다. 그리고 두 자리는 저마다 「성공」을 보고한다. 서버 저장소가 부분 체크아웃 상태로 한 달 넘게 멀쩡히 돈 일, 새 요청 필드가 있으면 백엔드를 먼저 내야 하는 일은 모든 직무의 원칙이다 → 1.6. 응답 스키마가 프론트 검증기에 조용히 지워지는 일은 → 6장.

계약 변경은 서버가 옛·새 클라이언트를 함께 받는 중간 단계를 거친다

클라이언트와 서버가 따로 배포되면, 둘 사이의 계약을 바꾸는 순간 어느 순서로 내도 한쪽이 깨지는 창이 생긴다. 서버를 「옛 클라이언트와 새 클라이언트를 둘 다 받는」 중간 상태로 먼저 낸다. 그다음 클라이언트를 바꾸고, 마지막에 서버를 새 계약만 받게 조인다. 다 끝나면 중간 단계용 우회 플래그는 남기지 않는다. 이 순서는 백엔드만 보면 설정 한 줄이라 한 번에 바꾸고 싶어지는 자리에서 특히 필요하다.

사례 · 쓰기 API 인증

무슨 일이 규범 확정이 모든 답변을 바꾸게 되자, 쓰기 API 전체에 토큰 인증을 걸어야 했다. 그때까지 신원은 요청 본문으로 받고 있었다. 그 위에 「2명 승인」을 얹으면 본문만 바뀌 위조할 수 있다→ 1.5. 백엔드를 한 번에 「필수」로 바꾸면, 아직 토큰을 안 붙이는 구 프론트가 쓰기마다 깨질 판이었다. **왜 몰랐나** 백엔드 쪽에서는 인증 모드 설정값 하나였다. **잡은 방법** 백엔드를 「선택」(토큰이 있으면 검증, 없으면 통과)으로 먼저 내고, 프론트에 토큰 부착을 배포하고, 마지막에 「필수」로 바꿨다. 전환 뒤 토큰 없는 쓰기는 401, 챗은 토큰 없이 정상인 것을 확인했다. 전시회 방문자는 로그인 없이 챗을 쓰므로 챗은 처음부터 인증 대상에서 뺐다. 이후 스크립트는 개발용 토큰 발급 모듈을 거쳐 쓰기를 한다.

배포·설정 자리가 둘 이상이면 「조용한 반쪽」을 점검 목록에 넣는다

배포 자리, 환경변수 자리, 빌드 산출물이 따로 있으면 각 자리는 저마다 성공을 보고한다. 그래서 「반쪽만 반영됨」은 실패가 아니라 정상처럼 보인다. 인프라를 옮긴 뒤에는 배포 스크립트·환경변수·배포된 번들 안의 주소를 전수 대조한다. 빌드 시점에 주입되는 환경변수는 값을 바꿔도 재배포해야 반영된다. 열어 볼 수 없는 비밀값은 값을 확보하기 전에는 지우지 않는다.

사례 · 서버 이전 직후

무슨 일이 백엔드를 새 VM으로 옮긴 직후, 배포 스크립트는 여전히 구 서버를 가리키고 있었다. 프론트 호스팅 쪽에서는 브라우저에 노출되는 공개 환경변수를 비밀값으로 저장하는 길이 정책 변경으로 막혀 있었다. 다시 만들 때 적용 환경(프로덕션 + 프리뷰)을 둘 다 잡지 않으면 프리뷰가 조용히 깨진다. **왜 몰랐나** 구 서버가 아직 살아 있어서, 스크립트가 구 서버에 배포해도 「배포 성공」이 떴을 것이다. **잡은 방법** 첫 배포 전에 스크립트를 교정했다. 이전 뒤 배포된 번들 24개를 스캔해 새 주소 2곳, 구 주소 0곳을 확인했다. 인증 키처럼 값을 다시 볼 수 없는 공개 환경변수는 건드리지 않았다.

브랜치를 올릴 때는 팔려 가는 커밋과 그것이 부를 배포를 먼저 본다

작업 트리를 건드리지 않고 원격 브랜치의 참조만 올리는 것은 편하고 안전해 보인다. 그러나 참조 갱신은 내용을 보여 주지 않는다. 그 사이 쌓인 다른 커밋이 함께 가고, 그 안에 마이그레이션 파일이나 프론트 변경이 있으면 자동 빌드나 다음 배포가 그것을 실어 나른다. 올리기 전에 함께 가는 커밋 목록을 뽑고, 각 커밋이 일으킬 배포·마이그레이션을 확인한다.

사례 · 메인 브랜치 갱신

무슨 일이 다른 세션이 고치던 파일 때문에 브랜치 전환이 막혔다. 그래서 배포 브랜치를 원격 메인으로 직접 밀고, 로컬 메인 참조를 받아 오는 식으로 전환 없이 메인을 올렸다. 커밋 4개(새 지식 묶음 작업 돌과 미적용 마이그레이션 파일)가 함께 갔다. **왜 몰랐나** 명령은 한 줄로 성공했고, 무엇이 함께 갔는지는 출력에 없었다. 메인에 올라간 프론트 변경은 곧바로 자동 배포된다. **잡은 방법** 팔려 간 커밋에 프론트 변경이 없는지, DB 적용이나 백엔드 배포가 따라가지 않았는지 사후에 확인했다. 다음부터는 올리기 전에 확인한다.

같은 앱을 한 번 더 띄우기 전에 프로세스 안 싱글톤을 목록으로 뽑는다

워커 하나, 인스턴스 하나로 운영하면 「프로세스는 하나」라는 전제가 코드 어디에도 적히지 않은 채 늘 참이다. 스케줄러, 멈춘 작업을 회수하는 리퍼, 프로세스 안 게이트가 모두 이 전제에 기댄다. 워커를 늘리거나 병행 유닛을 띄우는 순간 이 전제가 깨진다. 띄우기 전에 프로세스 안 싱글톤을 목록으로 뽑는다. 병행 유닛은 백그라운드

작업을 끄고, 외부 호출의 합계 상한은 각 프로세스 상한의 합으로 계산하고, 킬 스위치는 애플리케이션이 아니라 서비스 관리자 층에 둔다. 원복할 때도 새 쪽을 먼저 멈춘다. 기동 시 멈춘 작업을 회수하는 일반 원칙은 → 1.6에서 다룬다.

사례 · 병행 유닛

새 상담 흐름을 기존 서버 옆에 두 번째 서비스로 띄워 시험하려 했다. **무슨 일이** 새 유닛도 같은 앱이라 기동하면 백그라운드 폴러를 띄웠다. 그 첫 순회의 리퍼가 기존 유닛이 돌리던 작업을 「멈춘 작업」으로 보고 죽일 수 있었다. 프로세스 안 번호표 게이트는 두 프로세스가 공유하지 못해 실제 동시 상한이 두 배가 됐다. 환경변수 킬 스위치는 쿼리 파라미터에 우선순위가 밀렸다. 격리 설계는 셋 다 몰랐다. **왜 몰랐나** 그때까지 싱글톤 전제는 늘 참이라 코드에 드러날 일이 없었다. **잡은 방법** 리퍼를 부르는 곳이 하나뿐인 것을 코드로 추적했다. 전날 서버 이전에서 「워커는 1로 유지한다」는 규칙을 정리해 둔 덕에 같은 모양을 알아봤다. 새 유닛은 백그라운드 작업을 끈 채 띄우고, 평소에는 서비스 관리자에서 꺼 둔다.

스모크가 실패하면 요청이 서버 로직에 닿았는지부터 본다

배포 뒤 스모크 요청이 422나 400을 내면 서버 오류처럼 보인다. 그런데 요청이 서버 로직에 닿지도 않았을 수 있다. 셀과 요청 도구는 비ASCII 문자를 조용히 다른 인코딩으로 바꾼다. 한글 페이로드 스모크는 인코딩을 통제하는 스크립트로 보낸다. 실패하면 먼저 그 요청이 파이프라인 기록에 남았는지 본다.

사례 · 한글 질문 스모크

무슨 일이 Windows 셸의 curl로 한글 질문 JSON을 보내자 422와 400이 났다. 파이프라인 기록에는 아무것도 없었다. **왜 몰랐나** 응답 코드만 보면 자금 배포한 서버 코드가 깨진 것처럼 보였다. **잡은 방법** 같은 페이로드를 파이썬 표준 라이브러리로 보내 정상 처리를 확인했다. 한글이 든 명령은 셸 인라인 대신 파일로 쓰는 습관은 → 1.7에서 다룬다.

5.6 인프라 — 수요, 비용, 이전

인프라 결정에는 그럴듯한 숫자가 늘 먼저 와 있다. 큰 GPU를 주겠다는 제안, 단가표로 곱한 월 비용, 계획 문서에 적힌 콘솔 절차. 모두 틀린 숫자는 아니다. 다만 우리 코드가 그것을 실제로 쓰는지, 무료 한도를 뺐는지, 콘솔이 정말 그 흐름을 지원하는지는 아직 확인하지 않은 숫자다. 서버 이전을 「같은 결과」로 판정하고 라이브러리 업그레이드와 섞지 않는 원칙은 모든 직무의 원칙이다 → 1.6.

인프라 수요는 코드의 실제 의존성과 실측 병목으로 답한다

자원 신청에는 「받아 두면 손해 없다」는 유혹이 따른다. 그 유혹이 실제 병목을 가린다. 신청서는 의존성 목록과 실측 병목으로 근거를 낸다. 앞으로의 고도화 계획이 그 자원을 실제로 쓰는지도 방식별로 표를 만들어 확인한다. 상태가 어디에 있는지도 함께 본다. 상태가 전부 외부 DB에 있으면 서버 이전에 데이터 이관이 필요 없다.

사례 · 대회 인프라 조사

무슨 일이 대회 운영 측이 GPU 서버 지원 여부를 물었다. 코드를 전수 확인하니 LLM·임베딩·벡터 검색이 모두 외부 API와 관리형 DB였고, GPU 라이브러리는 하나도 없었다. 실제 병목은 GPU가 아니라 서버 메모리 1GB였다. **왜 몰랐나** 큰 GPU 질문이 메모리 문제를 가렸다. 「언젠가 쓸지도」가 근거처럼 보였다. **잡은 방법** 의존성 파일과 백엔드 API의 로컬 파일 의존을 검색했다. 계획 중인 고도화 방식들도 API 조합으로 된다고 판정했다. GPU는 신청하지 않고 스토리지만 신청했다. 메모리 병목은 뒤에 서버 증설로 풀었다.

클라우드 비용은 무료 한도를 먼저 빼고, 건적이 아니라 실측으로 확정한다

단가표의 숫자는 맞다. 틀리는 것은 계산 순서다. 무료 한도 → 단가 순으로 계산하고, 생성 화면 건적이 아니라 비용 분석 화면의 실측 청구로 확정한다. 예산 알림은 정상 청구의 몇 배로 잡는다. 그래야 알림이 울리는 것 자체가 이상 신호가 된다.

사례 · 서버 증설

무슨 일이 메모리 병목을 풀려고 서버를 2코어·12GB로 늘리는 계획을 세웠다. 계획서의 월 비용은 37,000원이었다. 실제로는 컴퓨트 0달러에 부트 볼륨 월 2.76달러였다. 10배 넘게 과대였다. 2코어·12GB가 그 클라우드의 상시 무료 한도(4코어·24GB) 안에 통째로 들어갔다. **왜 몰랐나** 단가표는 맞는 숫자였다. 무료 한도를 빼는 단계가 계획에 없었다. **잡은 방법** 생성 화면 건적에 컴퓨트 항목이 아예 없었고, 사용량 지표가 0이었다. 예산 알림은 정상 청구(약 3.7싱가포르달러)의 몇 배인 월 10싱가포르달러로, 50%와 100%에 걸었다.

다른 설정이 참조하는 값을 먼저 확정하고, 콘솔 표시보다 실상태를 믿는다

공인 IP나 도메인처럼 다른 설정이 줄줄이 참조하는 값은 바뀌면 연쇄 재작업이 생긴다. 그 값을 먼저 확정하고, 그다음에 리버스 프록시·프론트 설정 같은 의존 설정을 바꾼다. 계획 문서에 적힌 콘솔 흐름은 가정일 수 있다. 작업 뒤 판단은 콘솔의 부가 표시가 아니라 리소스 상태와 실제 접속으로 한다.

사례 · 고정 IP

무슨 일이 계획은 「새 서버에 처음부터 고정 IP를 할당한다」였다. 콘솔에는 그 경로가 없었다. 임시 IP로 생성 → 고정 IP 발급 → 네트워크 인터페이스에서 교체의 3단계였고, 그 사이 공인 IP가 한 번 더 바뀌었다. 교체 뒤에도 콘솔은 IP 수명을 「임시」로, 인터페이스 칸을 빈칸으로 보였다. **왜 몰랐나** 계획 문서가 콘솔 흐름을 가정으로 썼다. 교체 뒤의 표시는 실패처럼 보였다. **잡은 방법** 리소스 상태가 「할당됨」인 것과 실제 접속으로 판단했다. IP를 확정하기 전에는 리버스 프록시와 프론트 설정을 건드리지 않았기에, IP가 한 번 더 바뀌어도 재작업은 0이었다.

5.7 장시간 작업의 환경 — 밤, 절전, 재시작

세무 판례와 질의회신 원문 수천 건을 공공 사이트에서 모으는 수집, 158문항을 여러 경로로 채점하는 벤치는 몇 시간에서 하루가 넘게 돈다. 이런 작업의 실패는 코드가 아니라 환경에서 온다. PC가 절전에 들어가고, 외부 서버가 밤에 느려지고, 공유 DB 연결이 끊긴다. 그런데 실패는 코드 결함과 똑같은 모양으로 보인다. 감시를 감시받는 작업과 같이 죽지 않는 곳에 두는 일, 프로덕션 DB에 쓰기 전에 메모리 안에서 재는 일은 모든 직무의 원칙이다 → 1.6.

장시간 작업은 낮에 돌리고, 진행은 작업 전체를 합산해 센다

외부 서버와 공유 DB에는 시간대별 상태가 있다. 대량 수집과 무거운 벤치는 낮에 돌리고, 밤의 실패를 코드 결함으로 단정하지 않는다. 장시간 작업 전에는 절전 같은 공통 정지 원인을 끄고, 끝나면 되돌릴지 사용자에게 확인한다. 진행 감시는 파일 하나가 아니라 그 작업의 산출 전체를 합산해 센다. 재시작하거나 나눠 돌리면 산출 파일이 늘기 때문이다. 감시가 「진행 없음」을 말하면 작업이 멈췄을 수도, 감시가 엉뚱한 것을 세고 있을 수도 있다.

사례 · 밤의 수집과 벤치

무슨 일이 밤에는 수집 쪽의 2MB 응답이 연속으로 실패했다. 낮에는 문서당 1~3초면 받던 것이다. 같은 날 23시께 벤치를 돌리는 중 프로덕션 DB 연결이 반복해 끊겼다. 2문항에 545초, 접속만 21.8초가 걸렸다. **왜 몰랐나** 낮에는 같은 코드가 문제없이 돌아다. 밤의 원인(동시에 돈 4096차원 벡터 전수 비교일 가능성)은 끝내 확정하지 못했다. **잡은 방법** 실패와 지연을 보고 멈췄고, 벤치는 다음 낮으로 미뤘다. 원인을 확정하지 못했다는 사실도 그대로 적었다.

사례 · 수집 진행 감시

무슨 일이 질의회신 수집은 하루 넘게 돌면서 PC 절전으로 두 번 멈췄고, 그때마다 다시 시작했다. 재시작하면 원본 파일 이름의 날씨가 바뀌는데, 감시는 옛 파일만 세고 있었다. **왜 몰랐나** 옛 파일 수는 그대로라 「진행 없음」처럼 보였고, 새 파일의 진행은 보이지 않았다. **잡은 방법** 감시가 오탐을 한 번 내고서야 알았다. 감시를 그 작업의 전 파일 합산으로 바꿨다. 결국 질의회신 1,995건(2018년 이후)을 모았다.

작업 전후 점검표

- ☐ 「한 번에 하나만」 같은 운영 규칙이 앱 코드가 아니라 DB 제약으로 걸려 있는가?
- ☐ 보낸 사람·역할·시각처럼 사칭할 수 있는 값을 클라이언트가 아니라 DB(트리거·함수)가 정하는가? 내부 함수를 사용자가 직접 부를 수 없는가?
- ☐ 권한을 바꾼 뒤 역할별 토큰으로 API를 직접 불러 범위를 썼는가? 그 숫자가 미리 계산한 기대값과 맞는가?
- ☐ 권한 정책이 참조하는 데이터(배정, 묶음, 방)에 달하는 절차가 있는가?
- ☐ RLS 정책이 부르는 함수가 비로그인에서도 실행되고, 호출자가 없으면 거짓을 돌려주는가?
- ☐ 쓰기·삭제의 성공을 응답 코드가 아니라 갱신 행 수와 대상의 실제 상태로 확인했는가? 연쇄 삭제에 쓸 연결 칼럼이 실제로 채워져 있는가?
- ☐ 파생 데이터를 바꿔야 하는 쓰기 경로를 전부 나열했는가? 주기 작업은 안전망으로만 남았는가?
- ☐ 새 상태값을 만들었다면 세대교체·복구·상태 전환 쿼리를 전부 읽었는가? 기존 값을 다른 뜻으로 재사용하지 않았는가?
- ☐ 상태 전이 로직을 마이그레이션 승인 전에 실DB 롤백 트랜잭션에서 끝까지 돌렸는가? 트랜잭션 밖 변화가 0인가?
- ☐ 외부 LLM 호출이 모두 한 게이트를 지나는가? 혼잡이 500이 아니라 안내 응답으로 나가는가? 실제 동시 상한이 $k \times$ 프로세스 수임을 계산했는가?
- ☐ 새 경로를 끄면 기존 경로 코드가 한 줄도 안 바뀌었음을 diff와 같은 입력으로 확인했는가? 처리 경로 칸은 실행한 컴포넌트에서 기록되는가?
- ☐ 이번 배포에서 프론트 자리와 백엔드 자리를 둘 다 반영했는가? 함께 올라간 커밋과 환경변수·번들 속 주소까지 대조했는가?
- ☐ 두 번째 프로세스를 띄우기 전에 스케줄러·리퍼·게이트 같은 프로세스 안 싱글톤을 목록으로 뽑았는가?
- ☐ 비용·자원 계획이 무료 한도, 코드의 실제 의존성, 실측 병목에서 출발했는가?
- ☐ 장시간 작업 전에 절전을 켜는가? 진행 감시가 작업 전체의 산출을 세는가? 밤의 실패를 코드 탓으로 단정하지 않았는가?

6장

프론트엔드 엔지니어

실시간 목록·적재 상태·화면 문구·응답 계약·브라우저 검증을 맡는 사람

이 장은 상담 화면과 검수 화면을 만들고, 서버가 밀어 주는 변경을 목록에 반영하고, 백엔드 응답을 받아 그리는 사람을 위한 것이다. 사례는 한 AI 세무 상담 챗봇에서 나왔다. 사장님이 세무 질문을 하면 LLM이 근거 지식을 찾아(RAG, 검색으로 찾은 문서를 답변 생성에 붙이는 방식) 답하고, 세무사가 그 답을 검수해 다시 지식으로 되돌리는 제품이다. 국내 LLM API 하나로 만들었다. 화면 쪽에는 세무사 작업 공간, 관리자 검사실, 세무사와 사장님이 직접 대화하는 상담방이 있다. 이 자리의 실수는 거의 늘 그럴듯한 화면으로 나타난다. 이벤트를 놓친 목록은 「아직 아무것도 없다」로 보인다. 삼킨 오류는 빈 칸으로 보인다. 늦게 온 이름 대신 계정 id가 자연스럽게 자리를 채운다. 보호 장치가 일을 해냈는데 화면은 빨간 「실패」를 띄운다. 검증기가 모르는 필드를 지워도 오류는 없다. 새로고침 한 번이면 증상이 사라지니 아무도 다시 보지 않는다. 이 프로젝트에서 프론트엔드가 주 직무인 교훈은 21개였다. 그중 모든 직무에 통하는 9개는 1장이 말고, 여기서는 나머지 12개를 12개 원칙으로 정리했다. 뿌리는 다섯 갈래다. 실시간 구독의 순서와 신호, 빈칸과 폴백, 화면이 하는 말, 프론트와 백엔드의 계약, 실브라우저에서만 보이는 것이다.

이 장의 원칙

1. 실시간 목록은 구독을 먼저 붙이고, 적재한 뒤 그 사이 이벤트를 합친다
2. 구독 상태 이름이 아니라 서버가 실제로 받기 시작한 시점을 쟀다
3. 다른 테이블 때문에 「보이게 된」 행은 화면이 직접 당긴다
4. 적재 중·없음·있음을 가르고, 내부 id를 이름 자리에 비추지 않는다
5. 실패를 삼키는 catch를 두지 않고, 빈 화면이면 요청부터 본다
6. 상태 이름은 시스템의 사실이 아니라 사용자에게 일어난 일로 짓는다
7. 뒤로 가기는 브라우저 기록이 아니라 화면의 출처로 정한다
8. 스키마가 두 곳에 있는 필드는 한 커밋에서 바꾸고 왕복으로 시험한다
9. 한 층의 변경이 다른 층의 배포를 끌고 오지 않게 확장 지점을 쓴다
10. 브라우저가 가로채는 입력은 실브라우저에서 확인하고, 콘솔 경고도 결과로 본다
11. 자동화는 「보이는가」가 아니라 「누를 수 있는가」를 기다린다
12. 내 변경과 무관한 환경 오류는 기록하고, 측정 뜻을 안 바꾸는 길로 우회한다

6.1 실시간 구독 — 순서와 신호

이 제품은 DB의 행 변경을 웹소켓으로 화면에 밀어 주는 기능(Realtime)을 쓴다. 화면은 처음에 목록을 한 번 읽어 오고(적재), 그 뒤로는 밀려오는 변경을 받아 고친다. 이 둘 사이에 틈이 있다. 그 틈에 생긴 행은 오류 없이 사라지고, 새로고침하면 돌아온다. 그래서 증상은 짧고 드물며, 「가끔 늦게 뜬다」로만 기억된다. 이 절의 원칙은 모두 「언제부터 받고 있는가」를 정확히 아는 데서 출발한다. 행마다 누가 볼 수 있는지 DB가 정하는 정책(RLS, 행 단위 보안)이 끼면 틈은 하나 더 생긴다.

실시간 목록은 구독을 먼저 붙이고, 적재한 뒤 그 사이 이벤트를 합친다

「읽어 오기 → 화면에 놓기 → 구독」 순서면 읽은 뒤 구독이 붙기 전에 생긴 행은 다음 재적재까지 사라진다. 순서를 뒤집는다. 구독을 먼저 붙이고, 적재하고, 그동안 들어온 이벤트는 버퍼에 모았다가 적재 결과 위에 합친다. 그러면 같은 행이 두 번 들어올 수 있다. 겹침은 같은 키로 덮어쓰는(먹등 upsert) 합치기로 무해하다. 유실은 회복할 수 없다. 둘 중 하나를 고르려면 겹침을 고른다. 「구독 전에 생긴 건 놓친다」를 관찰만 하고 범위 밖으로 미뤄 뒀다면, 그것은 한 화면의 문제가 아니라 공용 계층의 결함일 가능성이 크다.

사례 · 공용 동기화 유틸

무슨 일 모든 컬렉션이 쓰는 공용 동기화 유틸이 「읽어 오기 → 전체 교체 → 구독」 순서였다. 그 사이에 들어온 INSERT는 모든 컬렉션에서 다음 재적재까지 보이지 않았다. 앞서 상담방 쪽에서 「구독 전에 생긴 대화를 놓친다」는 증상을 보고도 관찰만 하고 넘긴 적이 있었다. **왜 몰랐나** 새로고침하면 돌아와서 증상이 오래 남지 않았다. **잡은 방법** 최초 적재 응답을 네트워크 단에서 붙잡아 두고 그 사이 다른 세션이 행을 넣게 해서 경합을 재현했다. 고치기 전에는 유실, 고친 뒤에는 2회 연속 보였다. 처음 짰 재현 시험(「로그인 → 3초 대기 → INSERT」)은 고치기 전 코드에서도 통과했다. 3초 동안 구독이 이미 붙어 창이 닫혔기 때문이다. 경합은 시간을 기다리지 말고 창을 붙잡아 재현하고, 고치기 전 실패부터 본다는 원칙은 → 1.1에서 다룬다.

구독 상태 이름이 아니라 서버가 실제로 받기 시작한 시점을 잔다

라이브러리가 「구독됨(SUBSCRIBED)」을 알려도, 서버가 그 구독으로 변경을 보내기 시작했다는 뜻은 아니다. 「언제부터 받는가」는 상태 이름이 아니라 서버가 보낸 웹소켓 프레임으로 잔다. 서버의 준비 신호를 받은 뒤에 적재한다. 준비가 타임아웃보다 늦게 오면 준비 시점에 한 번 더 적재한다. 첫 구독은 로그인 세션 복원이 끝난 뒤에 하고, 같은 사용자의 토큰 갱신에는 다시 구독하지 않는다. 준비 대기를 걸면 화면이 그만큼 느려지는데, 그 느려짐이 숨어 있던 중복 구독을 드러내기도 한다.

사례 · 준비 대기

무슨 일 페이지를 연 지 몇 초 안에 생긴 상담방이나 방 닫힘이 화면에 오지 않았다. 프레임을 계측해 보니 SUBSCRIBED 뒤에도 서버가 실제 수신 준비를 알리기까지 2.5~7초가 더 걸렸다. 준비 대기를 전면 적용하자 대시보드가 4초 늘었다. 그제야 새 페이지마다 모든 컬렉션이 두 번 구독되고 있었다는 것이 드러났다(채널 27개). **왜 몰랐나** SUBSCRIBED만 기다릴 때는 두 번 붙어도 시간 차이가 보이지 않았다. **잡은 방법** 웹소켓 프레임 계측. 세션 복원 뒤 첫 구독, 같은 사용자 재인증 때 재구독 금지로 중복을 없앴다.

사례 · 채널 묶기

무슨 일이 프레임을 보고 「준비 신호는 채널당 약 250ms」 라고 해석했다. 이를 믿고 15개 컬렉션을 채널 하나로 묶었다. 그런데 관리자 첫 화면은 4,827ms에서 4,392ms로 거의 줄지 않았다(목표 2,500ms). **왜 몰랐나** 그때까지는 컬렉션 하나가 채널 하나여서, 채널 수와 채널 안의 테이블 구독 수(바인딩)가 늘 같이 움직였다. 해석은 관측과 모순되지 않았다. **잡은 방법** 바인딩과 채널의 조합 5종을 따로 켜다. 준비 대기는 채널당이 아니라 바인딩당 약 60~90ms에 연결 바닥 약 1.0초였다. 목표는 못 맞췄지만 뒤집힌 해석은 설계 문서에 정정해 남겼다. 늘 같이 움직이는 두 변수로 원인을 하나로 단정하지 않는다는 원칙은 → 1.3에서 다룬다.

다른 테이블 때문에 「보이게 된」 행은 화면이 직접 당긴다

Realtime은 행이 바뀔 때 이벤트를 보낸다. 행은 그대로인데 다른 테이블의 변화로 그 행을 볼 권한이 새로 생기면, 이벤트는 오지 않는다. 가시성이 다른 테이블로 결정되는 목록은 그 다른 테이블의 변경을 받아, 화면이 참조 id로 해당 행을 직접 읽어 온다. 권한(RLS)을 좁히는 변경을 설계할 때는 「안 보이게 되는 순간」 만이 아니라 「보이게 되는 순간」 시나리오를 따로 그린다. 구독하지 않고 한 번 읽기만 하는 화면은 늦게 온 데이터로 다시 그리지 않는다. 그런 화면에는 적재 완료를 기다리는 문과 구독을 같이 둔다.

사례 · 열람 축소

무슨 일이 세무사가 볼 수 있는 대화를 「자기 일감·신청·상담방에 걸린 것」 으로 좁혔다. 그러자 새 대화는 세무사에게 안 보인 채로 생기고, 뒤에 일감이나 신청이나 방이 생겨 보이게 되어도 이벤트가 오지 않았다. 화면에는 미리보기 「—」, 「대화 데이터를 찾을 수 없습니다」, 이름 대신 계정 id가 떴다. **왜 몰랐나** 대화 행 자체는 바뀌지 않았다. 설계는 「제목이 비는 것」 을 걱정했지만, 실제로 깨지는 것은 운영 흐름이었다. **잡은 방법** 설계 단계에서 시나리오를 따라갔다. 화면이 필요한 대화 id를 직접 당기는 혹을 두었다.

사례 · 검사실 상세

무슨 일이 회귀 1회차에서 관리자 검사실 상세가 한 번 「찾을 수 없습니다」 로 떴다. 재실행하면 정상이라 「무관한 적재 경합」 으로 적었다. 앞의 채널 묶기 뒤 재현이 100%가 됐다. **왜 몰랐나** 그 화면은 대화 목록을 구독하지 않고 한 번 읽기만 했다. 대화 목록이 자기 채널로 먼저 준비돼 경합에서 거의 늘 이겼다. 채널을 묶자 가장 큰 대화 목록이 꼴찌가 되어 경합이 뒤집혔다. **잡은 방법** 같은 자리를 고친 세무사 작업 공간의 전례대로, 적재 완료 문과 구독을 함께 붙였다. 간헐 실패를 잡음으로 적기 전에 그 화면이 무엇을 기다리는지 보라는 원칙은 → 1.3에서 다룬다. 구독이 맺힌 시점의 토큰으로 RLS가 평가되어, 정책 함수의 비로그인 실행 권한을 뺐다가 모든 구독자의 이벤트가 끊긴 일은 → 5장.

6.2 빈칸과 폴백

적재가 느리거나, 요청이 실패하거나, 데이터가 정말 없을 때 화면은 모두 비슷하게 빈다. 비어 있는 화면은 오류처럼 보이지 않는다. 사용자는 「아직 없구나」 로 읽고, 개발자는 컴포넌트를 의심한다. 그 빈자리를 「무언가로라도 채우는」 폴백은 더 그럴듯하게 틀린다. 계정 id가 이름 자리에 앉아도 깨진 화면으로 보이지 않는다.

적재 중·없음·있음을 가르고, 내부 id를 이름 자리에 비추지 않는다

「아직 안 왔다」 와 「다 왔는데 없다」 를 가르려면 적재가 끝났음을 알리는 상태(settled)가 따로 있어야 한다. 그것이 없으면 스피너와 「없음」 을 구분할 수 없다. 이름은 공개 프로필 → 명부 순으로 찾고, 적재 중에는

빈칸으로 두고, 끝났는데 없으면 「세무사」 같은 일반 명칭을 쓴다. 내부 id는 사람 이름 자리의 폴백으로 쓰지 않는다. 적재가 빠를 때는 폴백이 눈에 띌 틈이 없다. 적재 순서나 속도를 바꾸는 변경 뒤에 폴백이 한꺼번에 드러난다.

사례 · 명부 지연

무슨 일이 앞 절의 준비 대기를 걸자 명부 적재가 늦어졌다. 그 틈에 카드, 상담방 머리, 폴 탭에 계정 id(`auditor2`, `viewer`, `clinic`)가 사람 이름처럼 보였다. 같은 때 라이브 대화 감사 화면은 서버에서 그려지는 컴포넌트가 브라우저 전용 스토어를 불러 서버 오류를 냈다. **왜 몰랐나** 적재가 빠를 때는 폴백이 한순간이라 보이지 않았다. **잡은 방법** 준비 대기 적용 뒤 화면을 다시 열어 봤고, E2E(실제 브라우저로 처음부터 끝까지 눌러 보는 자동 시험)에 「id가 한 번도 안 보임」 검사를 넣었다.

실패를 삼키는 catch를 두지 않고, 빈 화면이면 요청부터 본다

`.catch(() => {})` 는 실패를 「데이터 없음」 으로 바꾼다. 에러도 콘솔 경고도 없이 화면만 빈다. 그러면 원인은 컴포넌트에 있는 것처럼 보인다. 실패는 최소한 콘솔에 남기고, 화면에는 「불러오지 못함」 을 「없음」 과 다르게 그린다. 빈 화면을 만나면 컴포넌트를 열기 전에 네트워크 탭에서 요청과 응답을 먼저 본다.

사례 · 대시보드 깔때기

무슨 일이 로컬 프론트를 3111 포트로 띄웠더니 대시보드의 커버리지 깔때기 차트가 나오지 않았다. 프로덕션 백엔드는 다른 출처의 요청을 `localhost:3000` 에서만 허용(CORS)하고 있었고, 그 요청 자리는 빈 catch로 실패를 삼켰다. **왜 몰랐나** 에러도 경고도 없었다. 차트 컴포넌트 버그로 오진하기 쉬웠다. **잡은 방법** 요청의 응답을 보니 400 「Disallowed CORS origin」 이었다. 우회 방법은 → 6.5 마지막 원칙.

6.3 화면이 하는 말

화면의 문구와 색과 버튼은 시스템의 상태를 사용자의 말로 옮긴 것이다. 옮기는 자리에서 뜻이 바뀌면 시스템은 맞게 돌았는데 사용자는 틀린 것을 믿는다. 이 절은 상태 이름과 화면 사이의 이동을 다룬다. 같은 뿌리의 공통 원칙 셋은 1장이 맡는다. 백엔드 값과 비교하는 문자열은 저장값 그대로 쓰고 꾸밈은 나중에 한다(괄호 하나 「(미분류)」 대 「미분류」 때문에 설명과 정렬이 함께 죽었다). 단위가 다른 값은 같은 칸에 있어도 종류로 갈라 합산한다(기여 크레딧과 원화 정산액이 한 잔액에 더해지고 있었다). 알려진 범주의 색은 해시가 아니라 미리 배정한다(18개 라벨 중 1~5° 차이 쌍이 다섯) → 1.4 · 1.5 · 1.7. 「보안과 정합은 DB가 강제하고, 무엇을 그릴지는 화면이 정한다」 는 경계는 → 5장.

상태 이름은 시스템의 사실이 아니라 사용자에게 일어난 일로 짓는다

작업 상태가 `error` 라고 해서 사용자에게 나쁜 일이 일어난 것은 아니다. 보호 장치가 일을 해서 멈춘 것이라면 사용자는 오히려 보호받았다. 그런 중단은 실패의 빨강이 아니라 주의의 호박색으로, 「무엇을 지켰는지」 를 앞세워 그린다. 판단 근거도 함께 보여 준다. 이번 값, 직전 값, 요구 값, 멈춘 사유다. 반대로 통과한 회차라도 빠진 건수가 0이 아니면 숨기지 않는다. 성공·실패 판정 자체를 부산물 지표가 아니라 목적 지표로 하라는 원칙은 → 1.2에서 다룬다(새로 만든 묶음 수가 0이면 「실패」 로 읽던 화면이, 기존 묶음을 재사용한 바람직한 회차를 실패로 그렸다).

사례 · 재구조화 가드

이 제품은 상담 지식을 문장 단위로 다시 써서 지식 사전을 만든다. 이 재구조화는 회차마다 새 세대를 만들고, 분류가 크게 무너진 회차는 가드가 멈춰 기존 세대를 그대로 둔다. **무슨 일이** 가드가 회차를 멈추면 관리 화면에 빨간 「실패」가 떴다. 재구조화 실패를 보면 운영자는 지식 사전이 비었을까 봐 놀란다. 가드는 정확히 그 반대를 한 것이다. **왜 몰랐나** 작업 상태가 error라는 사실만 보면 실패 표시가 자연스러웠다. **잡은 방법** 화면을 설계하며 사용자가 읽을 뜻을 따졌다. 호박색 「중단됨 — 기존 세대를 지켰습니다」에 이번·직전·요구 값과 사유를 붙였다.

뒤로 가기는 브라우저 기록이 아니라 화면의 출처로 정한다

앱 안에서 클릭으로만 들어오면 「브라우저 뒤로 가기」는 늘 맞는 곳으로 간다. 알림 메일 링크나 새로고침으로 들어온 사용자에게는 돌아갈 기록이 없다. 딥링크로 들어올 수 있는 화면의 뒤로 가기는 그 객체가 어디서 생겼는지로 정한다.

사례 · 상담방

무슨 일이 세무사와 사장님의 상담방에는 두 갈래로 들어온다. 사장님의 연결 신청에서 생긴 방과, 세무사의 제안에서 생긴 방이다. 방이 열리면 알림 메일이 가드로 메일 링크로 바로 들어오는 사람이 많다. **왜 몰랐나** 앱 안 클릭 경로만 시험하면 history back은 늘 동작한다. **잡은 방법** 설계에서 진입 경로(메일 링크·새로고침)를 따졌다. 신청에서 생긴 방은 그 신청 상세로, 제안에서 생긴 방은 제안 목록으로 돌려보낸다.

6.4 프론트와 백엔드의 계약

프론트와 백엔드는 따로 배포되고, 응답 모양을 각자 적어 둔다. 이 제품의 프론트는 응답을 스키마 검증 라이브러리(Zod)로 거른다. 기본 설정의 검증기는 모르는 키를 오류 없이 지운다. 그래서 백엔드가 필드를 더해도 화면에는 오지 않고, 아무것도 깨지지 않는다. 요청에 새 필드가 생겨도 옛 백엔드는 그것을 무시하고 다른 뜻으로 돈다. 배포 순서 원칙(새 요청 모양이면 백엔드 먼저)은 → 1.6에서, 인증 필수화처럼 서버가 옛 클라이언트와 새 클라이언트를 함께 받는 중간 단계는 → 5장에서 다룬다.

스키마가 두 곳에 있는 필드는 한 커밋에서 바꾸고 왕복으로 시험한다

백엔드 응답 스키마와 프론트 검증 스키마가 각자 있으면, 새 필드는 두 곳을 같은 커밋에서 바꾼다. 「응답에 필드가 있다」로 끝내지 않는다. 서버 응답 → 화면 스토어 저장 → 다음 요청까지 왕복 끝에서 필드가 살아 있는지 본다. 응답의 어느 부분이 스토어에 들어가고 어느 부분이 버려지는지도 함께 확인한다. 새 응답 모양이면 그 모양을 아는 프론트를 먼저 배포한다. 새 요청 모양일 때와 순서가 반대다.

사례 · 되묻기 상태

무슨 일이 질문을 받아 사실을 뽑고 부족하면 되묻는 앞단 모델을 붙이면서, 「지금 무엇을 되묻는 중인가」를 다음 턴까지 들고 가야 했다. 사용자가 누른 선택 버튼도 요청에 실어야 했다. 백엔드에만 필드를 더하면 프론트 검증기가 응답에서 지운다. 응답의 메타 영역은 화면 스토어에 아예 들어가지 않는다. **왜 몰랐나** 오류 없이 필드만 사라진다. **잡은 방법** 설계 단계에서 검증기 동작과 스토어에 들어가는 범위를 확인했다. 되묻기 상태는 메시지의 필드로, 버튼은 요청의 필드로 정하고 양쪽 스키마를 한 커밋에서 바꿨다. 브라우저 시험에서 2번째 요청의 대화 기록에 되묻기 상태가 남아 있는 것을 봤다.

한 층의 변경이 다른 층의 배포를 끌고 오지 않게 확장 지점을 쓴다

백엔드의 변경이 화면 배포를 함께 요구하면 두 배포가 어긋나는 창이 생긴다. 피할 수 있으면 피한다. 기존 단계 이름, 기존 메타 필드처럼 이미 있는 확장 지점 안에서 바꾼다. 응답 스키마 중 백엔드가 자유롭게 늘릴 자리는 모르는 필드를 지우지 않게(passthrough) 열어 둔다. 그러면 백엔드가 필드를 먼저 늘리고 화면은 나중에 읽어도 된다. 어느 스키마가 지우고 어느 스키마가 통과시키는지 알고 써야 한다. 앞 원칙의 메시지 스키마는 지웠고, 이 원칙의 작업 메타는 통과시켰다.

사례 · 지식 사전 정리 단계

무슨 일이 지식 사전을 정리하는 처리를 새로 더하면서, 관리 화면에 새 단계를 만들면 프론트 배포가 딸려 올 판이었다. 응답 메타에는 코퍼스 분포라는 새 필드도 생겼다. **잡은 방법** 새 처리는 기존 단계 이름 안에서 돌렸다. 작업 응답 스키마가 이미 passthrough라 새 필드는 지워지지 않았다. 프론트는 한 줄도 바꾸지 않았다.

6.5 실브라우저에서만 보이는 것

타입 검사와 빌드는 통과하고, 기능은 동작하는데, 실제 브라우저에서만 어긋나는 것들이 있다. 이벤트 리스너의 기본 옵션, 손가락의 미세한 떨림, 서버가 그린 HTML에 React가 이벤트를 붙여 살아나는 과정(하이드레이션), 주소창만 바꾸는 클라이언트 라우팅이다. 이 프로젝트는 브라우저 자동화 도구(Playwright)로 실브라우저 조작까지 검증했다. 그 도구 자체도 「보이는 것」과 「되는 것」을 혼동하게 만든다. 브라우저 E2E는 지금 보는 화면이 새 빌드인지부터 확인하고, 부분 실패는 화면이 아니라 DB 상태로 판정한다는 원칙은 → 1.1에서 다룬다.

브라우저가 가로채는 입력은 실브라우저에서 확인하고, 콘솔 경고도 결과로 본다

휠·터치·길게 누르기처럼 브라우저가 기본 동작을 가진 입력은 코드만 봐서는 실제 동작을 알 수 없다. 스크롤을 막아야 하는 휠·터치 핸들러는 React의 `onWheel`이 아니라 `{ passive: false }`로 직접 붙인 네이티브 리스너로 둔다. React는 휠 리스너를 `passive`로 등록해 `preventDefault()`를 무시한다. 기능이 동작해도 콘솔 경고가 남았다면 그것도 스모크 결과로 읽는다. 아무 알림 없이 조용히 취소되는 제스처는 단순 클릭으로 바꾼다.

사례 · 지식 그래프 휠 줌

무슨 일이 세무사가 지식 문서들의 연결을 보는 그래프 화면에서 휠 줌은 됐지만 페이지 스크롤이 같이 움직였다. **왜 몰랐나** 기능(줌)은 동작했고, 콘솔 경고로만 드러났다. **잡은 방법** 브라우저 스모크의 콘솔을 읽었다. 이펙트 안에서 `passive`가 아닌 네이티브 리스너를 붙이고, 최신 화면 좌표는 `ref`로 참조했다.

사례 · 길게 누르기

무슨 일이 한 항목을 길게 눌러 메뉴를 여는 동작이 있었다. 살짝만 움직여도 포인터가 요소를 벗어난 것으로 처리되어 타이머가 조용히 취소됐다. 실사용에서 두 번 헛갈린다는 피드백을 받았다. **왜 몰랐나** 제스처 취소에는 아무 알림이 없다. 코드상으로는 맞게 짜여 있었다. **잡은 방법** 롱프레스를 클릭으로 바꿨다.

자동화는 「보이는가」가 아니라 「누를 수 있는가」를 기다린다

하이드레이션 전에 입력창에 값을 채우면 값은 화면에 보이지만 React 상태는 비어 있다. 제출 버튼은 비활성으로 남는다. 클라이언트 라우팅은 페이지 load 이벤트를 다시 내지 않으므로, load를 기다리는 URL 대기는 풀리지

않는다. 자동화 스크립트는 버튼이 활성화될 때까지, 주소가 실제로 바뀔 때까지처럼 「상호작용 가능한 상태」를 명시적으로 기다린다. 사용자처럼 완료를 기다리게 짜야 스크립트 문제가 앱 버그로 둔갑하지 않는다.

사례 · 첫 실브라우저 검증

무슨 일이 처음으로 실브라우저 조작까지 검증했다(22건 통과, 이어 실사용 흐름 6고리). 하이드레이션 전에 채운 입력은 보였지만 제출 버튼이 disabled로 남았다. 화면 전환은 클라이언트 라우팅이라 URL 대기가 끝나지 않았다.

왜 몰랐나 입력값이 화면에 보이니 정상처럼 보였다. **잡은 방법** 버튼 활성을 기다린 뒤 진행하고, 주소 대기는 브라우저 안에서 `location.pathname` 을 직접 확인하는 대기로 바꿨다.

내 변경과 무관한 환경 오류는 기록하고, 측정 뜻을 안 바꾸는 길로 우회한다

오류가 런타임이나 OS 층에서 나고 방금 바꾼 코드와 무관하면, 그 원인을 끝까지 쫓느라 검증을 멈추지 않는다. 오류 코드로 층을 가린 뒤 기록하고 다른 실행 경로로 간다. 우회는 측정하려던 것의 의미를 바꾸지 않는 쪽으로 고른다. 남의 서버를 끄거나 프로덕션 설정을 고치는 것은 우회가 아니라 또 다른 변경이다.

사례 · 개발 서버

무슨 일이 로컬 개발 서버가 CSS 워커의 DLL 초기화 실패(`0xc0000142`)로 모든 요청에 500을 냈다. **왜 몰랐나** 매 요청 500이라 방금 바꾼 코드 탓으로 보이기 쉬웠다. **잡은 방법** 오류 코드로 환경 문제임을 가리고, 프로덕션 빌드를 로컬에서 띄우는 경로로 우회해 스모크를 마쳤다.

사례 · 포트와 CORS

무슨 일이 6.2의 빈 깔때기 차트는 프로덕션 백엔드가 3000 포트만 허용해서였다. 그런데 3000은 다른 프로젝트가 쓰고 있었다. **잡은 방법** 남의 서버를 죽이지도, 프로덕션 CORS를 고치지도 않았다. 로컬 프록시를 세워 화면이 같은 경로로 같은 백엔드를 부르게 했다. 측정 대상인 화면 경로의 의미는 그대로였다.

작업 전후 점검표

- ☐ 실시간 목록이 구독을 먼저 붙이고 적재하는가? 그 사이 들어온 이벤트를 버퍼에서 합치고, 합치기가 멎등인가?
- ☐ 경합을 고쳤다면 응답을 붙잡아 창을 강제로 연 시험이 고치기 전 코드에서 실패하는 것을 봤는가?
- ☐ 「구독됨」 신호가 아니라 서버가 실제로 받기 시작한 시점을 계측했는가? 같은 컬렉션이 두 번 구독되고 있지 않은가?
- ☐ 성능의 원인을 해석했다면, 같이 움직이던 변수(채널 수와 바인딩 수 등)를 하나씩 고정해 따로 켜는가?
- ☐ 이 목록의 가시성이 다른 테이블로 정해지는가? 그렇다면 「보이게 되는 순간」에 화면이 행을 직접 당기는가?
- ☐ 구독하지 않고 한 번 읽기만 하는 화면이 적재 완료를 기다리는가? 늦게 온 데이터로 다시 그려지는가?
- ☐ 화면이 적재 중·없음·불러오지 못함을 서로 다르게 그리는가? 적재가 끝났음을 알리는 상태가 있는가?
- ☐ 사람 이름 자리에 내부 id가 폴백으로 비칠 수 있는 곳이 있는가? 적재를 느리게 만들어 확인했는가?
- ☐ 코드에 빈 catch가 있는가? 빈 화면을 만났을 때 컴포넌트보다 네트워크 응답을 먼저 봤는가?
- ☐ 「실패」로 그리는 상태 중 보호 장치가 일을 해서 멈춘 경우가 섞여 있지 않은가? 지킨 것과 근거 수치를 함께 보여 주는가?
- ☐ 메일 링크나 새로고침으로 바로 들어와도 뒤로 가기가 갈 곳이 있는가?
- ☐ 백엔드에 더한 필드가 프론트 검증 스키마에도 있는가? 서버 → 스토어 → 다음 요청 왕복 끝에서 살아남는가? 배포 순서는 요청 모양인지 응답 모양인지에 맞췄는가?
- ☐ 이 변경이 다른 층의 배포를 끌고 오는가? 기존 확장 지점이나 passthrough 자리로 피할 수 있는가?
- ☐ 휠·터치·길게 누르기를 실브라우저에서 눌러 봤는가? 스모크의 콘솔 경고까지 읽었는가?
- ☐ 자동화 스크립트가 「보이는가」가 아니라 버튼 활성화·주소 변경 같은 「누를 수 있는가」를 기다리는가? 첫 단계에서 새 빌드인지 확인하는가?

7 장

리서치

제품을 재료로 실험을 설계하고, 채점하고, 논문으로 내보내는 사람

이 장은 제품에서 나온 데이터로 실험을 짜고, LLM이나 사람에게 채점을 맡기고, 그 결과를 논문으로 쓰고 투고하는 사람을 위한 것이다. 사례는 한 AI 세무 상담 챗봇에서 나왔다. 사장님이 세무 질문을 하면 LLM이 근거 지식을 찾아 답하고, 세무사가 그 답을 검수해 다시 지식으로 되돌리는 제품이다. 근거를 찾아 답에 붙이는 방식을 RAG(검색 증강 생성)라고 부른다. 제품은 국내 LLM API 하나로 만들었다. 이 제품을 재료로 논문 두 편이 나왔다. 하나는 국제 학회에 채택돼 카메라레디(인쇄용 최종 원고)까지 냈고, 하나는 국내 학회에 투고했다. 리서치 자리의 실수는 결과표 안에서 그럴듯하게 틀린다. 채점기는 숫자를 내고, 그 숫자는 소수점까지 정확해 보인다. 데이터에는 계정 이름이 붙어 있고, 그 이름은 누가 만들었는지를 말해 주는 것처럼 보인다. 학회 메일은 공식 문서처럼 보인다. 어느 것도 오류를 내지 않는다. 틀린 것이 드러나는 때는 대개 원고가 나간 뒤다. 그래서 이 장은 매 절에서 같은 질문을 던진다. **이 숫자가, 이 출처가, 이 절차가 그럴듯하게 틀렸다면 누가 언제 알아챌까?** 첫 태그가 리서치인 교훈은 18개다. 그중 11개는 모든 직무에 걸쳐 1장이 주인이라, 여기서는 리서치 관점에서 꼭 필요한 대목만 짧게 다시 짚고 교차 참조한다. 남은 7개에 다른 장 교훈 몇 개를 리서치 쪽에서 비춰 보태 10개 원칙으로 묶었다. 갈래는 여섯이다. 데이터를 누가 만들었나, 채점기와 판독기를 믿어도 되나, 사람 대조 표본을 어떻게 뽑고 보내나, 벤치가 무엇을 재나, 투고 절차가 무엇을 되돌릴 수 없게 하나, 연구 세션은 어떻게 운영하나다.

이 장의 원칙

1. 원고에는 행위 주체를 실제로 쓴다 — 자동화가 한 일을 사람에게 돌리지 않는다
2. 채점 분포가 평평하면 채점기를 의심하고 원문부터 연다
3. 「계산으로 설명되면 허용」 하는 판독은 계산 재료까지 출처로 묶는다
4. 사람 대조 표본은 검증자가 판정할 수 있는 범위에서만 뽑는다
5. 검증 요청에는 인용이 쓰인 문장을 붙이고, 대응은 식별자 키로 한다
6. 층화 표본에서는 재현율· κ 를 내지 않고 층별 비율만 쓴다
7. 평가 질문은 평가 대상 코퍼스를 보지 않고 만든다
8. 되돌릴 수 없는 제출 단계는 그 입력이 확정된 뒤에 밟는다
9. 마감 직전의 새 실험은 한계로 적고 다음 지면으로 넘긴다
10. 연구 세션은 제품 세션과 나누고, 끝날 때 기록을 남긴다

7.1 데이터를 누가 만들었나

이 제품의 검수실에는 세무사용 계정 이름으로 들어간 코멘트와 5점 평가가 많다. 그런데 그중 상당수는 사람이 쓴 것이 아니다. 개발 중에 「감사 모드」로 돌린 LLM이 평가자 계정 이름으로 남긴 산출물이다. 사람(세무사)이 실제로 개입한 자리는 승인 게이트와, 인용·원문 대조 두 곳뿐이다. 계정 이름과 화면 용도는 생산 주체를 말해 주지 않는다. 이 출처를 팀은 두 번 오독했다. 리서치 원고에서 이 오독은 가장 그럴듯한 과장이 된다.

원고에는 행위 주체를 실제대로 쓴다 — 자동화가 한 일을 사람에게 돌리지 않는다

누가 그 일을 했는지는 사실대로 쓴다. 「세무사가 편집했다」「전문가가 캘리브레이션했다」는 문장은 「편집 연산이 적용됐다」「표본의 정오를 확인했다」보다 늘 설득력 있어 보인다. 그래서 퇴고 때 가장 먼저 부풀어 오른다. 자동화가 한 일을 사람에게 돌리지 않고, 사람이 한 확인을 더 큰 개입으로 부풀리지 않는다. 데이터의 생산 주체는 계정 이름이나 화면 용도로 추정하지 말고 생성 경로로 적는다. 정정 문장도 같은 기준으로 한 번 더 확인한다. 원고에 들어갈 필요가 없는 내부 정보도 이 단계에서 뺀다.

사례 · 편집 조항 세 건

무슨 일이 국내 학회 원고에 법 조항을 편집하는 실험이 있었다. 편집이 적용된 조항은 세 건이었다. 초고 단계에서 이것을 「세무사가 편집한 조항」으로 쓸 수 있었다. **왜 몰랐나** 그 표현이 실험의 현장성을 더 잘 보여 주는 것처럼 읽혔다. 문장 자체는 오류를 내지 않는다. **잡은 방법** 원고 1·2판 검토에서 대표의 코멘트와 구현 판단으로 막았다. 「편집 연산이 적용된 조항」으로 썼다. 「실패 요인의 분리」 같은 내부 정보 절도 원고에서 뺐다.

사례 · 두 번 오독된 검수 대기열

무슨 일이 관리자 화면에 7월 중순의 미확정 검수 항목이 쌓여 있었다. 대표는 이것을 첫 논문의 실험 데이터로 기억했다. 실제로는 제품 개발 작업(100건 일괄 감사, RAG 재검증)이 남긴 대기열이었다. 논문이 DB에 남긴 흔적은 지식 문단 168건의 상태 변경 한 번뿐이었고, 그것도 이미 복구돼 있었다. 첫 오독을 바로잡으며 쓴 정정문은 다시 「세무사들이 실작업으로 남긴 검수 대기열」이라고 적었다. 두 번째 오독이다. 그 코멘트는 감사 모드 LLM의 산출물이었다. **왜 몰랐나** 첫 오독은 정황이 받쳤다. 같은 주에 RAG를 켜고 끈 비교 리포트가 연달아 나왔고, 그 리포트들이 논문 실험 기록으로 묶여 있었다. 두 번째 오독은 정정이라는 형식이 받쳤다. 정정은 이미 검증을 거친 문장처럼 느껴져 다시 의심받지 않는다. 계정 이름도 「auditor」였다. **잡은 방법** 이전 세션이 같은 질문을 조사해 남긴 문서를 찾아 첫 오독을 풀었다. 두 번째는 주간 기록을 다시 쓰면서 감사 데이터 출처 규칙에 비춰 고쳤다. 같은 표기 규칙을 논문 인용에도 적용한다. → 1.5

내부 문서의 줄임말이 외부 자료로 새는 자리도 같다. 세무사에게 보낼 확인 자료에 「세무사 코멘트」라는 내부 표기가 그대로 들어갈 뻔했다. 외부본에는 「검수 코멘트(AI 평가 도구 작성, 참고용)」로 고쳐 썼다. → 2장

7.2 채점기와 판독기를 믿어도 되나

이 제품의 실험은 대부분 LLM에게 채점을 맡겼다. 「이 근거 문단이 이 질문에 쓸모 있는가」를 0~2등급으로 매기는 식이다. 이렇게 LLM을 채점자로 쓰는 방식을 LLM-as-judge라고 한다. 답변 속 수치 날조나 판정형 문장은 정규식과 산술 규칙으로 기계 판독했다. 두 도구 모두 숫자 하나를 낸다. 숫자는 근거 문장을 보지 않게 만든다. 채점기의 절대치가 회차마다 몇 퍼센트포인트씩 움직인다는 점, 정규식이 첫 회차에 과대 집계한다는 점, 스캔 수치는 하한이라는 점은 모든 직무에 걸쳐 1장에서 다룬다 → 1.2. 여기서는 채점기가 관대한 방향과 판독기의 허용 규칙이 새는 자리를 본다.

채점 분포가 평평하면 채점기를 의심하고 원문부터 연다

기본 채점기는 일반론에 관대하다. 원칙형 문장으로 된 코퍼스는 어느 질문에나 그럴듯하게 붙고, 채점기는 「관련 있어 보이는」 문단에 점수를 준다. 검색 점수가 높은 문단이든 낮은 문단이든 같은 비율로 좋은 등급을 받는다면, 채점기가 아무것도 가르지 못하고 있다는 뜻이다. 점수와 등급의 관계가 평평하면 원문을 열어 본다. 「질문의 바로 그 지출·제도를 직접 다룬다」 처럼 좋은 등급의 조건을 좁힌 엄격 채점을 따로 만든다. 엄격 채점으로도 완료 조건을 못 넘으면 「무해」 까지만 주장한다.

사례 · 27개짜리 용어 사전

검색은 세 층으로 쪼갰다. 어느 질문에나 늘 붙는 규범 문단(L0), 세무 용어 사전(L1), 문장 단위로 다시 쓴 지식 사전(L2)이다. 여러 층의 검색 순위를 하나로 합치는 데는 RRF(순위 역수 융합)를 썼다. **무슨 일이** 청크 27개짜리 용어 사전을 단독으로 돌렸더니, 기존 지식과 거리가 먼 질문에서 「쌀모 있음」이 60.8%로 나왔다. 사전을 더한 융합은 68.2%에서 79.1%로 올랐다. **왜 몰랐나** 숫자가 그럴듯했고 방향도 기대와 같았다. 다만 검색 점수 구간과 상관없이 2등급이 약 40%로 평평했다. **잡은 방법** 원문을 열었다. 「의료장비 업그레이드는 수선비인가」라는 질문에 차량유지비를 다룬 청크가 2점을 받고 있었다. 2등급 조건을 좁힌 엄격 채점을 만들자 비로소 검색 점수에 따른 기울기가 생겼다. 사전 층은 유사도 컷 0.45로 넣었지만, 결론은 「이득 미입증·무해」로 적었다.

같은 벤치에서 채점 절대치가 회차마다 갈래 전반 4~6pp씩 움직였다. 비교 대상은 모두 같은 회차에서 채점하고 순위만 믿는다 → 1.2.

「계산으로 설명되면 허용」하는 판독은 계산 재료까지 출처로 묶는다

LLM 답변의 수치 날조를 판독할 때, 출처에 그대로 없는 숫자라도 출처의 숫자로 계산해 나올 수 있으면 「파생」으로 허용하고 싶어진다. 이 허용은 넓게 잡으면 우연을 통과시킨다. 아무 두 수의 사칙연산을 허용하면 거의 모든 숫자가 설명된다. 계산 재료(피연산자)를 신뢰할 수 있는 출처로 엄격히 제한하고, 숫자의 종류(금액·인원·비율)도 맞춘다. 그래도 우연히 맞는 틀린 산술은 남는다. 그 한계를 측정 결과와 함께 적는다.

사례 · 20명 = 5 + 15

무슨 일이 날조 판독 1차판은 답변 속 「20명」을 근거에 있는 5와 15의 합으로 설명하고 통과시켰다. 2차판은 다른 사안 근거에 있던 「1인당 10만원」을 재료로 삼아, 틀린 「20명」을 $2,000,000 \div 100,000$ 으로 설명하고 또 통과시켰다. **왜 몰랐나** 산술로 설명되는 숫자는 맞는 추론처럼 보인다. 판독기는 통과 건수만 셸다. **잡은 방법** 통과한 사례를 직접 읽었다. 피연산자를 사용자가 말한 수와 결정 엔진이 낸 수로만 한정하고, 종류 제한을 넣었다. 판독 규칙이 제품의 날조 가드와 같아야 하는 경우, 제품을 바꾸면 측정도 같이 고친다 → 1.6.

품질 지표를 어디서 세느냐도 같은 함정이다. 중복 배정을 후처리가 끝난 결과(사전)에서 세면 언제나 0이 나온다. 사전이 이미 중복을 지웠기 때문이다. 누락·중복·환각은 후처리 전 원응답에서 센다 → 3장.

7.3 사람 대조 표본을 어떻게 뽑고 보내나

국내 학회 논문의 핵심 수치는 「LLM이 낸 답변의 법령 인용 중 가짜 근거가 몇 개인가」였다. 판정은 개발 도구로 쓰던 다른 LLM의 맹검 채점이 맡았다. 맹검이란 채점자가 어느 조건의 답변인지 모르게 하는 것이다. 그 채점을 믿을 수 있는지 보려고 시드로 30건을 미리 뽑아 봉인하고, 세무사에게 판정을 받은 뒤 개봉해 맞췄다. LLM이 가짜로 본 16건은 16/16 결함이었다. 정상으로 본 7건 중 4건이 결함이었다. 과대 판정은 없었고 누락은 있었다. 그래서 LLM

채점 수치는 하한으로 썼다. 이 원칙 자체는 모든 직무에 걸쳐 1장이 다룬다 → 1.2. 이 절은 그 대조가 성립하도록 표본을 뽑고, 보내고, 집계하는 쪽을 다룬다. 사람의 역할은 끝까지 「표본의 정오 확인」으로만 쓴다.

사람 대조 표본은 검증자가 판정할 수 있는 범위에서만 뽑는다

사람 대조는 한 건 한 건이 비싸다. 검증자가 판정할 수 없는 건이 섞이면 그만큼 표본이 줄고, 회신을 받은 뒤에야 줄었다는 것을 안다. 추출 조건에 검증자의 전문 범위를 처음부터 넣는다. 범위 밖 건을 회신 뒤에 빼야 하면, 그 제외 규칙은 사후에 정한 것이므로 민감도를 함께 낸다.

사례 · 검증대상 X 여섯 건

무슨 일이 봉인 표본 30건 중 6건이 세무사에게서 「검증대상 X」로 돌아왔다. 의료법·근로기준법·특정금융정보법·주택임대차보호법 인용이었다. **왜 몰랐나** 추출 단계에서는 「법령 인용」이면 모두 같은 모집단으로 봤다. 세무사가 세법 밖 인용을 판정하지 않는다는 조건이 추출 규칙에 없었다. **잡은 방법** 회신을 보고 알았다. 회신을 보고 정한 제외 규칙에는 민감도(17/17, 15/16)를 함께 붙였다. 다음 표본부터는 추출할 때 세법 인용으로 한정한다.

검증 요청에는 인용이 쓰인 문장을 붙이고, 대응은 식별자 키로 한다

법령 번호만 보내면 검증자는 「그 조문이 실재하는가」만 확인할 수 있다. 가장 흔한 결함은 번호는 실재하는데 그 조문이 답변이 말한 내용이 아닌 경우다. 이것은 인용이 쓰인 문장을 함께 봐야 잡힌다. 회신과 봉인 키를 맞출 때도 문자열로 맞추면 「제○조의2」와 「제○조의 2」 같은 표기 차이에 깨진다. 법령명·조·의, 연도·부호·번호처럼 정규화한 식별자 키로 맞춘다. 집계 전에는 봉인 키와 대조표의 인용 문자열이 일치하는지 단언(assert)으로 먼저 확인한다.

사례 · 결함의 85%

무슨 일이 첫 논문 때 쓴 대조 형식에 「인용 맥락(답변 발췌)」 열을 더했다. 판정 대응도 문자열 대신 식별자 키로 바꿨다. 개봉해 보니 결함 20건 중 85%가 「번호는 실재, 표제·내용 불일치」였다. **왜 몰랐나** 번호만 있는 형식으로도 회신은 돌아온다. 돌아온 판정이 「실재함」이면 맞는 인용처럼 보인다. 맥락 없이는 가장 흔한 결함이 대조 단계에서 통째로 빠졌을 것이다. **잡은 방법** 대조를 설계하는 단계에서 고쳤다. 맥락 열이 없었다면 위 16/16과 4/7도 다른 숫자가 됐다.

성격이 다른 검토 요청은 한 표에 섞지 않는다. 같은 주에 제품용 세무사 검토 요청 31문장이 나갔다. 이것을 논문 맹검 표본 30건과 대조해 같은 문장이 없음을 확인했다(바이그램 유사도 최대 0.30). → 2장

총화 표본에서는 재현율·κ를 내지 않고 총별 비율만 쓴다

표본을 총마다 정해진 수로 뽑으면(총화 추출), 그 표본의 구성비는 모집단의 구성비가 아니다. 재현율이나 κ(우연 일치를 뺀 일치도)는 계산은 되지만 값이 표본 비율에 따라 달라진다. 숫자가 나온다고 쓸 수 있는 것은 아니다. 총화 표본에서는 모집단 비율에 기대는 지표를 내지 않는다. 「LLM이 가짜로 본 총에서 몇 건이 결함이었나」처럼 총별 비율만 쓴다.

사례 · 가짜 20 : 정상 10

무슨 일이 봉인 표본은 LLM 판정 기준으로 가짜 20건, 정상 10건으로 총화돼 있었다. 실제 답변 전체에서 가짜 판정은 이보다 훨씬 드물다. **왜 몰랐나** 대조표가 있으면 재현율과 κ는 스프레드시트 한 줄로 나온다. 리뷰어에게 익숙한 지표라 넣고 싶어진다. **잡은 방법** 집계하면서 판단했다. 원고에는 16/16, 4/7 같은 총별 비율만 썼다.

7.4 벤치는 무엇을 재고 있나

벤치 점수가 오르는 길은 둘이다. 시스템이 좋아지거나, 문항이 시스템 쪽으로 기울거나. 두 번째는 점수표에서 첫 번째와 구별되지 않는다. 측정 질문이 바꾼 코드를 실제로 지나는지, 작은 표본 차이를 처방으로 만들기 전에 분해해 다시 썼는지, 표본 0을 0%로 읽지 않았는지는 1장에서 다룬다 → 1.1 · 1.2. 여기서는 문항과 데이터가 정답 쪽으로 새는 자리를 리서치 관점에서 본다.

평가 질문은 평가 대상 코퍼스를 보지 않고 만든다

코퍼스 문서를 보고 만든 질문은 그 문서의 어휘를 물려받는다. 그런 질문은 검색이 잘 되는 질문이 되고, 점수는 오르지만 실제 사용자를 대변하지 못한다. 평가 질문은 사용자 분야에서 출발해 만든다. 비교 기준선의 설정(검색 컷 같은 값)은 프로덕션과 같게 맞춘다. 파생 데이터로 평가할 때는 표면 지름길도 막는다. 길이만 보고 찍는 오라클을 비교선으로 두고, 같은 원 사례에서 나온 형제 문항은 평가·예시·검색에 갈라 쓰지 않는다. 맹검 채점에는 라벨을 드러내지 않는 불투명 id를 쓴다.

사례 · 시민 질문 48문항

무슨 일이 국세청 심판례·질의회신으로 만든 새 지식 층을 재려고 시민 질문 48문항을 새로 뒀다. 그 지식 문서를 열어 보지 않고, 설계 단계에서 나눈 분야별로 만들었다. **왜 몰랐나** 문서를 보고 만들었다면 점수가 오른 이유가 시스템인지 문항인지 가를 수 없었다. 그럴듯한 개선으로 보였을 것이다. **잡은 방법** 작성 원칙으로 미리 막았다. 벤치의 다른 갈래 설정(사전 층 컷 0.45)도 프로덕션과 같게 맞췄다. → 4장

사례 · 길이·형제·id

무슨 일이 앞단 모델(질문에서 사실을 뽑고 되묻는 모델)의 「충분한 질문인가」 판정을 재는 데이터에 누출 경로가 셋 있었다. 충분한 질문은 중앙값 230자로 원본 111자보다 길었다. 같은 사례의 형제 문항 셋이 정규화된 질문을 공유했다. 원 id의 접미사가 라벨을 드러냈다. **왜 몰랐나** 셋 다 모델이 「잘 맞히는」 모습으로 나타난다. **잡은 방법** 데이터를 파생하는 단계에서 길이 통계를 보고, 맹검 설계 때 id를 점검했다. → 3장

7.5 투고 절차는 무엇을 되돌릴 수 없게 하나

논문 원고는 몇 번이고 고칠 수 있다. 제출 시스템의 단계는 그렇지 않다. 서명과 검증은 한 번 지나면 되돌리기 어렵고, 마감은 재수정 기회를 주지 않는다. 이 자리의 틀림은 학회가 보낸 메일, 독립된 화면처럼 보이는 단계, 「조금만 더 하면 넣을 수 있는」 실험의 모습으로 온다. 마감 직전에 공식 페이지를 한 번 더 여는 습관은 1장이 주인이다 → 1.4. 이 절은 그 습관이 구한 첫 논문의 카메라레디를 따라간다.

되돌릴 수 없는 제출 단계는 그 입력이 확정된 뒤에 밟는다

제출 시스템의 단계들은 각각 독립된 화면이라 순서가 있다는 것이 보이지 않는다. 먼저 단계 사이의 의존을 그려 본다. 서명·검증처럼 되돌릴 수 없는 단계는 그 입력(저자명, 원고)이 확정된 뒤에 한다. 이미 검증받은 산출물은 건드리지 않고, 고칠 것이 있으면 메타데이터 쪽을 고친다. 급해 보이는 항목은 자격부터 확인한다.

사례 · 다섯 단계 카메라레디

무슨 일이 카메라레디는 저자 등록 → 저자명 정정 → PDF 검증 → 저작권 서명 → 최종 업로드 순서가 강제됐다. 논문 화면의 저자 편집으로는 다른 저자의 이름을 바꿀 수 없어, 저자 네 명이 각자 프로필을 고쳐야 했다. **왜 몰랐나** 서명 화면은 정정 화면과 따로 열린다. 틀린 이름으로 서명해도 아무 경고가 없다. 그대로 두면 프로시딩에서 빠진다. **잡은 방법** 서명을 정정 뒤로 미뤘다. 원고를 고치면 PDF 검증을 다시 받아야 해서, 저자 표기는 원고가 아니라 제출 시스템 쪽을 고쳤다. 급해 보이던 지원 항목은 자격부터 봐서 대상이 아님을 확인했다. 다섯 단계를 마감 하루 전에 끝냈다.

마감 직전의 새 실험은 한계로 적고 다음 지면으로 넘긴다

리뷰어가 짙은 빈자리를 메우는 실험은 언제나 넣고 싶다. 그러나 마감 직전의 새 실험은 불리한 결과가 나와도 되돌릴 시간이 없다. 그 실험은 본문에서 한계로 명시하고 후속 설계를 적는 선에서 처리한 뒤 다음 지면으로 넘긴다. 같은 시기에 지면 제한이나 마감 같은 형식 조건은 공식 원천에서 다시 확인한다. 메일보다 공식 사이트가 상위다. 마감이 두 개로 적혀 있으면 이른 쪽을 따른다.

사례 · 6페이지와 4페이지

무슨 일이 첫 논문은 리뷰 점수 4·4·5로 채택됐다. 채택 메일은 지면을 6페이지라고 했고, 팀은 그 메일을 근거로 6페이지판을 주력으로 정했다. 제출 전날 학회 공식 제출 안내 페이지를 다시 열자 4페이지였다. 6페이지판 주력을 뒤집고 4페이지판을 냈다. 저작권 서명 마감도 사이트는 9/10, 메일은 9/17로 두 개였다. **왜 몰랐나** 메일은 학회가 보낸 명문 근거라 믿을 만해 보였다. 메일 본문은 여러 학회가 돌려 쓰는 템플릿이라 틀릴 수 있다. **잡은 방법** 제출 직전에 공식 페이지를 다시 열었다. 비용은 5분이었다. 안 열었다면 비용은 게재 실패였다. 리뷰어 한 명이 짙은 「권위 있는 원문 코퍼스(판례·심판례·유권해석 1,301건)를 쓴 RAG 베이스라인」 실험은 카메라레디에 넣지 않았다. 본문에 한계로 적고 저널 확장의 1순위로 넘겼다. → 1.4

7.6 연구 세션을 어떻게 운영하나

이 제품은 AI 코딩 도구와 세션 단위로 함께 만들었다. 제품 코드와 논문 실험이 같은 저장소에 있다. 연구 쪽에는 제품에 없는 예외가 있다. 오프라인 채점에는 외부 모델을 써도 된다는 것이다. 이 대회 트랙은 제품 구동 경로에서 외산 모델 호출을 금한다. 그런데 오프라인 채점은 제품 구동 경로 밖이라 트랙 조건과 무관하다. 이 예외가 제품 코드로 번지면 아무 오류 없이 트랙 조건이 깨진다. 연구 작업은 커밋 흐름 밖에 있어 기록이 빠져도 아무것도 깨지지 않는다. 두 원칙 모두 모든 직무에 걸쳐 1장이 주인이고, 여기서는 연구 쪽 사정만 묶어 둔다.

연구 세션은 제품 세션과 나누고, 끝날 때 기록을 남긴다

판단 기준은 「LLM을 채점자로 쓰는가」가 아니라 「제품 구동 경로 안의 추론 호출인가」다. 오프라인 예외는 연구 스코프를 명시하고, 제출물이 아니며, 런타임이 그 코드를 불러오지 않을 때만 쓴다. 제품 세션에서는 아예 쓰지 않는다. 외산 호출을 막는 정적 검사가 없다면 방어선이 문서와 사람의 주의력뿐이라는 것도 적어 둔다. 연구 세션이 끝나면 커밋이 없더라도 기록을 남긴다. 원고와 집계 파일이 로컬 전용 폴더에 있으면 저장소에는 아무 흔적도 남지 않는다.

사례 · 한 세션에 들어온 벤치 하네스

무슨 일이 세 방향 비교가 가능해지자 제품을 고치던 세션에서 벤치마크 하네스를 바로 만들자는 제안이 나왔다. **왜 몰랐나** 같은 저장소, 같은 세션이라 연구용 외부 모델 채점 코드가 제품 코드 옆에 자연스럽게 놓인다. 「LLM-as-judge라서 괜찮다」는 식으로 판단하면 기준이 흐려진다. **잡은 방법** 대표가 논문 작업과 제품 수정을 한 세션에 맡기는 위험을 짚었다. 정책 문서에 세션 분리 원칙을 새로 적었다. → 1.7

사례 · 흔적 없는 원고 세션

무슨 일이 국내 학회 원고를 쓴 사흘간의 세션이 기록을 남기지 않았다. 원고와 집계 파일은 로컬 전용 폴더에 있어 커밋에도 흔적이 없었다. **왜 몰랐나** 제품 커밋 흐름 밖의 작업이라 빠져도 깨지는 것이 없었다. **잡은 방법** 2주 뒤 봉인 대조를 개봉할 때 기록이 없어, 세션 로그를 검색해 무엇을 어떻게 봉인했는지 복원했다. 봉인 대조는 봉인한 방법을 정확히 알아야 성립한다. 기록 습관은 제품·연구 세션 구분 없이 같게 적용한다. → 1.7

작업 전후 점검표

- ☐ 원고의 모든 「누가 ~했다」 문장에서, 그 주체가 실제 생성 경로와 맞는가? 자동화가 한 일을 사람에게, 확인을 편집·캘리브레이션으로 돌리지 않았는가?
- ☐ 데이터의 생산 주체를 계정 이름이나 화면 용도로 추정하지 않았는가? 정정 문장도 같은 기준으로 다시 확인했는가?
- ☐ 외부로 나가는 자료에서 출처 표기(사람인가 AI인가)를 따로 점검했는가? 내부 줄임말이 남아 있지 않은가?
- ☐ LLM 채점 결과에서 검색 점수와 등급의 관계를 봤는가? 평평하다면 원문을 열고 엄격 채점으로 다시 짰는가?
- ☐ 비교 대상을 모두 같은 회차에서 채점했는가? 기준선과 함께 그 회차의 코퍼스 상태와 채점기 설정을 적었는가?
- ☐ 자동 판독기가 「계산으로 설명되면 허용」한다면, 계산 재료를 신뢰할 출처로 한정했는가? 통과한 사례를 직접 읽었는가?
- ☐ LLM 채점기를 미리 뽑아 봉인한 사람 대조 표본으로 짰는가? 누락이 있다면 수치를 하한으로 썼는가?
- ☐ 대조 표본을 뽑을 때 검증자가 판정할 수 있는 범위로 한정했는가? 사후에 정한 제외 규칙에 민감도를 붙였는가?
- ☐ 검증 요청에 인용이 쓰인 문장(맥락)을 함께 보냈는가? 회신 대응을 정규화한 식별자 키로 하고, 집계 전에 봉인 키와의 일치를 단언했는가?
- ☐ 층화 표본에서 재현율·κ처럼 모집단 비율에 기대는 지표를 내지 않았는가?
- ☐ 평가 질문을 평가 대상 코퍼스를 보지 않고 만들었는가? 길이·형제 문항·id로 라벨이 새지 않는가? 기준선 설정이 프로덕션과 같은가?
- ☐ 측정 질문이 바꾼 코드를 실제로 지나는지 로그로 확인했는가? 실사용 표본이 0이라면 「모른다」고 적었는가?
- ☐ 제출 시스템의 단계 순서를 먼저 그렸는가? 서명·검증 전에 그 입력(저자명, 원고)이 확정됐는가?
- ☐ 마감 직전에 공식 제출 안내 페이지를 다시 열어 지면 제한과 마감을 확인했는가? 마감이 돌이면 이른 쪽을 따랐는가?
- ☐ 이 세션이 연구 세션이라면 제품 코드를 건드리지 않았는가? 끝나기 전에 기록을 남겼는가?

그렇듯하게 틀리는 것들 — AI 상담 챗봇 현장 백서

제1판 · 2026년 10월

작은 팀이 6주 동안 AI 세무 상담 챗봇을 만들며 남긴 개발 기록에서 교훈 186개를 뽑아,
여섯 직무의 원칙 160개로 다시 묶었다.

자매편: 「조용히 틀리는 것들 — 게임 개발 현장 백서」